

Docker 핸즈온: PostgreSQL로 익히는 컨테이너 운영

데이터 저장, 꺼내기, 수정, 삭제(CRUD)

Docker Desktop과 WSL2로 PostgreSQL 컨테이너를 띄우고 운영하기 (Windows 11 기준)

Pro

Docker 27

win 환경설정 -> docker 설치 -> postgresql 설치 -> dbeaver설치 -> (db - post) 연동

목차

1장. 컨테이너란 무엇인가: 왜 Docker로 PostgreSQL을 운영하나 [입문]

- 컨테이너(container)와 가상 머신(virtual machine)의 차이
- 이미지(image)와 컨테이너(container)의 관계
- 왜 PostgreSQL을 Docker로 운영하나: 네이티브 설치 vs 컨테이너
- 이 책의 실습 지도와 사전 준비

2장. Windows 11에 Docker Desktop 설치하기 [입문]

- ^{확인} 시스템 요건과 WSL2 백엔드 이해하기
- Docker Desktop 설치하기
- 첫 동작 확인: docker run hello-world
- Docker Desktop 대시보드와 버전 확인

3장. 이미지와 컨테이너 기초: 첫 PostgreSQL 컨테이너 띄우기 [입문]

- 이미지 받아오기: docker pull과 태그(tag)
- 공식 postgres 이미지로 첫 컨테이너 띄우기: docker run
- 컨테이너 목록과 상태: docker ps / stop / start
- 컨테이너와 이미지 정리: docker rm / rmi

4장. 포트 매핑과 접속: 호스트에서 컨테이너 PostgreSQL 만나기 [입문]

- 컨테이너 네트워크와 포트 매핑(-p) 원리
- 호스트에서 psql로 컨테이너 PostgreSQL 접속하기 기본 환경
- DBeaver(GUI)로 접속하기
- 포트 충돌 진단과 해결

5장. 데이터 영속성: 컨테이너를 지워도 데이터를 지키기 [입문]

- 컨테이너는 사라진다: 데이터 휘발 함정
- 네임드 볼륨(named volume)으로 데이터 영속화하기
- 바인드 마운트(bind mount)와 비교하기
- 데이터 디렉터리 /var/lib/postgresql/data 이해하기

6장. 환경변수와 초기화: 컨테이너에 PostgreSQL 구성하기 [기초]

- 환경변수로 컨테이너 구성하기: POSTGRES_PASSWORD/USER/DB 비밀번호 등
- 초기화 스크립트: /docker-entrypoint-initdb.d
- "초기화는 최초 1회만" 함정과 볼륨의 관계
- .env 파일과 비밀번호 다루기 첫걸음

7장. 컨테이너 운영 명령: 안으로 들어가 다루고 살펴보기 [기초]

- docker exec로 컨테이너 안 psql 사용하기
- 컨테이너 셸 진입과 내부 둘러보기
- 로그 확인: docker logs
- 상태·자원 점검: docker inspect / stats / 헬스체크

8장. docker compose 기초: 한 파일로 PostgreSQL 선언하기 [기초]

- 왜 compose인가: 길어지는 명령과 재현성 문제
- docker-compose.yml로 PostgreSQL 선언하기
- docker compose up / down
- 볼륨·환경변수·env를 compose로 통합하기
- 팀 공유와 재현성: compose 파일을 git에 올리기 수업은 여기까지

9장. 백업·복원·업그레이드: 데이터를 안전하게 옮기기 [기초]

- 왜 백업이 필요한가: 볼륨만으로 충분치 않은 이유
- pg_dump로 백업하기
- 복원하기: 덤프로 데이터 되돌리기
- 메이저 버전 업그레이드: dump/restore가 필요한 이유

10장. 실무 패턴과 마무리: 안전 운영과 다음 단계 [기초]

- 시크릿과 .env: 비밀번호를 안전하게 다루기
 - app + db 다중 컨테이너와 사용자 정의 네트워크 맛보기
 - 정리와 디스크 관리: docker system prune
 - 마무리: 전체 운영 흐름 복습과 다음 단계
-

컨테이너란 무엇인가: 왜 Docker로 PostgreSQL을 운영하나

학습 목표 - 컨테이너(container)와 가상 머신(virtual machine)의 구조 차이를 그림으로 설명할 수 있습니다. - 이미지(image)와 컨테이너의 관계를 설계도와 실행 실체로 구분해 설명할 수 있습니다. - PostgreSQL을 네이티브로 설치할 때와 Docker로 운영할 때의 장단점을 비교할 수 있습니다. - 이 책의 실습 흐름을 이해하고 공식 postgres 이미지 페이지에서 태그(버전)를 확인할 수 있습니다.

App실행

이 책은 PostgreSQL이라는 데이터베이스를 **Docker(도커) 컨테이너로 띄워 보고**, 접속하고, 데이터를 안전하게 지키고, 백업까지 해 보는 실습 위주의 교재입니다. 첫 장인 이 장에서는 아직 직접 명령을 입력하지는 않습니다. 대신 "컨테이너가 무엇이고, 왜 데이터베이스를 컨테이너로 운영하면 편한지"라는 큰 그림을 먼저 잡습니다.

여기서 다루는 명령과 화면은 모두 **미리 보기**입니다. 실제 설치와 실행은 2장부터 Windows 11의 **PowerShell(파워셸, 글자로 명령을 입력하는 검은 창)**과 **Docker Desktop(도커 데스크톱)**으로 직접 해 봅니다. 이 장에서는 "이런 모습이구나" 정도로 눈에 익히면 충분합니다.

이 장에서 보여 주는 명령 블록은 개념 이해를 돕기 위한 예시이며, 지금 따라 입력할 필요는 없습니다. 손으로 실행하는 단계는 2장 이후에 안내합니다.

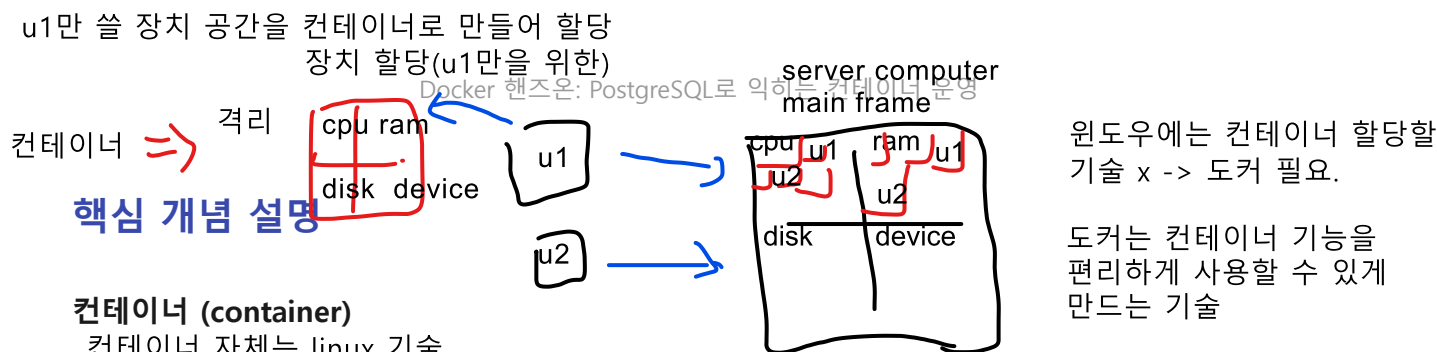
Linux

컨테이너(container)와 가상 머신(virtual machine)의 차이

기술

도입

새 프로그램을 내 PC에 깔았다가 설정이 꼬여 컴퓨터가 지저분해진 경험은 누구나 한 번쯤 있습니다. **컨테이너**는 프로그램과 그 실행에 필요한 것들을 하나의 격리된 상자에 담아, 내 PC를 더럽히지 않고 깔끔하게 실행하는 기술입니다. 이 절에서는 컨테이너를 그보다 먼저 등장한 **가상 머신**과 나란히 놓고, 무엇이 더 가볍고 빠른지 그림으로 비교합니다.



컨테이너(container)란, 애플리케이션과 그 실행에 필요한 것들(라이브러리, 설정 등)을 호스트(내 PC)와 격리된 하나의 단위로 묶어 가볍게 실행하는 기술이자, 그렇게 실행된 단위 그 자체입니다.

비유하자면 **규격화된 화물 컨테이너**와 같습니다. 배에 싣든 트럭에 싣든 컨테이너 안의 내용물은 그대로 보존되듯, 어느 컴퓨터에서 실행하든 컨테이너 안의 프로그램은 같은 환경으로 돌아갑니다.

- **왜(why) 쓰나:** 프로그램마다 필요한 환경이 제각각이라 직접 설치하면 충돌이 잦습니다. 컨테이너는 각 프로그램을 자기만의 상자에 담아 서로 간섭하지 않게 합니다.
- **언제(when) 쓰나:** 데이터베이스처럼 설정이 까다로운 프로그램을 깔끔하게 띄우고, 다 쓰면 흔적 없이 지우고 싶을 때 특히 유용합니다.
- **어떻게(how) 동작하나:** 컨테이너는 호스트의 운영체제(OS) 커널을 함께 쓰면서, 그 위에서 격리된 공간만 따로 만듭니다. 그래서 운영체제를 통째로 또 올리지 않아 가볍습니다.

여기서 입문자가 자주 하는 오해가 있습니다. "컨테이너도 가상 머신처럼 운영체제를 통째로 품고 있다"는 생각입니다. 그렇지 않습니다. 컨테이너는 운영체제 전체가 아니라 **프로그램과 꼭 필요한 부속만** 담습니다. 또 하나, "컨테이너 안에서 바꾼 내용은 항상 영구히 남는다"는 오해도 흔한데, 이는 5장에서 데이터 영속성(data persistence)을 다룰 때 바로잡습니다.

가상 머신 (virtual machine)

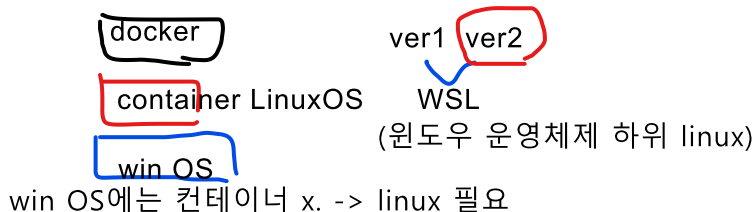
가상 머신(virtual machine)이란, 하이퍼바이저(hypervisor)라는 가상화 소프트웨어 위에서 게스트 운영체제를 통째로 구동해 물리 컴퓨터 한 대를 통째로 흉내 내는 방식입니다.

비유하자면 **집 안에 작은 집을 통째로 또 짓는 것**과 같습니다. 벽부터 지붕까지 완전히 갖추므로 튼튼하게 격리되지만, 그만큼 무겁고 켜는 데 시간이 오래 걸립니다.

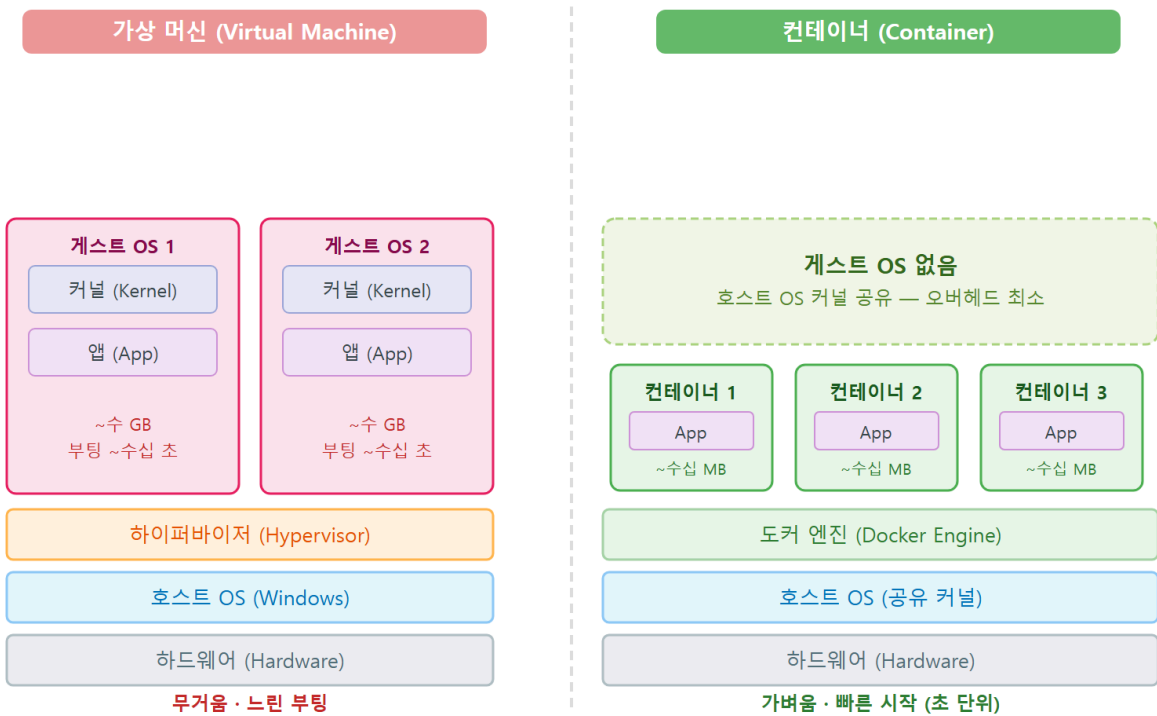
가상 머신은 게스트 운영체제 전체(예: 리눅스 한 벌)를 메모리에 올립니다. 반면 컨테이너는 호스트 운영체제를 빌려 쓰므로, 같은 일을 훨씬 적은 자원으로 더 빨리 시작합니다. 다만 "컨테이너가 가상 머신보다 모든 면에서 항상 낫다"고 단정하면 곤란합니다. 강한 격리가 필요한 경우에는 가상 머신이 더 적합할 때도 있습니다.

컨테이너 기술 -> 컴퓨터

↓
유저에게 빌려줌



가상 머신 vs 컨테이너 구조 비교



이렇게 동작합니다

win 로컬 직접 설치: 과정 복잡, 삭제 시 쓰레기 존재
docker 이용 설치: 설치 간단, 삭제 간단

다음은 PostgreSQL 컨테이너 하나를 띄우는 명령의 **미리 보기**입니다. 명령 한 줄이면 데이터베이스가 몇 초 만에 준비된다는 점에 주목합니다(직접 실행은 3장).

```
PowerShell 도커 실행
PS C:\Users\hyun> docker run -d --name pg postgres:16
PS C:\Users\hyun> docker ps
```

컨테이너 생성 컨테이너 이름 이미지

예상 화면: 백그라운드실행

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
a1b2c3d4e5f6	postgres:16	"docker-entrypoint.s..."	Up 3 seconds	5432/tcp	pg

STATUS 칸의 Up 3 seconds 가 보여 주듯, 컨테이너는 거의 즉시 실행됩니다. 가상 머신으로 운영 체제를 통째로 부팅해 그 안에 데이터베이스를 깔았다면 수 분이 걸렸을 일입니다.

명령·출력 항목 설명

항목	의미
<code>docker run -d</code>	이미지로 새 컨테이너를 만들어 백그라운드(-d)로 실행합니다
<code>--name pg</code>	컨테이너에 <code>pg</code> 라는 이름을 붙입니다
<code>postgres:16</code>	사용할 이미지 이름과 버전(태그)입니다
A <code>docker ps</code>	지금 실행 중인 컨테이너 목록을 보여 줍니다
<code>Up 3 seconds</code>	컨테이너가 3초 전부터 실행 중이라는 상태 표시입니다

팁: 컨테이너가 가볍다는 말은 "운영체제를 새로 안 올린다"는 뜻으로 이해하면 쉽습니다. 그래서 같은 PC에서 PostgreSQL 컨테이너를 여러 개 띄워 버전별로 테스트하는 일도 부담이 적습니다.

절 요약

- 컨테이너는 프로그램을 격리된 상자에 담아 내 PC를 더럽히지 않고 가볍게 실행합니다.
- 가상 머신은 운영체제를 통째로 올려 무겁고, 컨테이너는 호스트 커널을 빌려 써 가볍고 빠릅니다.
- 컨테이너는 운영체제 전체가 아니라 프로그램과 꼭 필요한 부속만 담습니다.

예. CD-ROM(ReadOnlyMemory) CD로 프로그램 설치

이미지(image)와 컨테이너(container)의 관계

설치하고 싶은 프로그램(이미지) 찾아 실행(컨테이너 생성)

PostgreSQL → run → PostgreSQL container

도입

앞 절에서 `postgres:16` 이라는 **이미지**로 컨테이너를 띄웠습니다. 그런데 이미지와 컨테이너는 같은 말이 아닙니다. 이 절에서는 둘의 관계를 "설계도와 실제로 조립된 제품"으로 또렷하게 구분합니다. 이 구분을 한 번 잡아 두면 이후 모든 명령이 훨씬 쉽게 이해됩니다.

핵심 개념 설명

이미지 (image)

이미지(image)란, 컨테이너를 만들기 위한 읽기 전용 템플릿으로, 애플리케이션과 그 실행 환경이 여러 레이어(layer, 층)로 담겨 있습니다.

비유하자면 **쿠키를 찍어 내는 틀**과 같습니다. 틀 하나(이미지)로 똑같은 쿠키(컨테이너)를 여러 개 찍어 낼 수 있고, 틀 자체는 쿠키를 찍어도 변하지 않습니다.

- **왜(why) 중요한가:** 이미지가 곧 "어디서나 같은 환경"을 보장하는 원본입니다. 같은 이미지를 쓰면 내 PC에서든 동료의 PC에서든 똑같은 PostgreSQL이 뜹니다.
- **읽기 전용이라는 점:** 이미지는 실행해도 변하지 않습니다. 그래서 "이미지를 실행하면 이미지가 바뀐다"는 생각은 오해입니다. 실행 중 생기는 변화는 컨테이너 쪽에 쌓입니다.

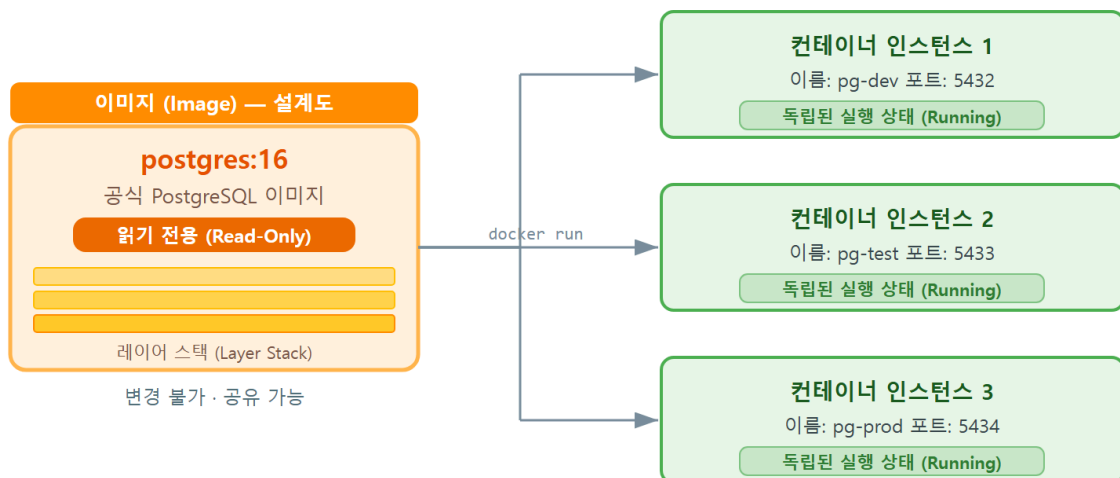
컨테이너 인스턴스 (container instance)

컨테이너 인스턴스(container instance) 란, 하나의 이미지로부터 실제로 만들어져 실행되는 독립된 실체입니다. 같은 이미지에서 찍어 냈더라도 각 컨테이너는 자기만의 실행 상태(메모리, 변경된 파일 등)를 따로 가집니다.

쿠키 틀 비유로 돌아가면, 틀이 이미지라면 그 틀로 찍어 낸 **각각의 쿠키 하나하나**가 컨테이너입니다. 쿠키 하나에 초콜릿을 얹어도 다른 쿠키는 그대로이듯, 한 컨테이너에서 만든 데이터가 다른 컨테이너에 영향을 주지 않습니다.

이미지와 컨테이너를 같은 말로 혼동하기 쉽지만, "설계도(이미지)"와 "조립되어 전원이 켜진 제품(컨테이너)"으로 나누어 생각하면 분명해집니다.

이미지와 컨테이너의 관계



하나의 이미지(설계도)로 여러 컨테이너(실체)를 동시에 실행할 수 있습니다
 이미지는 읽기 전용으로 변경되지 않고, 각 컨테이너는 독립된 상태를 가집니다
 ch_01_s02 · 이미지와 컨테이너의 관계

이렇게 동작합니다

다음은 이미지 하나에서 컨테이너 여러 개가 나오는 모습의 **미리 보기**입니다. 이미지는 한 개인 데(`docker images`), 그 이미지로 만든 컨테이너는 두 개입니다(`docker ps`).

```
PS C:\Users\hyun> docker images   현재 이미지들 확인
PS C:\Users\hyun> docker ps      현재 실행 중인 컨테이너 확인
```

예상 화면:

```
# docker images (설계도: postgres:16 한 개)
REPOSITORY TAG IMAGE ID SIZE
postgres 16 f8e9d0c1b2a3 438MB

# docker ps (실행 실체: 같은 이미지로 만든 컨테이너 두 개)
CONTAINER ID IMAGE NAMES STATUS
a1b2c3d4e5f6 postgres:16 pg1 Up 10 seconds
b2c3d4e5f6a7 postgres:16 pg2 Up 5 seconds
```

이미지 `postgres:16` 은 하나뿐이지만, 그 이미지로 `pg1` 과 `pg2` 라는 서로 다른 컨테이너를 찍어 냈습니다. 두 컨테이너는 같은 설계도에서 나왔어도 각각 독립적으로 돌아갑니다.

명령·출력 항목 설명

항목	의미
<code>docker images</code>	로컬에 내려받은 이미지(설계도) 목록을 보여 줍니다
<code>docker ps</code>	실행 중인 컨테이너(실행 실체) 목록을 보여 줍니다
<code>postgres / 16</code>	이미지 이름과 태그(버전)입니다
<code>pg1 , pg2</code>	같은 이미지에서 찍어 낸 서로 다른 컨테이너 이름입니다
<code>438MB</code>	이미지 한 개가 차지하는 크기입니다

주의: 컨테이너 안에서 만든 데이터는 기본적으로 **그 컨테이너에만** 쌓입니다. 컨테이너를 지우면 그 안의 데이터도 함께 사라집니다. 이미지가 아니라 컨테이너에 변화가 쌓이기 때문입니다. 데이터를 안전하게 지키는 방법은 5장에서 볼륨(volume)으로 자세히 다룹니다.

절 요약

- 이미지는 읽기 전용 설계도, 컨테이너는 그 설계도로 만든 실행 실체입니다.
- 같은 이미지로 여러 컨테이너를 찍어 낼 수 있고, 각 컨테이너는 독립적입니다.

- 실행 중 생기는 변화는 이미지가 아니라 컨테이너에 쌓입니다.

왜 PostgreSQL을 Docker로 운영하나: 네이티브 설치 vs 컨테이너

도입

PostgreSQL을 쓰는 방법은 두 가지입니다. 하나는 설치 프로그램으로 내 PC에 직접 까는 **네이티브(native) 설치**, 다른 하나는 이 책에서 배울 **컨테이너 운영**입니다. 이 절에서는 두 방식을 비교해, 왜 학습과 실습에 컨테이너가 편한지 설명합니다.

핵심 개념 설명

환경 격리 (environment isolation)

환경 격리(environment isolation)란, 프로그램과 그 설정을 호스트(내 PC)와 **분리된 공간에 두어 서로 간섭하지 않게** 하는 것입니다.

PostgreSQL CRUD 교재 1장을 떠올려 봅시다. 거기서는 **EDB 설치 관리자(EDB installer)**라는 설치 프로그램을 내려받아 마법사를 따라가며 PostgreSQL을 Windows에 직접 깔았습니다. 이 방식은 입문용으로 충분했지만, 설치하면서 Windows에 서비스가 등록되고, 레지스트리(시스템 설정 저장소)에 항목이 추가되며, 5432 포트가 호스트에 고정으로 묶입니다. 즉 PostgreSQL이 **내 PC 곳곳에 박히는** 셈입니다.

컨테이너로 운영하면 PostgreSQL이 격리된 상자 안에서만 돌아갑니다. 호스트에는 흔적이 거의 남지 않으므로, 다 쓰면 상자만 버리면 됩니다.

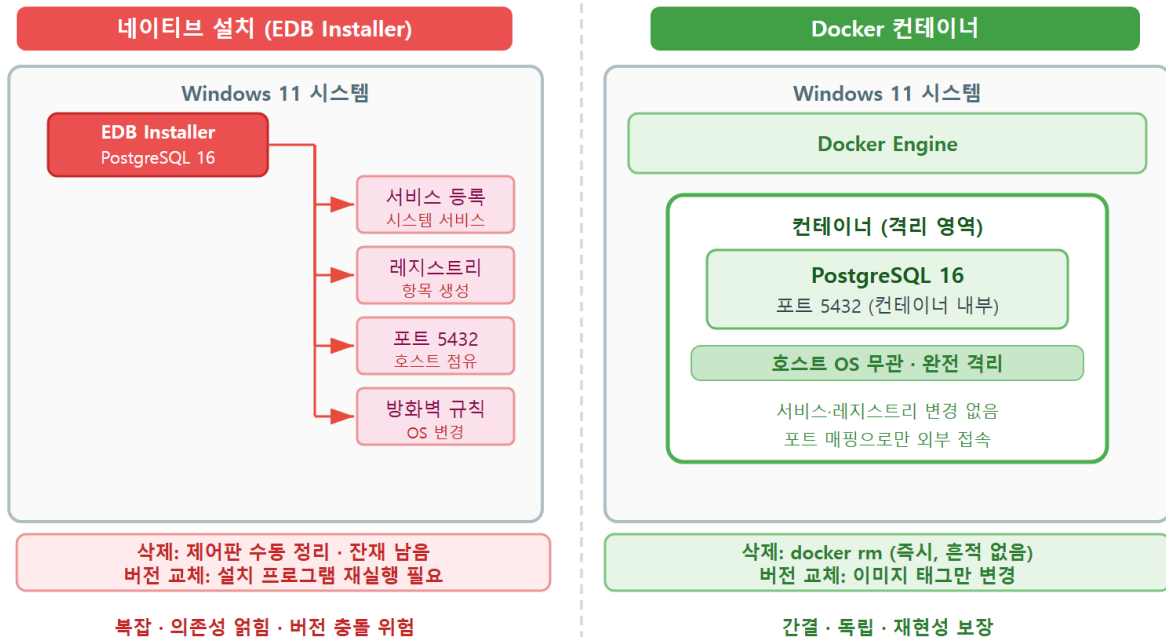
재현성 (reproducibility) 같은 이미지로 여러 개발자 사용하면 같은 환경에서 작업

재현성(reproducibility)이란, 같은 이미지와 구성으로 **어느 컴퓨터에서나 동일한 실행 환경**을 다시 만들어 낼 수 있는 성질입니다.

비유하자면 **같은 레시피**와 같습니다. 레시피(이미지와 구성)만 같으면 누가 어디서 만들어도 같은 요리(PostgreSQL 환경)가 나옵니다. "내 PC에서는 되는데 동료 PC에서는 안 된다"는 단골 문제를, 같은 이미지를 공유하는 것만으로 크게 줄일 수 있습니다.

다만 재현성이 "큰 팀에서만 의미 있다"는 생각은 오해입니다. 혼자 공부할 때도 PC를 바꾸거나 환경을 갈아엎고 다시 만들 일이 잦은데, 컨테이너는 이때 동일 환경을 즉시 되살려 줍니다.

네이티브 설치 vs Docker 컨테이너



ch_01_s03 · 네이티브 설치 vs Docker 컨테이너

이렇게 비교됩니다

다음은 같은 PostgreSQL을 두 방식으로 다룰 때의 **미리 보기**입니다. 컨테이너 방식은 띄우기도, 지우기도 명령 한 줄입니다.

```
# 컨테이너로 PostgreSQL 16 띄우기 (3장에서 직접 실행)
PS C:\Users\hyun> docker run -d --name pg postgres:16

# 다 썼으면 깨끗이 제거 (호스트에 흔적 거의 없음)
PS C:\Users\hyun> docker rm -f pg
```

예상 화면:

```
a1b2c3d4e5f6... # run: 컨테이너가 생성·실행됨
pg              # rm: 컨테이너 pg가 제거됨
```

네이티브 설치에서는 같은 일을 하려면 제어판에서 프로그램을 제거하고, 남은 서비스와 데이터 폴더, 레지스트리 항목까지 손봐야 할 수 있습니다. 컨테이너는 `docker rm -f` 로 상자째 지우면 끝입니다.

네이티브 설치 vs 컨테이너 비교

항목	네이티브 설치(EDB 설치 관리자)	Docker 컨테이너
설치	마법사로 단계별 설치	이미지 받아 <code>docker run</code> 한 줄
호스트 영향	서비스·레지스트리·포트가 PC에 박힘	상자 안에 격리, 흔적 적음
여러 버전 동시 사용	충돌하기 쉬움	태그만 바꿔 나란히 실행
제거	제거 후 잔여물 정리 필요	<code>docker rm</code> 으로 상자째 삭제
동일 환경 재현	수동 재설치·설정	같은 이미지로 즉시 재현

베스트 프랙티스: 학습과 실습 환경은 "언제든 지우고 다시 만들 수 있는" 상태로 두는 것이 좋습니다. 컨테이너는 이 원칙에 잘 맞아, 실수로 설정이 꼬여도 컨테이너를 지우고 다시 띄우면 깨끗한 상태로 돌아갑니다.

주의: 컨테이너를 지우면 호스트에 흔적이 안 남는다는 말은, 뒤집어 보면 **컨테이너 안의 데이터도 함께 사라진다는** 뜻입니다. 실습 데이터를 계속 보존하려면 5장의 볼륨을 반드시 함께 써야 합니다. 지금은 "지우기 쉽다 = 데이터도 쉽게 날아간다"는 양면을 기억해 둡니다.

절 요약

- 네이티브 설치(EDB 설치 관리자)는 PostgreSQL을 호스트 곳곳에 박지만, 컨테이너는 격리된 상자에 둡니다.
- 컨테이너는 버전 분리, 간편한 삭제, 동일 환경 재현에서 학습·실습에 유리합니다.
- 단, 컨테이너를 지우면 데이터도 사라지므로 영속성(5장)을 함께 챙겨야 합니다.

이 책의 실습 지도와 사전 준비

도입

큰 그림을 잡았으니, 이제 이 책이 어디로 향하는지 지도를 펼쳐 봅니다. 이 절에서는 앞으로의 학습 흐름을 미리 보고, 첫 실습으로 Docker Hub의 공식 **postgres** 이미지 페이지를 둘러보며 우리가 쓸 버전(태그)을 정합니다.

핵심 개념 설명

실습 흐름 (hands-on flow)

실습 흐름(hands-on flow) 이란, 이 책이 따라가는 학습 순서입니다. PostgreSQL 컨테이너를 **띄우고(run) → 접속하고(connect) → 데이터를 영속하고(persist) → 백업하는(backup)** 한 줄기로 이어집니다.

큰 흐름을 미리 알면 각 장이 전체에서 어디쯤인지 보여 길을 잃지 않습니다. 단계별로 정리하면 다음과 같습니다.

이 책의 학습 지도

단계	무엇을	다루는 장
준비	Docker Desktop 설치·동작 확인	ch_02
띄우기	이미지 받기·첫 컨테이너 실행	ch_03
접속	포트 매핑·psql·DBeaver로 접속	ch_04
영속	볼륨으로 데이터 지키기	ch_05
구성	환경변수·초기화 스크립트	ch_06
운영	exec·logs·inspect로 다루기	ch_07
선언	docker compose로 한 파일 관리	ch_08
백업	pg_dump 백업·복원·업그레이드	ch_09
마무리	실무 패턴과 정리	ch_10

공식 이미지 (official image)

공식 이미지(official image) 란, **Docker Hub**(도커 허브, 이미지를 올리고 내려받는 공개 저장소)에서 검증해 제공하는 신뢰할 수 있는 기본 이미지입니다. **postgres** 처럼 널리 쓰이는 소프트

웨어가 공식으로 배포됩니다.

비유하자면 **제조사가 직접 내놓은 정품 부품**과 같습니다. 출처가 분명하고 사용법이 문서로 잘 정리되어 있어, 입문 단계에서는 공식 이미지를 쓰는 것이 안전합니다. 다만 "Docker Hub의 모든 이미지가 공식"이라는 생각은 오해입니다. 공식 이미지에는 별도의 표시가 있으며, 그 외에는 개인·단체가 올린 것입니다.

태그(버전)는 `postgres:16`, `postgres:17` 처럼 이름 뒤에 붙습니다. 태그를 생략한 `postgres` 는 `postgres:latest` 를 뜻하는데, `latest` 가 늘 안정 최신판을 보장하지는 않으므로 학습 환경에서는 `postgres:16` 처럼 **버전을 명시**하는 습관이 좋습니다.

이렇게 확인합니다

다음은 공식 이미지를 받기 전에 버전을 확인하는 흐름의 **미리 보기**입니다. 명령으로도 받을 수 있지만, 어떤 태그가 있는지는 Docker Hub의 `postgres` 이미지 페이지에서 눈으로 보는 것이 가장 편합니다.

```
# 공식 postgres 이미지의 16 버전 받기 (3장에서 직접 실행)
PS C:\Users\hyun> docker pull postgres:16          이미지 가져와라
```

예상 화면:

```
16: Pulling from library/postgres
Digest: sha256:...
Status: Downloaded newer image for postgres:16
docker.io/library/postgres:16
```

출력의 `library/postgres` 에서 `library` 는 Docker Hub가 공식 이미지를 모아 두는 공간을 가리킵니다. 즉 우리가 받은 것이 공식 `postgres` 이미지라는 표시입니다.

명령·출력 항목 설명

항목	의미
<code>docker pull postgres:16</code>	공식 postgres 이미지의 16 태그를 내려받습니다
<code>library/postgres</code>	Docker Hub의 공식 이미지 공간에 있는 postgres입니다
<code>Downloaded newer image</code>	새 이미지를 내려받아 로컬에 저장했다는 뜻입니다
<code>docker.io/...</code>	이미지를 받아 온 레지스트리 주소입니다

팁: Docker Hub의 `postgres` 이미지 페이지에는 사용 가능한 태그 목록과 함께

`POSTGRES_PASSWORD` 같은 환경변수 설명, 볼륨 경로 안내가 정리되어 있습니다. 앞으로 자주 들여다보게 되는 문서이니, 즐겨찾기에 추가해 두면 좋습니다.

절 요약

- 이 책은 띄우기 → 접속 → 영속 → 백업으로 이어지는 한 줄기 흐름으로 진행됩니다.
- 공식 이미지는 출처가 분명한 정품 부품과 같아 입문에 안전합니다.
- 태그로 버전을 명시(`postgres:16`)하는 습관이 `latest` 보다 안전합니다.

실습 과제

해보기: 공식 `postgres` 이미지 정찰하고 학습 계획 세우기

이 장은 개념 위주이므로, 실습도 "조사하고 계획표를 만드는" 과제입니다. 명령 실행은 다음 장부터이니, 지금은 자료를 살펴보고 한 장짜리 표로 정리합니다.

- 단계 1: 웹 브라우저에서 Docker Hub의 공식 `postgres` 이미지 페이지를 엽니다(검색창에 `postgres official image` 로 검색). `OFFICIAL IMAGE` 표시가 있는지 확인합니다.
- 단계 2: 페이지의 태그(Tags) 목록에서 제공되는 버전을 살펴보고, 이 책에서 띄울 **목표 버전을 하나 정합니다**(권장: 16). 함께 적힌 `POSTGRES_PASSWORD`, `POSTGRES_USER`, `POSTGRES_DB` 환경변수 설명도 훑어봅니다.
- 단계 3: PostgreSQL CRUD 교재 1장에서 EDB 설치 관리자로 설치하던 방식과, 이 책의 컨테이너 방식을 비교해 한 장짜리 계획표로 정리합니다. 표에는 "설치 방법 / 호스트 영향 / 버전 / 삭제 방법 / 재현 방법" 칸을 둡니다.
- 점검: 계획표에 목표 이미지 태그(예: `postgres:16`)와, 그 버전을 고른 이유 한 줄이 적혀 있는지 확인합니다.

FAQ

Q1. Docker를 쓰면 PostgreSQL CRUD 교재에서 배운 SQL 지식은 못 쓰게 되나요? → A1. 전혀 그렇지 않습니다. Docker는 PostgreSQL을 **답아 실행하는 방식만** 바꿀 뿐, 그 안의 PostgreSQL은 똑같습니다. `INSERT`, `SELECT` 같은 SQL과 `psql` 사용법은 그대로 통하며, 4장에서 컨테이너 PostgreSQL에 접속해 그 SQL을 다시 써 봅니다.

Q2. 컨테이너와 가상 머신 중 무엇을 배워야 하나요? → A2. 이 책의 목적인 "데이터베이스를 가볍게 띄우고 깔끔하게 운영하기"에는 컨테이너가 적합합니다. 가상 머신은 운영체제를 통째로 격리해야 하는 다른 상황에서 쓰이며, 둘은 경쟁 관계라기보다 쓰임이 다른 도구입니다.

Q3. 이미지와 컨테이너가 자꾸 헷갈립니다. 한마디로 정리하면요? → A3. **이미지는 설계도(또는 쿠키 틀), 컨테이너는 그 설계도로 만든 실제 제품(찍어 낸 쿠키)** 입니다. 이미지는 변하지 않고, 실행하면서 생기는 변화는 컨테이너 쪽에 쌓입니다. 그래서 같은 이미지로 컨테이너를 여러 개 만들어도 서로 독립적입니다.

Q4. 이 장의 명령을 지금 따라 입력해야 하나요? → A4. 아닙니다. 이 장의 명령과 화면은 개념을 돕는 미리 보기입니다. Docker Desktop 설치(2장)를 마친 뒤부터 직접 실행하므로, 지금은 "이렇게 생겼구나" 정도로 눈에 익히면 충분합니다.

장 요약

이 장에서는 본격적인 실습에 앞서 큰 그림을 잡았습니다. 컨테이너는 프로그램을 격리된 상자에 담아 가볍게 실행하는 기술로, 운영체제를 통째로 올리는 무거운 가상 머신과 다릅니다. 이미지는 읽기 전용 설계도이고 컨테이너는 그 설계도로 찍어 낸 실행 실체이며, 같은 이미지로 여러 컨테이너를 독립적으로 만들 수 있습니다. PostgreSQL을 Docker로 운영하면 호스트를 더럽히지 않는 환경 격리, 같은 이미지로 어디서나 같은 환경을 만드는 재현성, 간편한 버전 분리와 삭제라는 이점을 얻습니다. 끝으로 이 책의 학습 지도(띄우기 → 접속 → 영속 → 백업)를 미리 보고, 공식 postgres 이미지에서 버전(태그)을 확인하는 첫 실습 과제를 다뤘습니다.

핵심 키워드

컨테이너(container), 가상 머신(virtual machine), 이미지(image), 컨테이너 인스턴스(container instance), 환경 격리(environment isolation), 재현성(reproducibility), 공식 이미지(official image), 이미지 태그(image tag)

다음 장 미리보기

다음 장에서는 Windows 11에 **Docker Desktop**을 직접 설치합니다. Docker가 WSL2(경량 리눅스) 위에서 동작하는 구조를 이해하고, 시스템 요건을 점검한 뒤, `docker run hello-world` 로 설치가 정상인지 확인하는 첫 실행까지 진행합니다.

Windows 11에 Docker Desktop 설치하기

학습 목표 - Windows 11에서 Docker가 WSL2 백엔드 위에서 동작하는 구조를 설명할 수 있습니다. - 시스템 요건을 점검하고 Docker Desktop을 설치할 수 있습니다. - docker run hello-world로 설치가 정상인지 확인할 수 있습니다. - docker version-info와 대시보드로 엔진 상태를 점검할 수 있습니다.

1장에서 우리는 "컨테이너가 무엇이고, 왜 PostgreSQL을 Docker로 운영하면 편한가"를 그림으로 살펴봤습니다. 머리로는 이해했지만, 아직 내 PC에는 컨테이너를 띄울 도구가 하나도 없습니다. 이 장은 그 도구를 손에 쥐는 단계입니다.

요리에 비유하면, 1장이 "왜 이 주방 도구가 좋은가"를 설명한 장이라면, 이 장은 **그 도구를 실제로 사서 주방에 설치하고 전원을 켜 보는 장**입니다. 도구가 제대로 켜지는지 확인하는 첫 시운전(docker run hello-world)까지 마치면, 다음 장부터는 본격적으로 PostgreSQL 컨테이너를 다룰 수 있습니다.

이 책은 **Windows 11**과 **PowerShell(파워셸)**을 기준으로 합니다. PowerShell은 **글자로 명령을 입력하는 검은 창**(터미널)으로, 시작 메뉴에서 "PowerShell"을 검색해 열 수 있습니다. 화면 앞에 보이는 **PS C:\Users\사용자이름>** 표시는 "지금 PowerShell이 내 사용자 폴더에서 명령을 기다리는 중"이라는 안내입니다. 이 책에서 ````powershell` 이라고 표시한 블록은 여러분이 그 창에 직접 입력하는 명령이고, ````text`로 표시한 블록은 그 결과로 화면에 나타나는 모습입니다.

시스템 요건과 WSL2 백엔드 이해하기

도입

 컨테이너 기술을 쉽게 사용하게 해주는 프로그램

Docker Desktop을 설치하기 전에, 먼저 "Windows에서 Docker가 도대체 어떻게 도는가"를 한번 그려 두면 설치 과정이 훨씬 덜 헛갈립니다. **컨테이너는 본래 리눅스(Linux) 기술**인데, 우리가 쓰는 운영체제는 Windows이기 때문입니다. 이 절에서는 Windows 안에서 컨테이너가 도는 구조와, 그 핵심인 **WSL2 백엔드**를 살펴보고, 내 PC가 설치 요건을 갖췄는지 점검합니다.

핵심 개념 설명

Docker Desktop

Docker Desktop(도커 데스크톱)이란, Windows나 macOS에서 컨테이너를 다루는 데 필요한 것들을 하나로 묶어 주는 설치형 프로그램입니다. 컨테이너를 실제로 돌리는 **도커 엔진(Docker Engine)**, 명령을 입력하는 도구(**docker 명령**), 마우스로 컨테이너를 들여다보는 **대시보드(dashboard, 그래픽 화면)**, 그리고 Windows에서 리눅스 환경을 마련해 주는 연결 장치까지 한 묶음으로 들어 있습니다.

쉽게 말하면, 컨테이너를 다루기 위한 도구 세트를 낱개로 사 모을 필요 없이 "정품 종합 세트 한 박스"로 받는 것과 같습니다. 입문자가 직접 엔진과 부속을 따로 설치하려면 복잡하지만, Docker Desktop은 그 과정을 설치 마법사 한 번으로 끝내 줍니다.

여기서 입문자가 흔히 오해하는 점이 있습니다. "Docker Desktop = 도커 전부"라고 생각하기 쉽지만, 정확히는 Docker Desktop은 도커 엔진을 **감싸서 Windows에서 편하게 쓰게 해 주는 포장**입니다. 실제로 컨테이너를 돌리는 일꾼은 그 안의 도커 엔진이며, 그 엔진은 다시 WSL2라는 리눅스 환경 위에서 동작합니다.

WSL2 백엔드 (WSL2 backend)

WSL2 백엔드(WSL2 backend)란, Windows에서 Docker Desktop이 컨테이너를 실제로 구동하기 위해 사용하는 **경량 리눅스 실행 환경**입니다. WSL은 "Windows Subsystem for Linux"의 줄임말로, 우리말로 풀면 "**윈도우 안에서 리눅스를 돌리는 장치**"이고, 2는 그 두 번째 버전을 뜻합니다.

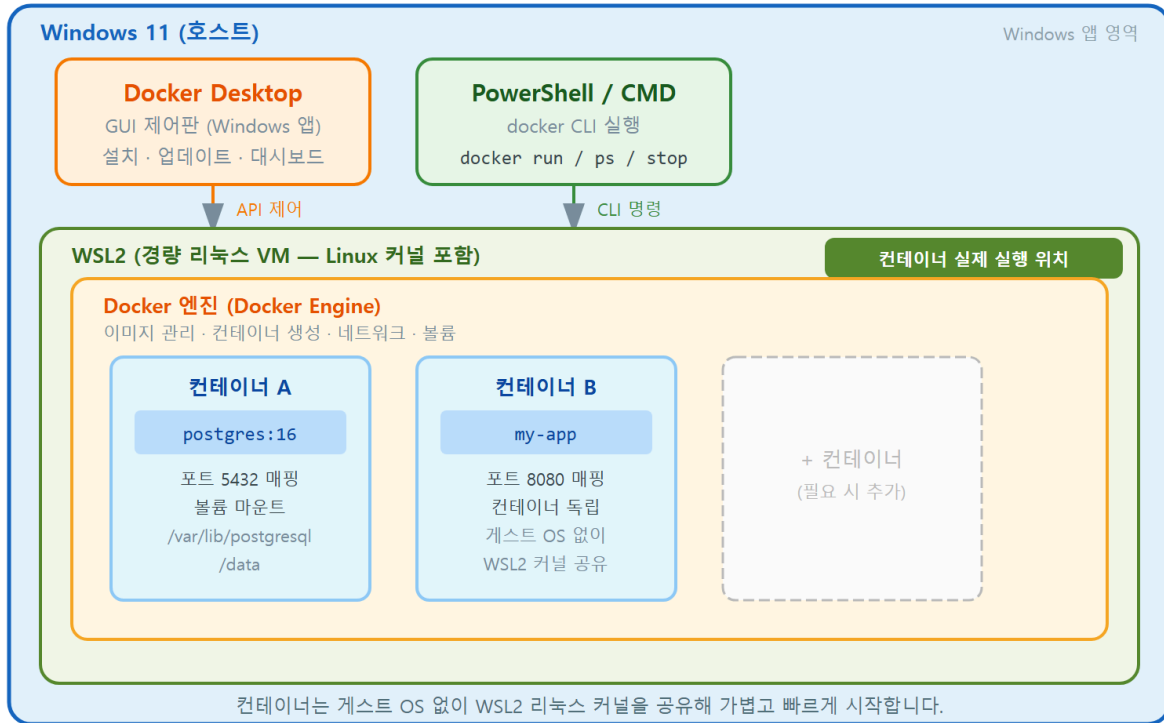
비유하자면, Windows라는 큰 건물 안에 **작은 리눅스 작업실**을 한 칸 마련해 둔 것과 같습니다. 우리가 PowerShell에서 `docker` 명령을 내리면, 컨테이너는 Windows 바탕화면 위에서 곧바로 도는 것이 아니라 이 리눅스 작업실 안에서 돌아갑니다. PostgreSQL 컨테이너도 마찬가지로 이 작업실 안에서 실행됩니다.

- **왜(why) 필요한가:** 컨테이너는 리눅스 커널(운영체제의 핵심)에 기대어 동작하는 기술이라, 리눅스 환경이 있어야 돌아갑니다. WSL2가 그 리눅스 환경을 가볍고 빠르게 제공합니다.
- **언제(when) 등장하나:** Docker Desktop을 설치할 때 "WSL2 백엔드 사용"이라는 선택지가 나오며, 컨테이너를 띄울 때마다 그 뒤에서 늘 동작합니다.
- **어떻게(how) 연결되나:** Docker Desktop의 대시보드(그래픽 화면)가 위에 있고, 그 아래 도커 엔진이, 다시 그 아래 WSL2 리눅스가, 맨 아래 Windows 11이 있는 층층 구조입니다.

입문자가 자주 하는 오해 하나를 짚겠습니다. "컨테이너가 Windows 위에서 곧바로 돈다"고 생각하는 경우인데, 실제로는 WSL2 리눅스를 한 단계 거칩니다. 또 "WSL2 없이도 Docker

Desktop이 똑같이 동작한다"고 여기기도 하지만, 최신 Docker Desktop은 이 WSL2 백엔드를 기본 방식으로 삼습니다.

Windows 11 에서 Docker 가 도는 구조: WSL2 계층 아키텍처



ch_02_s01 · 시스템 요건과 WSL2 백엔드 이해하기

다음은 지금까지 설명한 층층 구조를 글로 정리한 것입니다. 위에서 아래로 갈수록 더 아래쪽(바탕)에 있는 요소입니다.

```
[ Docker Desktop 대시보드 ] <- 마우스로 보는 그래픽 화면
|
[ 도커 엔진(Docker Engine) ] <- 컨테이너를 실제로 돌리는 일꾼
|
[ WSL2 (경량 리눅스) ] <- 엔진이 올라앉는 리눅스 작업실
|
[ Windows 11 (내 PC) ] <- 모든 것이 올라가는 바탕
```

구조 계층 설명

계층	역할
Docker Desktop 대시보드	컨테이너·이미지·볼륨을 마우스로 보고 다루는 화면입니다
도커 엔진	이미지를 받아 컨테이너로 실행하는 핵심 프로그램입니다
WSL2 (경량 리눅스)	엔진이 동작할 리눅스 환경을 Windows 안에 제공합니다
Windows 11	위 모든 것이 올라가는 우리 PC의 운영체제입니다

시스템 요건 점검하기

Docker Desktop을 설치하려면 내 PC가 몇 가지 조건을 갖춰야 합니다. 다음 명령으로 Windows 버전을 먼저 확인합니다.

win + R

```
winver
```

예상 화면:

(작은 정보 창이 뜨며 다음과 같은 내용이 표시됩니다)

Windows 11

버전 23H2 (OS 빌드 22631.xxxx)

`winver` 는 Windows 버전을 보여 주는 명령입니다. Windows 11이면 요건을 충분히 만족합니다. 이어서 WSL이 설치되어 있는지, 어떤 버전인지 확인합니다.

```
wsl --status
```

예상 화면:

기본 배포: (없음 또는 설치된 배포 이름)

기본 버전: 2

기본 버전: 2 로 표시되면 WSL2를 쓸 준비가 된 것입니다. 만약 `wsl` 명령 자체를 알 수 없다는 오류가 나면, 아직 WSL이 설치되지 않은 상태입니다. 이 경우는 다음 절의 설치 과정에서 함께 해결되므로 지금은 걱정하지 않아도 됩니다.

시스템 요건 점검 명령

명령	확인하는 것
<code>winver</code>	Windows 버전(11이면 권장 요건 충족)
<code>wsl --status</code>	WSL 설치 여부와 기본 버전(2여야 함)
<code>wsl --version</code>	WSL 세부 버전 정보(설치되어 있을 때)

이 밖에 권장되는 하드웨어 조건은 다음과 같습니다. 메모리(RAM) 4GB 이상(8GB 이상이면 더 여유롭습니다), 64비트 프로세서, 그리고 **가상화 기능(virtualization) 켜짐**입니다. 가상화 기능은 WSL2가 리눅스 작업실을 만들 때 쓰는 CPU 기능으로, 보통 최신 PC에서는 기본으로 켜져 있습니다.

가상화가 켜졌는지는 작업 관리자에서 빠르게 확인할 수 있습니다. `Ctrl + Shift + Esc` 로 작업 관리자를 연 뒤 "성능" 탭의 CPU 항목을 보면 "가상화: 사용"이라고 표시됩니다.

주의: "가상화: 사용 안 함"으로 나오면 WSL2가 동작하지 않아 Docker Desktop이 켜지지 않습니다. 이 경우 PC를 켤 때 BIOS/UEFI(메인보드 기본 설정 화면)에 들어가 가상화 기능(인텔은 VT-x, AMD는 SVM/AMD-V로 표기)을 켜 줘야 합니다. 메인보드 제조사마다 진입 키(보통 F2·Delete)와 메뉴 이름이 다르므로, "메인보드 모델명 + 가상화 활성화"로 찾아보면 정확한 위치를 알 수 있습니다.

팁: 회사·학교에서 지급한 PC는 보안 정책으로 가상화나 WSL 설치가 막혀 있을 수 있습니다. 설치가 자꾸 막히면 PC 관리 담당자에게 "WSL2와 Docker Desktop 사용 권한"을 요청하는 편이 빠릅니다.

절 요약

- Docker Desktop은 도커 엔진·명령 도구·대시보드를 한 묶음으로 설치해 주는 프로그램입니다.
- Windows에서 컨테이너는 Windows 위에서 곧바로 돌지 않고, WSL2(경량 리눅스) 작업실 안에서 동작합니다.
- 구조는 위에서부터 대시보드 → 도커 엔진 → WSL2 → Windows 11 순으로 쌓여 있습니다.
- `winver` · `wsl --status` 로 버전을 확인하고, 작업 관리자에서 가상화가 켜졌는지 점검합니다.

Docker Desktop 설치하기

도입

구조와 요건을 확인했으니, 이제 실제로 Docker Desktop을 내려받아 설치합니다. 설치 자체는 설치 마법사를 따라가면 되는 단순한 과정이지만, "WSL2 백엔드 사용" 같은 핵심 선택지를 놓치지 않는 것이 중요합니다. 이 절에서는 설치 파일을 받는 단계부터 최초 실행, 그리고 PC를 켤 때 자동으로 함께 켜지도록 설정하는 단계까지 안내합니다.

핵심 개념 설명

설치 관리자 (installer)

설치 관리자(installer)란, 프로그램을 내 PC에 올바르게 설치해 주는 실행 파일입니다. Docker Desktop의 설치 관리자는 `Docker Desktop Installer.exe` 라는 이름으로 내려받아집니다. 이 파일을 실행하면 필요한 구성 요소를 알맞은 위치에 풀어 놓고, WSL2 연동까지 자동으로 준비합니다.

내려받기는 Docker 공식 사이트(<https://www.docker.com/products/docker-desktop/>)의 "Download for Windows" 버튼에서 받습니다. **반드시 공식 사이트에서 받습니다.** 검색 결과 상단의 광고나 출처가 불분명한 사이트에서 받은 설치 파일은 변조되었을 위험이 있습니다.

WSL2 활성화

WSL2 활성화란, 앞 절에서 살펴본 WSL2 리눅스 작업실을 실제로 켜서 Docker가 그 위에서 돌도록 만드는 과정입니다. 최신 Docker Desktop 설치 관리자는 설치 중에 "Use WSL 2 instead of Hyper-V"(Hyper-V 대신 WSL 2 사용) 같은 항목을 **체크된 상태로** 보여 주므로, 그대로 두면 됩니다.

이렇게 설치합니다

설치는 마우스로 진행하는 과정이라 PowerShell 명령이 따로 필요하지 않습니다. 다만 설치 전에 WSL을 미리 최신으로 맞춰 두면 설치가 더 매끄럽습니다. 다음 명령을 관리자 권한 PowerShell에서 실행합니다(시작 메뉴에서 PowerShell을 오른쪽 클릭하여 "관리자 권한으로 실행").

```
wsl --update
```


예상 화면:

업데이트를 확인하는 중...
WSL(Linux용 Windows 하위 시스템)의 최신 버전을 다운로드하는 중...
설치 중: WSL(Linux용 Windows 하위 시스템)
WSL(Linux용 Windows 하위 시스템)이(가) 설치되었습니다.

이 명령은 WSL 자체를 최신 버전으로 갱신합니다. 이미 최신이면 "최신 버전입니다"와 비슷한 안내가 나옵니다. 이제 설치 마법사를 다음 순서로 진행합니다.

1. 받은 Docker Desktop Installer.exe 를 더블 클릭한다.
2. "Use WSL 2 instead of Hyper-V" 항목이 체크된 채로 그대로 둔다.
3. "Add shortcut to desktop"(바탕화면 바로 가기) 등은 취향대로 둔다.
4. OK / Install 을 눌러 설치가 끝날 때까지 기다린다.
5. "Close and restart" 가 보이면 눌러 PC를 재시작한다.

설치 중 핵심 선택지

선택지	의미	권장
Use WSL 2 instead of Hyper-V	WSL2 백엔드로 컨테이너를 구동	체크 유지
Add shortcut to desktop	바탕화면 바로 가기 추가	취향대로
Close and restart	설치 마무리 후 PC 재시작	눌러 재시작

재시작 후 시작 메뉴에서 "Docker Desktop"을 실행하면 최초 실행 화면이 나타납니다. 이때 다음을 기억하면 입문 단계에서 막힘이 없습니다.

- 서비스 약관(Accept the terms) 화면 -> 동의에 체크하고 진행한다.
- 로그인/회원가입(Sign in) 화면 -> "Continue without signing in" 또는 건너뛰기를 눌러도 된다(학습에는 로그인이 필요 없다).
- 설문(survey) 화면 -> Skip 으로 건너뛸 수 있다.

잠시 기다리면 Docker Desktop 창의 왼쪽 아래 상태 표시가 초록색이 되며 "Engine running"(엔진 실행 중)으로 바뀝니다. 이 상태가 되어야 PowerShell에서 `docker` 명령이 동작합니다.

팁: Docker Desktop을 매번 직접 켜기 번거롭다면, 설정에서 PC 부팅 시 자동 실행을 켤 수 있습니다. 오른쪽 위 톱니바퀴(Settings) → General(일반) → "Start Docker Desktop when you sign in to your computer"(로그인 시 Docker Desktop 시작)에 체크하면 됩니다. 다만 PC가 켜질 때마다 메모리를 조금 쓰므로, 사양이 낮은 PC라면 필요할 때만 직접 켜는 편이 가법습니다.

주의: 설치 직후 PowerShell에서 `docker` 명령이 "명령을 찾을 수 없다"고 나올 수 있습니다. 이는 보통 두 가지 이유입니다. 첫째, 설치 후 PowerShell 창을 새로 열지 않아 경로 설정이 반영되지 않은 경우(창을 닫고 다시 엽니다). 둘째, Docker Desktop이 아직 완전히 켜지지 않은 경우(상태가 "Engine running" 초록색이 될 때까지 기다립니다).

절 요약

- 설치 파일은 반드시 Docker 공식 사이트에서 받습니다.
- 설치 중 "Use WSL 2 instead of Hyper-V"는 체크된 상태로 둡니다.
- 최초 실행 시 로그인·설문은 건너뛰어도 학습에 지장이 없습니다.
- 왼쪽 아래 상태가 "Engine running" 초록색이 되어야 `docker` 명령이 동작합니다.

첫 동작 확인: docker run hello-world

도입

새 전자제품을 사면 전원을 켜서 정상 작동하는지 한 번 확인합니다. Docker도 마찬가지입니다. 설치가 끝났으면 가장 작고 단순한 컨테이너를 한 번 띄워 봄으로써 "엔진이 이미지를 받아 컨테이너로 실행하는" 전체 흐름이 막힘없이 도는지 확인합니다. 이 시운전에 쓰는 것이 `hello-world` 이미지입니다.

핵심 개념 설명

동작 확인 (smoke test)

동작 확인(smoke test)이란, 복잡한 작업에 들어가기 전에 가장 기본적인 동작 하나가 정상인지 빠르게 점검하는 일입니다. 원래는 새 기계에 전원을 넣었을 때 "연기(smoke)가 나지 않는지" 보던 데서 온 말로, 우리말로 "기본 시운전" 정도로 옮길 수 있습니다.

Docker에서 동작 확인은 `hello-world` 라는 아주 작은 이미지를 컨테이너로 실행해 보는 것입니다. 이 컨테이너는 인사 메시지 한 번을 출력하고 곧바로 종료됩니다. 메시지가 정상적으로 보이면, 엔진이 이미지를 내려받아 컨테이너로 띄우는 핵심 과정이 모두 정상이라는 뜻입니다.

docker run

docker run(도커 런) 이란, 이미지로부터 새 컨테이너를 만들어 실행하는 명령입니다. 1장에서 배운 비유를 빌리면, 이미지가 "설계도"이고 컨테이너가 "실제로 조립해 전원을 켜 제품"인데, `docker run` 이 바로 그 "설계도를 보고 제품을 조립해 켜는" 동작입니다.

`docker run hello-world` 를 실행하면 내부적으로 다음 순서가 자동으로 일어납니다. 이 흐름은 앞으로 PostgreSQL 이미지를 띄울 때도 똑같이 적용되는 핵심 원리이니 눈여겨봅니다.

1. 내 PC에 hello-world 이미지가 있는지 찾는다.
2. 없으면 Docker Hub(공식 이미지 저장소)에서 자동으로 내려받는다(pull).
3. 내려받은 이미지로 컨테이너를 하나 만든다.
4. 컨테이너가 인사 메시지를 출력한다.
5. 할 일을 마쳤으므로 컨테이너는 곧바로 종료된다.

이렇게 실행합니다

PowerShell을 열고(Docker Desktop이 "Engine running" 상태인지 확인) 다음을 입력합니다.

```
docker run hello-world
```

예상 화면:

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Digest: sha256:d000bc569937abbe195e20322a0bde6b2922d805332fd6d8a68b19f524b7d21d
Status: Downloaded newer image for hello-world:latest
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

"Hello from Docker!"가 보이면 설치가 정상입니다. 처음 실행할 때는 이미지를 내려받느라 (Pulling, Pull complete) 몇 초가 걸리고, 두 번째 실행부터는 이미 받아 둔 이미지를 쓰므로 그 줄들이 사라지고 곧바로 인사 메시지가 나옵니다.

출력 줄별 의미

출력 줄	의미
Unable to find image ... locally	내 PC에 이미지가 없어 원격에서 받겠다는 안내입니다
Pulling from library/hello-world	Docker Hub에서 이미지를 내려받기 시작합니다
Pull complete	이미지 한 조각(레이어) 내려받기를 마쳤습니다
Status: Downloaded newer image	이미지 내려받기가 끝났습니다
Hello from Docker!	컨테이너가 정상 실행되어 출력한 인사말입니다
The Docker client contacted the Docker daemon	명령 도구가 엔진에 작업을 요청했다는 설명입니다

출력 안의 "Docker client"(도커 클라이언트)는 우리가 입력한 `docker` 명령 도구이고, "Docker daemon"(도커 데몬)은 뒤에서 늘 켜져 일을 처리하는 엔진을 가리킵니다. 즉 "내가 명령을 내리면(client) → 엔진(daemon)이 이미지를 받아 컨테이너로 실행한다"는 1장의 구조가 이 한 번의 실행에 모두 담겨 있습니다.

흔한 오류 두 가지 - `error during connect ... docker daemon is not running` → 엔진이 아직 안 켜진 상태입니다. Docker Desktop을 실행하고 "Engine running" 초록색이 될 때까지 기다린 뒤 다시 시도합니다. - `docker : 'docker' 용어가 cmdlet ... 이름으로 인식되지 않습니다` → PowerShell이 `docker` 명령을 못 찾는 경우입니다. PowerShell 창을 닫고 새로 열거나, Docker Desktop이 설치·실행되었는지 확인합니다.

팁: `docker run hello-world`로 만들어진 컨테이너는 메시지를 출력하고 종료되지만, 곧바로 사라지지 않고 "정지된 컨테이너"로 목록에 남습니다. 이 정지된 컨테이너를 보고 정리하는 방법(`docker ps -a`, `docker rm`)은 3장에서 컨테이너 생명주기와 함께 자세히 배웁니다. 지금은 "한 번 띄워 보는 것" 자체가 목적이니 그대로 뒀도 됩니다.

절 요약

- `docker run hello-world`는 설치 정상 여부를 확인하는 기본 시운전입니다.

- 이 한 번의 실행에 "이미지 찾기 → 없으면 내려받기 → 컨테이너 실행 → 출력 → 종료" 흐름이 모두 담깁니다.
- "Hello from Docker!"가 보이면 엔진·내려받기·실행이 모두 정상입니다.
- 오류가 나면 대부분 엔진이 안 켜졌거나(`daemon not running`) 명령을 못 찾는(`docker` 인식 안 됨) 경우입니다.

Docker Desktop 대시보드와 버전 확인

도입

시운전까지 마쳤으니, 마지막으로 "내가 설치한 도커가 어떤 상태인지" 들여다보는 방법을 익힙니다. 명령으로 버전과 환경을 확인하는 `docker version`·`docker info`, 그리고 마우스로 컨테이너·이미지·볼륨을 둘러보는 대시보드입니다. 이 두 가지는 앞으로 무언가 막혔을 때 "지금 상태가 어떤가"를 가장 먼저 확인하는 점검 도구가 됩니다.

핵심 개념 설명

`docker version` / `docker info`

`docker version`(도커 버전) 은 도커의 버전 정보를 보여 주는 명령입니다. 중요한 점은 도커가 **클라이언트(명령 도구)** 와 **서버(엔진)** 두 부분으로 나뉘어 있고, `docker version` 이 그 둘의 버전을 따로 보여 준다는 것입니다. 1장과 앞 절에서 본 "명령을 내리는 쪽"과 "실제로 일하는 엔진 쪽"이 여기서 버전으로 구분되어 나타납니다.

`docker info`(도커 인포) 는 한 단계 더 들어가, 지금 컨테이너가 몇 개 떠 있는지, 이미지가 몇 개 있는지, 어떤 운영체제(여기서는 WSL2 리눅스) 위에서 도는지 같은 **환경 전체 요약**을 보여 줍니다. `version` 이 "버전표"라면, `info` 는 "현재 상태 요약표"에 가깝습니다.

대시보드 (dashboard)

대시보드(dashboard) 란, 컨테이너·이미지·볼륨을 명령 없이 마우스로 보고 다룰 수 있는 Docker Desktop의 그래픽 화면입니다. 자동차 계기판처럼, 지금 무엇이 돌아가고 있고 무엇이 멈춰 있는지를 한눈에 보여 줍니다.

명령(`docker ps` 등)으로도 같은 정보를 볼 수 있지만, 입문 단계에서는 대시보드로 전체 그림을 눈으로 먼저 익히면 명령의 결과도 더 잘 이해됩니다. 다만 실무에서는 명령이 더 빠르고 자동화에 유리하므로, 이 책은 명령을 중심에 두고 대시보드를 보조로 사용합니다.

이렇게 확인합니다

먼저 버전을 확인합니다.

```
docker version
```

예상 화면:

```
Client:
  Version:      27.4.0
  API version:  1.47
  Go version:   go1.22.10
  OS/Arch:     windows/amd64
  Context:     desktop-linux

Server: Docker Desktop 4.37.0
Engine:
  Version:      27.4.0
  API version:  1.47 (minimum version 1.24)
  OS/Arch:     linux/amd64
```

Client (클라이언트)는 Windows에서 도는 명령 도구이고, Server (엔진)는 WSL2 리눅스 위에서 도는 부분입니다. Server 쪽 OS/Arch 가 linux/amd64 로 나오는 점에 주목합니다. 우리는 Windows를 쓰지만, 엔진은 앞서 배운 대로 **리눅스 위에서 동작하기** 때문입니다.

docker version 주요 항목

항목	의미
Client Version	명령 도구(도커 클라이언트)의 버전입니다
Server Engine Version	컨테이너를 돌리는 엔진의 버전입니다
Server OS/Arch	엔진이 도는 환경(WSL2이라 linux/amd64)입니다
Context: desktop-linux	현재 Docker Desktop의 리눅스 환경에 연결되어 있다는 표시입니다

이어서 환경 요약을 봅니다.

```
docker info
```

예상 화면:

```
Client:
  Version: 27.4.0
  Context: desktop-linux

Server:
  Containers: 1
    Running: 0
    Paused: 0
    Stopped: 1
  Images: 1
  Server Version: 27.4.0
  Operating System: Docker Desktop
  Total Memory: 7.6GiB
  Name: docker-desktop
```

Containers: 1 과 Stopped: 1 은 앞 절에서 hello-world 를 실행하고 남은 정지 컨테이너 한 개를 가리킵니다. Images: 1 은 그때 내려받은 hello-world 이미지 한 개입니다. 이렇게 `docker info` 는 지금 내 도커 환경에 무엇이 있는지를 숫자로 요약해 줍니다.

docker info 주요 항목

항목	의미
Containers	전체 컨테이너 수(실행+정지)입니다
Running / Stopped	실행 중 / 정지된 컨테이너 수입니다
Images	내려받아 둔 이미지 수입니다
Total Memory	도커 엔진이 쓸 수 있는 메모리 크기입니다
Operating System	엔진이 도는 환경(Docker Desktop = WSL2 리눅스)입니다

마지막으로 대시보드를 둘러봅니다. Docker Desktop 창을 열면 왼쪽에 메뉴가 있습니다.

- Containers : 떠 있거나 정지된 컨테이너 목록(hello-world 가 보인다)
- Images : 내려받은 이미지 목록(hello-world 이미지가 보인다)
- Volumes : 데이터가 저장되는 볼륨 목록(지금은 비어 있을 수 있다)

각 메뉴를 눌러 보면, 방금 `docker info` 로 본 "컨테이너 1개, 이미지 1개"가 화면에 그대로 표시 되는 것을 확인할 수 있습니다. 명령과 대시보드가 같은 상태를 다른 방식으로 보여 주는 것입

니다. Volumes(볼륨) 메뉴는 지금은 비어 있지만, 5장에서 PostgreSQL 데이터를 영속화할 때 다시 찾게 됩니다.

팁: 무언가 잘 안 될 때 가장 먼저 할 일은 `docker version` 으로 `Server` 항목이 정상적으로 보이는지 확인하는 것입니다. `Server` 정보가 나오지 않고 오류만 뜬다면 엔진이 안 켜진 것이므로, 다른 명령을 시도하기 전에 Docker Desktop이 "Engine running" 상태인지부터 점검합니다.

절 요약

- `docker version` 은 클라이언트(명령 도구)와 서버(엔진)의 버전을 나눠 보여 주며, 서버는 리눅스 위에서 돌립니다.
- `docker info` 는 컨테이너·이미지 수와 메모리 등 환경 전체를 요약해 줍니다.
- 대시보드는 같은 정보를 마우스로 둘러보는 그래픽 화면입니다.
- 문제가 생기면 `docker version` 의 `Server` 항목으로 엔진 상태부터 확인합니다.

실습 과제

해보기: 내 PC에 Docker Desktop 설치하고 동작 확인표 작성하기

이 장에서 배운 순서대로 내 Windows 11 PC에 Docker Desktop을 설치하고, 결과를 점검표에 적어 봅니다.

- 단계 1: `winver` 와 `wsl --status` 로 Windows 버전과 WSL 기본 버전을 확인하고, 작업 관리자 "성능" 탭에서 가상화가 "사용"인지 적습니다.
- 단계 2: Docker 공식 사이트에서 설치 파일을 받아 "Use WSL 2 instead of Hyper-V"를 체크한 채 설치하고 재시작합니다. 최초 실행에서 로그인 건너뛰기.
- 단계 3: Docker Desktop 상태가 "Engine running"(초록색)이 되면 PowerShell에서 `docker run hello-world` 를 실행하고, "Hello from Docker!"가 보이는지 확인합니다.
- 단계 4: `docker version` 과 `docker info` 를 실행해 다음 점검표를 채웁니다.

[동작 확인 점검표]

- | | |
|---------------------|-------------------------------------|
| - Windows 버전 | : (winver 결과, 예: Windows 11 23H2) |
| - WSL 기본 버전 | : (wsl --status 결과, 예: 2) |
| - 가상화 사용 여부 | : (작업 관리자, 예: 사용) |
| - hello-world 실행 성공 | : (Hello from Docker! 표시 여부, 예/아니오) |
| - Client 버전 | : (docker version 결과) |
| - Server(엔진) 버전 | : (docker version 결과) |
| - Server OS/Arch | : (예: linux/amd64) |
| - 정지 컨테이너 수 | : (docker info 의 Stopped) |

- 점검: 대시보드의 Containers-Images 메뉴를 열어, `docker info` 에서 본 컨테이너 1개·이미지 1개가 화면에도 똑같이 보이는지 눈으로 대조합니다.

FAQ

Q1. Docker Desktop을 설치했는데 "WSL 2 installation is incomplete"라는 오류가 떠요. 어떻게 하나요? → A1. WSL2의 리눅스 커널 구성 요소가 최신이 아닐 때 나오는 메시지입니다. 관리자 권한 PowerShell에서 `wsl --update` 를 실행해 WSL을 최신으로 갱신한 뒤, PC를 재시작하고 Docker Desktop을 다시 켜면 대부분 해결됩니다. 그래도 안 되면 오류 창에 함께 표시되는 안내 링크의 커널 업데이트를 적용합니다.

Q2. Hyper-V 백엔드와 WSL2 백엔드는 무엇이 다른가요? 무엇을 골라야 하나요? → A2. 둘 다 Windows에서 컨테이너용 리눅스 환경을 마련하는 방식인데, WSL2 백엔드가 더 가볍고 빠르며 Windows 11에서 권장되는 기본 방식입니다. 입문자는 설치 시 기본값인 WSL2 백엔드를 그대로 쓰면 됩니다. Hyper-V는 과거 방식이거나 특정 기업 환경에서 쓰는 경우가 많아, 이 책에서는 다루지 않습니다.

Q3. `docker` 명령은 PowerShell에서만 써야 하나요? 명령 프롬프트(cmd)에서도 되나요? → A3. 명령 프롬프트(cmd)에서도 동작합니다. 다만 이 책은 Windows 11에서 더 권장되는 PowerShell을 기준으로 명령과 프롬프트 표시(`PS C:\Users\...>`)를 통일합니다. 어떤 창에서 입력하든 `docker` 명령 자체와 결과는 같습니다.

Q4. Docker Desktop은 무료인가요? → A4. 개인 사용, 교육, 소규모 사업, 비상업적 오픈소스 용도로는 무료로 쓸 수 있습니다. 일정 규모 이상의 기업에서 업무용으로 쓸 때는 유료 구독이 필요할 수 있으니, 회사에서 사용한다면 최신 라이선스 조건을 한 번 확인하는 것이 좋습니다. 개인 학습 목적인 이 책의 실습에는 무료 범위로 충분합니다.

장 요약

이 장에서는 Windows 11에 Docker Desktop을 설치하고 정상 동작을 확인했습니다. 먼저 Windows에서 컨테이너가 대시보드 → 도커 엔진 → WSL2(경량 리눅스) → Windows 11의 층층 구조로 동작한다는 점을 이해하고, `winver · wsl --status` 와 작업 관리자로 시스템 요건을 점검했습니다. 이어 공식 사이트에서 설치 파일을 받아 WSL2 백엔드로 설치하고, `docker run hello-world` 로 "이미지 내려받기 → 컨테이너 실행 → 출력 → 종료"의 핵심 흐름이 정상인지 시운전했

습니다. 마지막으로 `docker version` · `docker info` 와 대시보드로 엔진 상태와 환경을 확인하는 방법을 익혔습니다. 이제 컨테이너를 다룰 준비가 끝났으니, 다음 장부터 본격적으로 PostgreSQL 컨테이너를 띄웁니다.

핵심 키워드

Docker Desktop, WSL2 백엔드(WSL2 backend), 도커 엔진(Docker Engine), `docker run`, 동작 확인(smoke test), `docker version`, `docker info`, 대시보드(dashboard)

다음 장 미리보기

다음 장에서는 드디어 공식 `postgres` 이미지를 내려받아(`docker pull`) 첫 PostgreSQL 컨테이너를 띄웁니다. 이미지와 컨테이너의 관계를 명령어로 직접 확인하고, `docker ps` · `stop` · `start` · `rm` 으로 컨테이너의 생명주기 한 바퀴를 다뤄 봅니다.

이미지와 컨테이너 기초: 첫 PostgreSQL 컨테이너 띄우기

학습 목표 - docker pull로 태그를 지정해 공식 postgres 이미지를 받을 수 있습니다. - docker run으로 이름과 옵션을 주어 PostgreSQL 컨테이너를 띄울 수 있습니다. - docker ps와 stop-start로 컨테이너 상태와 생명주기를 다룰 수 있습니다. - docker rm-rmi로 컨테이너와 이미지를 안전하게 정리할 수 있습니다.

앞 장에서 우리는 Windows 11에 Docker Desktop(도커 데스크톱)을 설치하고 `docker run hello-world`로 엔진이 잘 도는지 확인했습니다. 준비는 끝났습니다. 이제 이 책의 진짜 주인공인 **PostgreSQL 컨테이너**를 직접 띄워 볼 차례입니다.

이 장은 짧지만 책 전체의 뼈대입니다. 이미지를 내려받고(`docker pull`), 그 이미지로 컨테이너를 띄우고(`docker run`), 상태를 확인하고(`docker ps`), 멈추고 다시 켜고(`docker stop / start`), 마지막에 깨끗이 치우는(`docker rm / docker rmi`) 한 바퀴를 모두 다룹니다. 뒤 장에서 배울 포트 연결·데이터 영속·백업은 전부 이 한 바퀴 위에 얹히는 내용입니다.

이 책은 Windows 11의 **PowerShell(파워셸)**을 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, 프롬프트는 `PS C:\Users\me>`처럼 생겼습니다. 아래 예시에서 `PS C:\Users\me>`는 "여러분이 입력하는 자리"를 뜻하고, 그 아래 줄들은 Docker가 돌려주는 출력입니다.

용어 미리 보기: **레지스트리(registry)**는 이미지를 모아 두고 내려받게 해 주는 인터넷 저장소입니다. 가장 대표적인 곳이 **Docker Hub(도커 허브)**이며, postgres 같은 공식 이미지가 여기에 올라와 있습니다. **로컬(local)**은 내 PC를 가리킵니다.

이미지 받아오기: docker pull과 태그(tag)

도입

요리를 시작하려면 먼저 레시피와 재료가 손에 있어야 합니다. 컨테이너도 마찬가지입니다. PostgreSQL 컨테이너를 띄우기 전에, 그 설계도에 해당하는 **이미지(image)** 를 먼저 내 PC로 가져와야 합니다. 이 절에서는 인터넷 저장소(레지스트리)에서 이미지를 내려받는 `docker pull` 명령과, 어떤 버전을 받을지 고르는 **태그(tag)** 를 배웁니다.

핵심 개념 설명

docker pull (이미지 내려받기)

`docker pull` 은 원격 레지스트리(Docker Hub)에서 이미지를 내 PC(로컬)로 내려받는 명령입니다. 스마트폰 앱 스토어에서 앱을 내 기기로 내려받는 것과 똑같습니다. 내려받기만 할 뿐, 아직 실행하지는 않습니다.

- **왜(why) 필요한가:** 컨테이너는 이미지를 바탕으로 만들어집니다. 재료인 이미지가 내 PC에 없으면 컨테이너를 만들 수 없습니다. `docker pull` 은 그 재료를 미리 확보하는 단계입니다.
- **언제(when) 쓰나:** 새 소프트웨어를 처음 다룰 때, 또는 이미 받아 둔 이미지의 새 버전을 미리 받아 두고 싶을 때 씁니다.
- **어떻게(how) 쓰나:** `docker pull 이미지이름:태그` 형태로 적습니다. 예를 들어 `docker pull postgres:16` 입니다.

입문자가 자주 하는 오해 하나를 짚겠습니다. "`docker run` 전에는 반드시 `docker pull` 을 따로 해야 한다"고 생각하기 쉽지만, 그렇지 않습니다. `docker run` 은 필요한 이미지가 로컬에 없으면 알아서 내려받습니다. 그래도 이 절에서 `pull` 을 먼저 배우는 이유는, 이미지를 받아 오는 과정을 눈으로 똑똑히 보고 이해하기 위해서입니다.

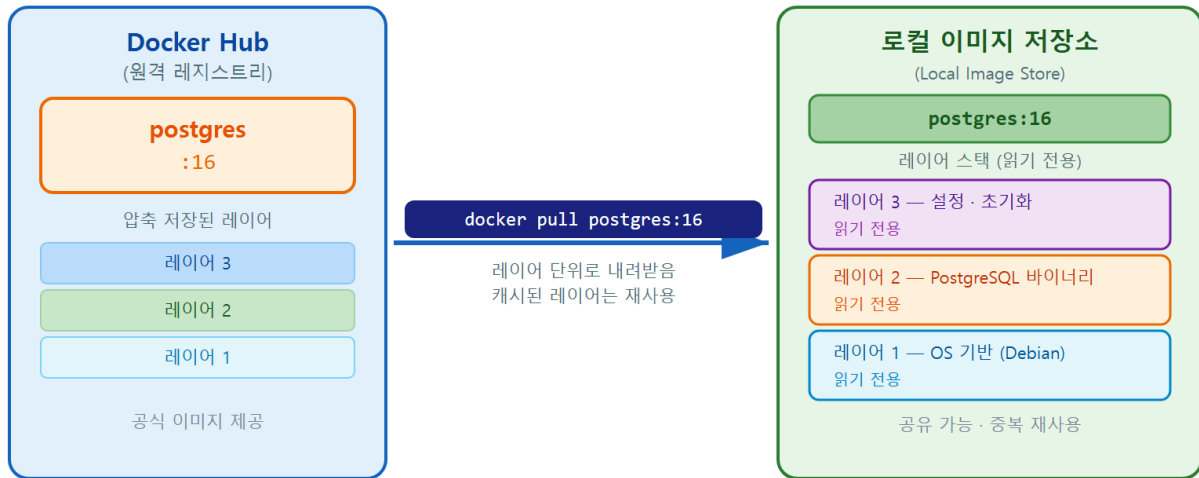
이미지 태그 (image tag)

이미지 태그(image tag) 는 같은 이미지의 여러 버전을 구분하기 위해 이름 뒤에 콜론(:)으로 붙이는 라벨입니다. `postgres:16` 에서 `16` 이 태그이며, "PostgreSQL 16번 메이저 버전"을 뜻합니다. 같은 책의 1판·2판처럼 판차를 표시하는 것과 같습니다.

태그에 대해 입문자가 가장 많이 걸려 넘어지는 지점은 **태그를 생략했을 때**입니다. `docker pull postgres` 처럼 태그를 빼면 안전한 고정 버전을 받는 것이 아니라, 자동으로 `latest` (가장 최근) 태그를 받습니다. 그런데 `latest` 는 시점에 따라 가리키는 실제 버전이 바뀝니다. 오늘 받은 `latest` 와 다음 달에 받은 `latest` 가 서로 다른 버전일 수 있다는 뜻입니다.

그래서 이 책에서는 항상 `postgres:16` 처럼 **버전 번호를 명시**합니다. 재현성(reproducibility), 즉 "어느 PC에서든 같은 환경이 다시 만들어지는 성질"을 지키기 위해서입니다.

docker pull — 레지스트리에서 이미지 내려받기



이미지 = 여러 읽기 전용 레이어의 스택 — 중복 레이어는 한 번만 내려받음

ch_03_s01 · docker pull과 이미지 레이어(layer)

이렇게 설정/실행합니다

다음은 공식 postgres 이미지의 16 버전을 내 PC로 내려받는 명령입니다.



```
PS C:\Users\me> docker pull postgres:16
....\big21>
```

예상 화면:

```
16: Pulling from library/postgres
a2318d6c47ec: Pull complete
6c3e7df31590: Pull complete
b8c8b9d35b2a: Pull complete
...
Digest: sha256:9a3a4d3f...e21b
Status: Downloaded newer image for postgres:16
docker.io/library/postgres:16
```

Pull complete 가 여러 줄 나오는 것이 보입니다. 이미지가 **여러 개의 읽기 전용 레이어(layer)**로 나뉘어 있어, 각 레이어를 따로 내려받기 때문입니다. 레이어는 이미지를 이루는 얇은 층입니다. 운영체제 기반 위에 PostgreSQL 설치 단계가 한 층씩 쌓여 하나의 이미지를 이룬다고 생각하면 됩니다. 같은 레이어를 다른 이미지가 공유하면 다시 받지 않아 저장 공간과 시간을 아낄 수 있습니다.

마지막 줄의 Status: Downloaded newer image for postgres:16 은 "새 이미지를 정상적으로 내려받았다"는 뜻입니다. 이미 받아 둔 상태에서 같은 명령을 다시 실행하면 Status: Image is up to

date for postgres:16 처럼 "이미 최신이라 받을 것이 없다"는 메시지가 나옵니다.

내려받은 이미지가 내 PC에 잘 들어왔는지는 `docker images` 로 확인합니다.

```
PS C:\Users\me> docker images
```

예상 화면:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
postgres	16	9a3a4d3f1c2b	2 weeks ago	438MB

명령과 출력 항목 설명

항목	의미
<code>docker pull</code>	레지스트리에서 이미지를 로컬로 내려받습니다
<code>postgres</code>	이미지 이름(Docker Hub의 공식 postgres 이미지)입니다
<code>:16</code>	받을 버전을 고르는 태그입니다(생략 시 <code>latest</code>)
<code>REPOSITORY</code>	이미지 이름입니다
<code>TAG</code>	받아 둔 태그(버전)입니다
<code>IMAGE ID</code>	이미지를 구분하는 고유 식별자입니다
<code>SIZE</code>	이미지가 차지하는 디스크 용량입니다

팁: 더 작은 이미지를 원하면 `postgres:16-alpine` 태그를 고려할 수 있습니다. `alpine` 은 아주 가벼운 리눅스를 바탕으로 만든 변형이라 용량이 작습니다. 다만 입문 단계에서는 기능이 가장 표준적인 `postgres:16` 을 권합니다. 익숙해진 뒤 용량이 부담되면 그때 바뀌도 늦지 않습니다.

주의: `docker pull postgres` 처럼 태그를 빼면 `latest` 를 받습니다. `latest` 가 가리키는 실제 버전은 시점마다 달라지므로, 학습·실습에서는 혼란을 줄이기 위해 항상 `postgres:16` 처럼 버전을 명시합니다. "분명 어제는 됐는데 오늘 동작이 다르다"는 문제의 흔한 원인이 바로 태그를 비워둔 것입니다.

절 요약

- `docker pull` 이미지:태그 로 레지스트리에서 이미지를 로컬로 내려받습니다.
- 이미지는 여러 읽기 전용 레이어로 이루어지며, 내려받기 화면의 여러 `Pull complete` 가 그 증거입니다.
- 태그를 생략하면 `latest` 를 받으므로, 재현성을 위해 `postgres:16` 처럼 버전을 명시합니다.
- 받아 둔 이미지는 `docker images` 로 확인합니다.

공식 postgres 이미지로 첫 컨테이너 띄우기: docker run

도입

start
실행
stop

재료(이미지)를 손에 넣었으니, 이제 실제로 요리를 시작합니다. 이미지라는 설계도를 바탕으로 살아 움직이는 PostgreSQL 컨테이너를 만들어 켜는 단계입니다. 이 절에서는 컨테이너를 만들고 실행하는 핵심 명령 `docker run` 을 배우고, 컨테이너에 이름을 붙이는 `--name` 과 PostgreSQL 에 꼭 필요한 비밀번호 설정 `POSTGRES_PASSWORD` 를 다룹니다.

핵심 개념 설명

기능 1

기능 2

docker run (컨테이너 만들어/실행하기)

docker run 은 이미지로부터 새 컨테이너를 만들어 곧바로 실행하는 명령입니다. 설계도(이미지)를 보고 실제 제품(컨테이너)을 조립한 뒤 전원을 켜는 것과 같습니다.

- **왜(why) 필요한가:** 이미지는 가만히 있는 설계도일 뿐입니다. 실제로 PostgreSQL이 돌아가게 하려면 컨테이너로 실행해야 하고, 그 일을 `docker run` 이 합니다.
- **언제(when) 쓰나:** 새 컨테이너를 띄울 때마다 씁니다. 중요한 점은, `docker run` 을 다시 실행하면 같은 컨테이너가 재사용되는 것이 아니라 **매번 새 컨테이너가 만들어진다는 것**입니다. 멈춰 둔 컨테이너를 다시 켜는 일은 다음 절의 `docker start` 가 맡습니다.
- **어떻게(how) 쓰나:** `docker run [옵션] 이미지:태그` 형태입니다. 옵션으로 이름·실행 방식·환경변수 등을 함께 지정합니다.

컨테이너 이름과 백그라운드 실행 (--name, -d)

컨테이너에는 자동으로 무작위 이름이 붙습니다(예: `pensive_turing`). 매번 이런 이름을 외우기는 어려우므로, `--name` 옵션으로 알아보기 쉬운 이름을 직접 붙입니다. 이 책에서는 `pg16` 처럼 짧고 분명한 이름을 씁니다.

`-d` 는 "detached(분리)"의 약자로, 컨테이너를 **백그라운드(뒤에서 조용히 도는 상태)** 로 실행하라는 뜻입니다. `-d` 를 빼면 컨테이너가 터미널을 점유해 PostgreSQL 로그가 화면에 계속 흐르고, 그 창에서는 다른 명령을 입력할 수 없습니다. 데이터베이스처럼 계속 **떠 있어야 하는 서비스** 는 `-d` 로 띄우는 것이 정석입니다.

데이터베이스 서버들
백그라운드 실행

POSTGRES_PASSWORD (필수 환경변수)

공식 postgres 이미지는 보안을 위해 **관리자 비밀번호를 반드시 지정하도록** 요구합니다. 이 값을 `-e POSTGRES_PASSWORD=...` 형태의 **환경변수(environment variable)** 로 전달합니다. 환경변수는 컨테이너를 켤 때 미리 정해 두는 초기 설정값입니다.

이 옵션을 빠뜨리면 컨테이너가 뜨자마자 곧바로 멈춥니다. 입문자가 "분명 명령을 쳤는데 컨테이너가 안 보인다"며 당황하는 가장 흔한 원인이 바로 이 비밀번호 누락입니다.

이렇게 설정/실행합니다

다음은 `pg16` 이라는 이름의 PostgreSQL 컨테이너를 백그라운드로 띄우는 명령입니다.

```
PS C:\Users\me> docker run -d --name pg16 -e POSTGRES_PASSWORD=mysecret postgres:16
```

예상 화면:

```
3f8a1c5d9e2b7a4c6f0d8b1e3a5c7d9f2b4a6c8e0d2f4a6c8e0b2d4f6a8c0e2d
```

길고 어지러운 글자 한 줄이 출력됩니다. 이것은 방금 만들어진 컨테이너의 **고유 ID(전체 식별자)** 입니다. 이 줄이 보이면 컨테이너가 정상적으로 생성·실행되었다는 뜻입니다. 보통은 이 긴 ID 대신 우리가 붙인 이름 `pg16` 으로 컨테이너를 다룹니다.

`docker run [-d] [--name 이름] [-e 변수=값] 이미지:태그`

옵션 설명

옵션	의미
<code>-d</code>	백그라운드(분리 모드)로 실행해 터미널을 점유하지 않습니다
<code>--name pg16</code>	컨테이너에 <code>pg16</code> 이라는 이름을 붙입니다
<code>-e</code> <code>POSTGRES_PASSWORD=mysecret</code>	관리자 비밀번호를 환경변수로 지정합니다(필수)
<code>postgres:16</code>	컨테이너의 바탕이 될 이미지와 태그입니다

컨테이너가 잘 났는지는 다음 절에서 배울 `docker ps` 로 확인합니다. 또한 PostgreSQL이 안에서 정상으로 기동했는지는 `docker logs` 로 들여다볼 수 있습니다.

```
PS C:\Users\me> docker logs pg16
```

예상 화면:

```
PostgreSQL init process complete; ready for start up.

LOG:  starting PostgreSQL 16.x on x86_64-pc-linux-gnu
LOG:  listening on IPv4 address "0.0.0.0", port 5432
LOG:  database system is ready to accept connections
```

마지막 줄 `database system is ready to accept connections` 가 보이면 PostgreSQL이 접속을 받을 준비를 마쳤다는 신호입니다.

주의: 여기서는 학습 편의를 위해 비밀번호를 명령에 직접 적었습니다. 실제 업무에서는 비밀번호를 명령이나 코드에 그대로 남기지 않습니다. 명령 기록(히스토리)에 비밀번호가 통째로 남기 때문입니다. 명령에 비밀번호를 분리해 두는 안전한 방법(`.env` 파일)은 6장에서 따로 배웁니다.

팁: 같은 이름으로 컨테이너를 또 만들려고 하면 `Conflict. The container name "/pg16" is already in use` 라는 오류가 납니다. 이름은 한 PC 안에서 중복될 수 없기 때문입니다. 이때는 기존 `pg16` 을 정리(이 장 마지막 절의 `docker rm`)한 뒤 다시 만들거나, 다른 이름을 씁니다.

절 요약

- `docker run` [옵션] 이미지:태그 로 이미지에서 새 컨테이너를 만들어 실행합니다.
- `-d` 는 백그라운드 실행, `--name` 은 알아보기 쉬운 이름 붙이기입니다.

- 공식 postgres 이미지는 `-e POSTGRES_PASSWORD=...` 가 없으면 곧바로 멈춥니다.
- 출력된 긴 글자는 컨테이너 고유 ID이며, 보통은 이름으로 컨테이너를 다룹니다.

컨테이너 목록과 상태: `docker ps / stop / start`

도입

컨테이너를 띄웠다면, 이제 그것이 지금 어떤 상태인지 살펴보고 켜고 끄는 법을 알아야 합니다. 전등을 켜고 끄듯, 컨테이너도 실행했다가 멈췄다가 다시 켤 수 있습니다. 이 절에서는 컨테이너 목록을 보는 `docker ps` 와, 상태를 바꾸는 `docker stop / start / restart`, 그리고 그 전체 흐름인 컨테이너 생명주기(container lifecycle)를 배웁니다.

핵심 개념 설명

`docker ps` (컨테이너 목록 보기)

`docker ps` 는 지금 실행 중인 컨테이너의 목록을 보여 주는 명령입니다. 여기서 `ps`는 "process status(프로세스 상태)"에서 온 이름입니다. 기본값으로는 실행 중인 컨테이너만 보여 줍니다.

여기서 입문자가 가장 자주 놓치는 부분이 있습니다. 멈춰 있는(정지된) 컨테이너는 `docker ps` 에 나오지 않습니다. "분명 만들었는데 목록에 없다"면 멈춰 있을 가능성이 큼니다. 정지된 것까지 모두 보려면 `-a` (all, 전부) 옵션을 붙여 `docker ps -a` 로 확인합니다.

컨테이너 생명주기 (container lifecycle)

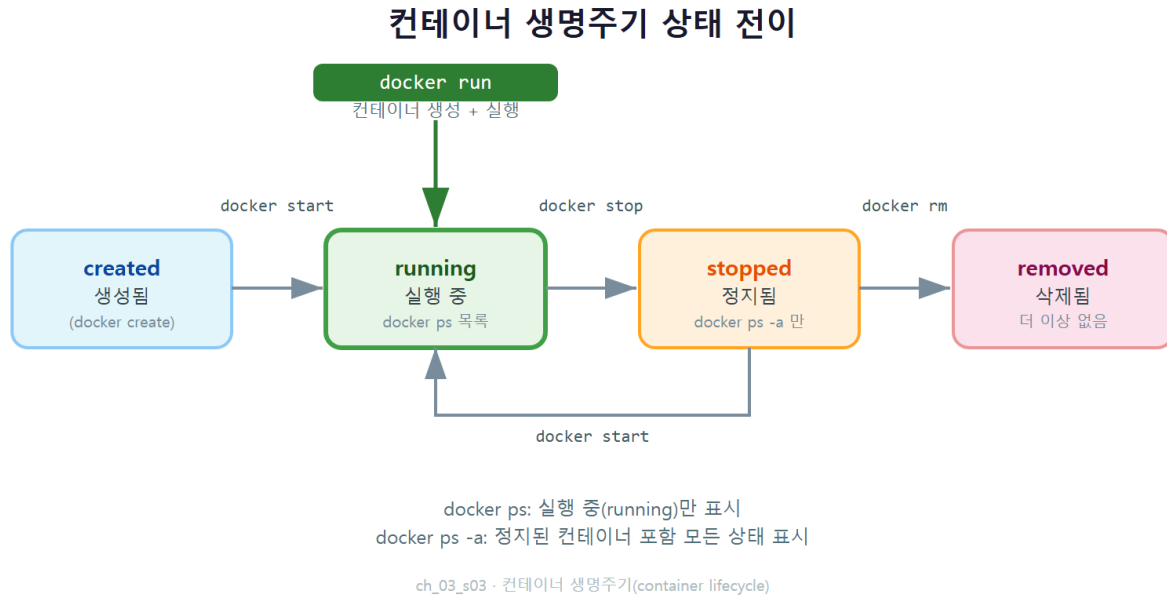
컨테이너 생명주기(container lifecycle)란, 컨테이너가 생성(created)·실행(running)·정지(stopped)·삭제(removed) 상태를 오가는 흐름입니다. 기기를 조립해 켜고, 끄고, 다시 켜고, 결국 폐기하는 제품의 한살이와 같습니다.

각 상태 전환은 명령으로 일어납니다.

암기

- `docker run`: 이미지에서 컨테이너를 만들어 실행 상태로 올립니다.
- `docker stop`: 실행 중인 컨테이너를 멈춰 정지 상태로 둡니다. 컨테이너는 사라지지 않고 그대로 남습니다.
- `docker start`: 정지된 컨테이너를 다시 실행 상태로 켵니다.
- `docker restart`: 멈췄다가 다시 켜는 동작을 한 번에 합니다. `stop -> start`
- `docker rm`: 정지된 컨테이너를 완전히 삭제합니다(다음 절).

여기서 꼭 구분할 점은 `docker stop` 과 `docker rm` 의 차이입니다. `stop` 은 잠시 끄는 것이고, `rm` 은 완전히 없애는 것입니다. 멈춘 컨테이너는 `docker start` 로 언제든지 다시 켤 수 있지만, 삭제한 컨테이너는 되살릴 수 없습니다.



이렇게 설정/실행합니다

먼저 실행 중인 컨테이너를 봅니다.

```
PS C:\Users\me> docker ps
```

예상 화면:

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
3f8a1c5d9e2b	postgres:16	"docker-entrypoint.s..."	Up 5 minutes	5432/tcp	pg16

STATUS 가 Up 5 minutes 이면 5분째 정상 실행 중이라는 뜻입니다. 이제 이 컨테이너를 멈춰 봅니다.

```
PS C:\Users\me> docker stop pg16
```

예상 화면:

```
pg16
```

멈춘 뒤 `docker ps` 를 다시 실행하면 목록이 비어 있습니다. 정지된 컨테이너는 기본 목록에 안 나오기 때문입니다. `-a` 를 붙이면 보입니다.

```
PS C:\Users\me> docker ps -a
```

예상 화면:

CONTAINER ID	IMAGE	COMMAND	STATUS	NAMES
3f8a1c5d9e2b	postgres:16	"docker-entrypoint.s..."	Exited (0) 10 seconds ago	pg16

STATUS 가 Exited (0) 으로 바뀌었습니다. 컨테이너가 정상 종료되어 정지 상태로 남아 있다는 뜻입니다. 멈춰 있을 뿐 사라지지는 않았습니다. 다시 켜 봅시다.

```
PS C:\Users\me> docker start pg16
```

예상 화면:

```
pg16
```

`docker ps` 로 다시 확인하면 STATUS 가 다시 `Up ...` 으로 돌아옵니다. 중요한 점은, 멈췄다 켜도 컨테이너 안에 쌓인 데이터가 그대로 남아 있다는 것입니다(데이터가 정말로 영구히 안전한지는 5장의 볼륨에서 따로 다룹니다).

명령과 STATUS 표기 설명

항목	의미
<code>docker ps</code>	실행 중인 컨테이너만 보여 줍니다
<code>docker ps -a</code>	정지된 것까지 모든 컨테이너를 보여 줍니다
<code>docker stop pg16</code>	실행 중인 <code>pg16</code> 을 멈춥니다(삭제 아님)
<code>docker start pg16</code>	정지된 <code>pg16</code> 을 다시 켵니다
<code>Up 5 minutes</code>	정상 실행 중이며 떠 있는 시간을 표시합니다
<code>Exited (0)</code>	정상 종료되어 정지 상태임을 표시합니다

암기

팁: `docker restart pg16` 은 `stop` 다음에 `start` 를 한 번에 실행합니다. 설정을 바꾼 뒤 컨테이너를 다시 띄워 적용하고 싶을 때 편리합니다. 목록이 길어 한 컨테이너만 보고 싶을 때는 `docker ps -a --filter name=pg16` 처럼 이름으로 걸러 볼 수 있습니다.

주의: `docker stop` 은 데이터를 지우지 않습니다. "컨테이너를 멈추면 데이터가 사라진다"는 것은 오해입니다. 데이터가 실제로 위험해지는 순간은 멈출 때가 아니라 다음 절의 `docker rm` 으로 컨테이너를 **삭제**할 때입니다. 이 차이는 5장에서 데이터 영속성을 배울 때 다시 강조합니다.

절 요약

- `docker ps` 는 실행 중인 컨테이너만, `docker ps -a` 는 정지된 것까지 모두 보여 줍니다.
- 컨테이너는 생성·실행·정지·삭제 상태를 `run · stop · start · rm` 명령으로 오갑니다.
- `stop` 은 잠시 끄기, `rm` 은 완전 삭제로 서로 다릅니다.
- 멈췄다 다시 켜도 컨테이너 안 데이터는 그대로 남습니다.

컨테이너와 이미지 정리: `docker rm / rmi`

도입

실습을 반복하다 보면 멈춰 둔 컨테이너와 다 쓴 이미지가 차곡차곡 쌓입니다. 안 쓰는 짐을 비워 창고를 정리하듯, 컨테이너도 주기적으로 치워야 디스크와 이름이 깔끔하게 유지됩니다. 이 절에서는 컨테이너를 지우는 `docker rm` 과 이미지를 지우는 `docker rmi` 를 배우고, 강제 삭제 옵션 `-f` 를 다룰 때의 주의점을 짚습니다.

핵심 개념 설명

`docker rm` (컨테이너 삭제)

`docker rm` 은 컨테이너를 완전히 삭제하는 명령입니다(rm은 remove). 앞 절의 `docker stop` 이 잠시 끄는 것이라면, `docker rm` 은 컨테이너 자체를 없애 다시 살릴 수 없게 합니다.

- **왜(why) 필요한가:** 정지된 컨테이너도 자리와 이름을 계속 차지합니다. 같은 이름(`pg16`)으로 새 컨테이너를 만들려면 먼저 기존 것을 지워야 합니다.
- **언제(when) 쓰나:** 실습이 끝났거나, 옵션을 바꿔 컨테이너를 새로 만들고 싶을 때 씁니다.
- **어떻게(how) 쓰나:** `docker rm` 컨테이너이름 입니다. 단, **실행 중인 컨테이너는 바로 지울 수 없습니다.** 먼저 `docker stop` 으로 멈춘 뒤 지우는 것이 안전한 순서입니다.

docker rmi (이미지 삭제) 이미지가 은근 용량이 큼.

docker rmi 는 내려받아 둔 이미지를 삭제하는 명령입니다(rmi는 remove image). 컨테이너 (`rm`)와 이미지(`rmi`)는 지우는 대상이 다릅니다. 컨테이너를 다 지웠다고 이미지까지 사라지는 것은 아닙니다. 이미지는 `docker rmi` 로 따로 지워야 디스크 용량이 회수됩니다.

여기서 순서가 중요합니다. **어떤 컨테이너가 사용 중인 이미지는 지울 수 없습니다.** 그 이미지를 바탕으로 만든 컨테이너(정지된 것 포함)가 하나라도 남아 있으면, Docker가 "이 이미지를 쓰는 컨테이너가 있다"며 삭제를 막습니다. 따라서 보통 **컨테이너를 먼저 `rm` 으로 지운 뒤, 이미지를 `rmi` 로 지우는** 순서를 따릅니다.

이렇게 설정/실행합니다

앞 절에서 멈춰 둔 `pg16` 을 삭제합니다(실행 중이면 `docker stop pg16` 을 먼저 실행합니다).

```
PS C:\Users\me> docker rm pg16
```

예상 화면:

```
pg16
```

지운 컨테이너 이름이 그대로 출력되면 삭제 성공입니다. `docker ps -a` 로 확인하면 목록에서 사라진 것을 볼 수 있습니다. 만약 실행 중인 컨테이너를 그냥 `rm` 하려 하면 다음과 같은 오류가 납니다.

```
Error response from daemon: You cannot remove a running container 3f8a...
Stop the container before attempting removal or force remove
```

이제 컨테이너가 모두 사라졌으니, 더 이상 쓰지 않는 이미지도 정리합니다.

```
PS C:\Users\me> docker rmi postgres:16
```

예상 화면:

```
Untagged: postgres:16
Deleted: sha256:9a3a4d3f1c2b...
Deleted: sha256:6c3e7df31590...
```

Untagged 와 여러 줄의 Deleted 가 보입니다. 이미지를 이루던 레이어들이 차례로 지워지며 디스크가 회수된 것입니다.

명령 설명

명령	의미
<code>docker rm pg16</code>	정지된 컨테이너 <code>pg16</code> 을 삭제합니다
<code>docker rm -f pg16</code>	실행 중이어도 강제로 멈춰 삭제합니다(주의)
<code>docker rmi postgres:16</code>	이미지 <code>postgres:16</code> 을 삭제합니다
<code>docker rmi -f postgres:16</code>	사용 중이어도 강제로 이미지를 삭제합니다(주의)

위험: `-f` (force, 강제) 옵션은 실행 중인 컨테이너를 멈추는 절차나 사용 중 확인을 건너뛰고 곧바로 지웁니다. 편해 보이지만, 안에 든 데이터를 의도치 않게 함께 날릴 수 있어 위험합니다. 입문 단계에서는 `-f` 를 습관처럼 쓰지 말고, `docker stop` 다음에 `docker rm` 을 차례로 실행하는 안전한 순서를 몸에 익히길 권합니다.

팁: 정지된 컨테이너가 여러 개 쌓였다면 `docker container prune` 으로 한 번에 정리할 수 있습니다. 다만 실행하면 "정말 지울지" 한 번 물어보며, 멈춰 있는 컨테이너를 모두 지우므로 남길 것이 없는지 확인한 뒤 진행합니다. 전체 리소스를 한꺼번에 정리하는 `docker system prune` 은 10장에서 주의점과 함께 다룹니다.

절 요약

- `docker rm` 은 컨테이너를, `docker rmi` 는 이미지를 삭제합니다(대상이 다름).
- 실행 중인 컨테이너는 먼저 `docker stop` 으로 멈춘 뒤 지우는 것이 안전합니다.
- 어떤 컨테이너가 쓰는 이미지는 지울 수 없으므로, 컨테이너를 먼저 정리합니다.
- `-f` 강제 삭제는 데이터를 함께 날릴 수 있어 입문 단계에서는 자제합니다.

실습 과제

해보기: postgres 컨테이너 띄우고 생명주기 한 바퀴 돌리기

앞에서 배운 명령을 이어서 직접 실행하며, 컨테이너의 생성부터 삭제까지 한 바퀴를 손으로 돌려 봅니다. 각 단계의 출력은 메모해 두면 좋습니다.

- 단계 1: `docker pull postgres:16` 으로 이미지를 받고, `docker images` 로 받아진 것을 확인합니다.
- 단계 2: `docker run -d --name pg16 -e POSTGRES_PASSWORD=mysecret postgres:16` 으로 컨테이너를 띄웁니다.
- 단계 3: `docker ps` 로 실행 중인지 확인하고, `docker logs pg16` 에서 `ready to accept connections` 줄을 찾아봅니다.
- 단계 4: `docker stop pg16` 으로 멈춘 뒤 `docker ps` (안 보임)와 `docker ps -a` (Exited 로 보임)의 차이를 직접 확인합니다.
- 단계 5: `docker start pg16` 으로 다시 켜고 `docker ps` 로 Up 상태를 확인합니다.
- 단계 6: `docker stop pg16` 다음 `docker rm pg16` 으로 컨테이너를 지우고, 마지막으로 `docker rmi postgres:16` 으로 이미지까지 정리합니다.
- 점검: 단계 6 이후 `docker ps -a` 와 `docker images` 가 모두 비어 있으면(또는 다른 실습물만 남아 있으면) 생명주기 한 바퀴를 깔끔히 마친 것입니다. 어느 단계에서 STATUS 가 어떻게 바뀌었는지 한 줄로 정리해 봅니다.

FAQ

Q1. `docker pull` 을 안 하고 바로 `docker run` 을 해도 되나요? → A1. 됩니다. `docker run` 은 지정한 이미지가 로컬에 없으면 알아서 내려받은 뒤 컨테이너를 실행합니다. 즉, `pull` 은 사실 선택입니다. 그래도 처음에는 `pull` 로 내려받는 과정을 눈으로 보는 것이 이미지·레이어 개념을 이해하는 데 도움이 됩니다.

Q2. 컨테이너를 띄웠는데 `docker ps` 에 안 보입니다. 왜 그런가요? → A2. 두 가지가 가장 흔합니다. 첫째, 컨테이너가 떴다가 곧바로 멈춘 경우입니다. 공식 `postgres` 이미지는 `-e POSTGRES_PASSWORD=...` 가 없으면 시작하자마자 종료됩니다. 둘째, 정지된 컨테이너는 `docker ps` 에 안 나오므로 `docker ps -a` 로 봐야 합니다. `docker logs` 컨테이너이름 으로 종료 원인 메시지를 확인하면 좋습니다.

Q3. `docker stop` 과 `docker rm` 은 무엇이 다른가요? → A3. `docker stop` 은 컨테이너를 잠시 끄는 것이고, 컨테이너는 정지 상태로 남아 `docker start` 로 다시 켤 수 있습니다. `docker rm` 은 컨테이너 자체를 완전히 삭제해 되살릴 수 없습니다. 멈추기는 일시정지, 삭제는 폐기라고 기억하면 됩니다.

Q4. 컨테이너를 `docker rm` 으로 지웠는데 이미지도 같이 사라지나요? → A4. 아닙니다. 컨테이너와 이미지는 별개입니다. `docker rm` 은 컨테이너만 지우고, 내려받아 둔 `postgres:16` 이미지는 그대로 남습니다. 이미지까지 지워 디스크를 회수하려면 `docker rmi postgres:16` 을 따로 실행해야 하며, 그 이미지를 쓰는 컨테이너가 모두 정리된 뒤라야 지워집니다.

장 요약

이 장에서는 PostgreSQL 컨테이너를 다루는 한살이 전체를 한 바퀴 돌았습니다. `docker pull` 로 태그를 명시해 공식 `postgres` 이미지를 내려받고, 이미지가 여러 레이어로 이루어진다는 것을 확인했습니다. `docker run` 에 `-d` · `--name` · `-e POSTGRES_PASSWORD` 를 주어 컨테이너를 백그라운드로 띄웠고, `docker ps / ps -a` 로 실행 중·정지 상태를 구분해 보며 `stop` · `start` 로 컨테이너 생명주기를 다뤘습니다. 끝으로 `docker rm` 과 `docker rmi` 로 컨테이너와 이미지를 안전한 순서로 정리하는 법까지 익혔습니다. 이제 컨테이너를 자유롭게 띄우고 치울 수 있으니, 다음 장에서는 이 컨테이너 안의 PostgreSQL에 실제로 접속해 보겠습니다.

핵심 키워드

`docker pull`, 이미지 태그(image tag), 레이어(layer), `docker run`, `--name`, `-d`, `POSTGRES_PASSWORD`, `docker ps`, 컨테이너 생명주기(container lifecycle), `docker rm`, `docker rmi`

다음 장 미리보기

다음 장에서는 격리된 컨테이너 속 PostgreSQL에 호스트(내 PC)에서 닿는 법을 배웁니다. 포트 매핑(`-p 5432:5432`)의 원리를 익히고, `psql` 과 DBeaver로 직접 접속해 보며, 포트가 겹칠 때 생기는 충돌을 진단하고 해결하는 방법까지 다룹니다.

포트 매핑과 접속: 호스트에서 컨테이너 PostgreSQL 만나기

학습 목표 - 컨테이너 격리와 **포트 매핑**(port mapping, **-p**)의 원리를 그림으로 설명할 수 있습니다. - 포트를 매핑한 컨테이너에 **호스트 psql**로 접속할 수 있습니다. - **DBeaver**로 컨테이너 PostgreSQL에 연결할 수 있습니다. - 포트 충돌(port conflict)을 진단하고 호스트 포트를 바꿔 해결할 수 있습니다.

앞 장에서 우리는 `docker run` 으로 PostgreSQL 컨테이너를 띄우고, `docker ps` 로 잘 돌고 있는지 확인했습니다. 그런데 막상 "그 안의 데이터베이스에 들어가 보자" 하면 막막합니다. 컨테이너는 분명히 실행 중인데, 내 PC에서 `psql`로 접속하면 연결이 안 됩니다. 왜 그럴까요.

이 장은 그 답을 다룹니다. 컨테이너는 기본적으로 **바깥과 단절된 격리 상자**라서, 그 안의 PostgreSQL에 닿으려면 **출입문을 하나 뚫어 줘야** 합니다. 그 출입문을 여는 옵션이 바로 **-p**(포트 매핑)입니다. 출입문을 열고 나면, 그다음은 익숙한 세계입니다. **PostgreSQL CRUD 교재 1장에서 익힌 psql 접속과 DBeaver 연결 방법이 그대로** 통하기 때문입니다.

명령 화면에 자주 보이는 `PS C:\Users\사용자>` 는 **PowerShell**(윈도우의 명령 입력 창)에서 명령을 기다리는 중이라는 표시입니다. `>` 뒤에 우리가 명령을 입력한다고 생각하면 됩니다.

컨테이너 네트워크와 포트 매핑(-p) 원리

도입

택배 기사가 아파트 단지에 들어왔다고 해 봅시다. 단지 안에 동·호수가 분명히 있어도, 정문이 잠겨 있으면 안으로 들어갈 수 없습니다. 컨테이너도 똑같습니다. 안에서 PostgreSQL이 멀쩡히 돌고 있어도, 컨테이너 바깥(호스트)에서 들어갈 **문**이 없으면 접속할 수 없습니다. 이 절에서는 그 문을 여는 **-p** 옵션이 정확히 무슨 일을 하는지 살펴봅니다.

핵심 개념 설명

컨테이너 격리와 포트 (container isolation / port)

컨테이너의 핵심 성질은 **격리(isolation)** 입니다. 컨테이너는 자기만의 독립된 네트워크 공간을 가지고 태어납니다. 즉 컨테이너 안의 PostgreSQL이 **5432번 포트**에서 요청을 기다리고 있어도, 그 5432번은 **컨테이너 내부의** 5432번일 뿐, 호스트 Windows의 5432번과는 전혀 다른 문입니다.

여기서 **포트(port)** 는 한 컴퓨터 안에서 프로그램마다 가진 **출입문 번호**라고 생각하면 됩니다. PostgreSQL은 관례적으로 5432번 문에서 손님을 기다립니다. 문제는, 이 문이 **컨테이너라는 닫힌 단지 안쪽**에 있다는 점입니다. 호스트에서 곧장 두드릴 수가 없습니다.

포트 매핑 (port mapping)

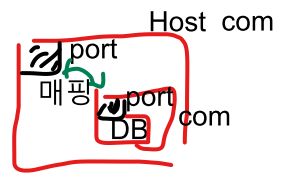
포트 매핑(port mapping) 은 `-p` 옵션으로 **호스트의 포트와 컨테이너 내부 포트를 연결**해, 격리된 컨테이너 서비스에 외부에서 닿을 수 있게 하는 설정입니다. 건물 안 특정 호실(컨테이너 포트)로 곧장 이어지는 **외부 전용 출입문(호스트 포트)** 을 하나 뚫어 두는 것과 같습니다.

표기법은 `-p 호스트포트:컨테이너포트` 입니다. 콜론(:)을 기준으로 **앞이 호스트, 뒤가 컨테이너** 라는 순서를 꼭 기억해야 합니다.

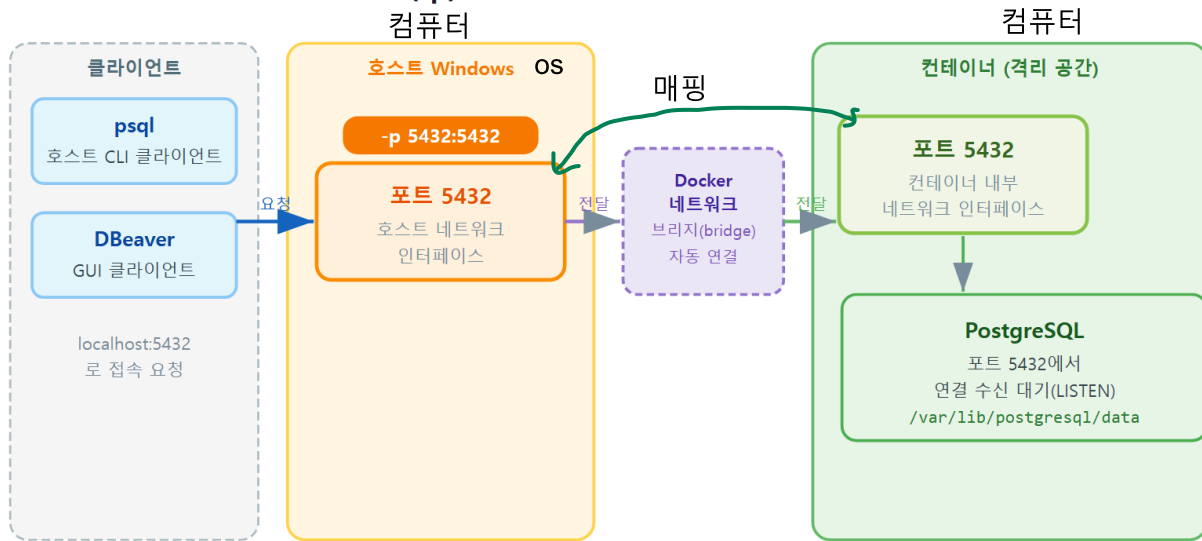
- **왜(why) 필요한가:** 컨테이너는 격리되어 있어, 매핑하지 않으면 호스트의 psql/DBever가 컨테이너 안 PostgreSQL에 닿을 수 없습니다.
- **언제(when) 쓰는가:** 호스트(또는 다른 PC)에서 컨테이너 안 서비스에 접속해야 할 때마다 씁니다. 데이터베이스, 웹 서버 등 거의 모든 서비스 컨테이너에 등장합니다.
- **어떻게(how) 동작하는가:** 호스트 포트로 들어온 요청을 도커가 가로채 컨테이너 내부 포트로 **전달(forwarding)** 합니다. 외부에서는 호스트 포트만 두드리면 됩니다.

port 외부:내부 매핑

`-p 5432:5432` 를 풀어 읽으면 "호스트의 5432번 문으로 들어온 요청을, 컨테이너 안 5432번 (PostgreSQL이 듣는 문)으로 이어 줘라" 가 됩니다.



포트 매핑(-p) 원리: 호스트 포트 → 컨테이너 포트



docker run -p 5432:5432 postgres 명령이 호스트 5432와 컨테이너 내부 5432를 자동으로 잇습니다

ch_04_s01 · 포트 매핑(-p) 원리

다음은 포트를 매핑하며 PostgreSQL 컨테이너를 띄우는 명령입니다(참고용 구성 — 실제로 PowerShell에 입력하는 명령입니다).

```
PS C:\Users\사용자> docker run -d --name pg-lab -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
                        생성&start                    환경                    이미지명
```

예상 화면:

```
3f8a1c9d2e7b6a4f0c5d9e8b1a2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c
```

위처럼 긴 문자열(컨테이너 ID)이 한 줄 출력되면 컨테이너가 백그라운드로 뒀다는 뜻입니다. 이어서 `docker ps` 로 매핑 상태를 확인합니다.

```
PS C:\Users\사용자> docker ps
```

예상 화면:

CONTAINER ID	IMAGE	COMMAND	...	PORTS	NAMES
3f8a1c9d2e7b	postgres:16	"docker-entrypoint.s..."	...	0.0.0.0:5432->5432/tcp	pg-lab

명령 옵션 설명

옵션	의미
<code>-d</code>	컨테이너를 백그라운드(detached)로 실행합니다
<code>--name pg-lab</code>	컨테이너에 <code>pg-lab</code> 이라는 이름을 붙입니다
<code>-e</code> <code>POSTGRES_PASSWORD=secret</code>	접속에 쓸 슈퍼유저 비밀번호를 지정합니다(필수)
<code>-p 5432:5432</code>	호스트 5432번을 컨테이너 5432번에 연결합니다(앞:호스트, 뒤:컨테이너)
<code>postgres:16</code>	사용할 이미지와 태그(버전 16)입니다

위 `docker ps` 결과의 `PORTS` 칸에 보이는 `0.0.0.0:5432->5432/tcp` 가 바로 매핑이 살아 있다는 증거입니다. "호스트의 모든 주소(`0.0.0.0`)의 5432번 → 컨테이너 5432번" 으로 읽으면 됩니다. 이 표시가 없다면 포트를 매핑하지 않은 것이고, 그러면 호스트에서 접속이 되지 않습니다.

흔한 오해 두 가지 - "ports를 매핑하지 않아도 호스트에서 컨테이너에 접속되겠지?" → 아닙니다. `-p` 가 없으면 컨테이너는 격리된 상태라 호스트 `psql`/DBBeaver가 닿지 못합니다. - " `-p 5432:5432` 는 앞뒤 숫자가 꼭 같아야 한다." → 아닙니다. `-p 5433:5432` 처럼 **호스트 쪽 번호는 자유롭게 바꿀 수 있습니다**. 뒤(컨테이너 포트)만 PostgreSQL이 듣는 5432로 맞추면 됩니다.

팁: 콜론 앞뒤 순서가 헷갈릴 때는 "**내 PC가 먼저, 컨테이너가 나중**" 으로 외우면 됩니다. `-p` 내 포트:컨테이너포트 . 외부에서 두드리는 문(내 PC 쪽)을 먼저 적는다고 생각하면 자연스럽습니다.

절 요약

- 컨테이너는 **격리**되어 있어, 안의 PostgreSQL에 닿으려면 포트를 열어 줘야 합니다.
- **포트 매핑** (`-p 호스트:컨테이너`) 은 호스트 포트로 온 요청을 컨테이너 포트로 전달합니다.
- `docker ps` 의 `PORTS` 칸에 `0.0.0.0:5432->5432/tcp` 가 보이면 매핑이 살아 있는 것입니다.



문을 뚫었으니 이제 들어가 볼 차례입니다. 여기서 반가운 사실 하나. 컨테이너 안에 있든 내 PC에 직접 설치돼 있든, **PostgreSQL은 PostgreSQL**입니다. 그래서 PostgreSQL CRUD 교재 1장에서 익힌 psql 접속법과 메타 명령이 컨테이너에도 **글자 하나 바꾸지 않고 그대로** 통합니다. 이 절에서 그것을 직접 확인합니다.

핵심 개념 설명

psql 접속 정보 (host / port / user / database)

psql 접속(psql connection) 은 호스트·포트·사용자·데이터베이스 정보를 주어 PostgreSQL 서버에 연결하는 일입니다. 택배를 보낼 때 주소(호스트)·문 번호(포트)·받는 사람(사용자)을 적는 것과 비슷합니다. 컨테이너로 띄웠더라도 접속에 필요한 네 가지 정보는 동일합니다.

항목	의미	이번 실습 값
호스트(host)	서버가 있는 위치	localhost (내 PC)
포트(port)	서버의 출입문 번호	5432 (매핑한 호스트 포트)
사용자(user)	접속할 계정 이름	postgres
데이터베이스(database)	접속할 창고 이름	postgres (기본)

여기서 핵심은 **호스트가 localhost** 라는 점입니다. 컨테이너는 내 PC 안의 WSL2 리눅스에서 돌지만, `-p` 로 호스트 5432에 문을 뚫어 두었기 때문에, 호스트 입장에서는 "내 PC(localhost)의 5432로 접속" 하면 도커가 알아서 컨테이너로 이어 줍니다. 즉 **컨테이너라는 사실을 의식할 필요 없이** 평소처럼 접속하면 됩니다.

CRUD 교재 1장의 접속법이 그대로 통한다

PostgreSQL CRUD 교재 1장에서는 `psql -h ip 주소localhost -p 유저5432 -U 유저명(슈퍼맨(모든 권한))postgres` 형태로 접속하고, `\l · \dt · \q` 같은 **메타 명령(백슬래시로 시작하는 둘러보기 명령)** 으로 내부를 둘러봤습니다. 컨테이너 PostgreSQL도 정확히 같은 명령으로 접속하고 둘러볼 수 있습니다. 서버가 어디서 돌든 psql 입장에서는 차이가 없기 때문입니다.

사전 준비: 이 방법은 호스트(Windows)에 psql 클라이언트가 설치돼 있어야 합니다. PostgreSQL CRUD 교재 1장대로 EDB 설치 관리자를 깔았다면 이미 `psql` 이 있습니

다. 호스트에 psql이 없다면, 7장에서 배울 `docker exec` 로 컨테이너 안의 psql 을 쓰는 방법도 있습니다.

이렇게 접속합니다

windows
PowerShell에서 호스트 psql로 컨테이너 PostgreSQL에 접속하는 명령입니다. CRUD 교재 1장에 서 쓰던 명령과 동일합니다. 터미널에서

`docker exec -it pg-lab psql -U postgres`
PS C:\Users\사용자> ~~psql -h localhost -p 5432 -U postgres~~

실행하면 컨테이너를 띄울 때 `-e POSTGRES_PASSWORD=secret` 으로 지정한 비밀번호(secret)를 물어보고, 맞으면 psql 프롬프트로 바뀝니다.

```

Password for user postgres:
psql (16.2)
Type "help" for help.

postgres=#
    
```

postgres=# 프롬프트가 나오면 접속 성공입니다. 이제 CRUD 교재 1장에서 익힌 메타 명령이 그 대로 통하는지 확인해 봅니다.

```

postgres=# \l
                                List of databases
  Name      | Owner   | Encoding | ...
-----+-----+-----+---
 postgres   | postgres | UTF8     | ...
 template0  | postgres | UTF8     | ...
 template1  | postgres | UTF8     | ...
(3 rows)

postgres=# \conninfo
You are connected to database "postgres" as user "postgres" on host "localhost" at port "5432".

postgres=# \q
PS C:\Users\사용자>
    
```

옵션·메타 명령 설명

입력	의미
<code>-h localhost</code>	내 PC(호스트)에 접속합니다. 매핑 덕분에 컨테이너로 이어집니다
<code>-p 5432</code>	매핑한 호스트 포트 5432로 접속합니다
<code>-U postgres</code>	슈퍼유저 <code>postgres</code> 계정으로 접속합니다
<code>\l</code>	데이터베이스 목록을 봅니다(CRUD 교재 1장과 동일)
<code>\conninfo</code>	지금 어디에 어떻게 접속했는지 접속 정보를 보여 줍니다
<code>\q</code>	psql을 종료하고 PowerShell로 돌아옵니다

팁: `\conninfo`의 출력에 `host "localhost" at port "5432"`라고 찍히는 것에 주목합니다. psql은 자신이 컨테이너에 접속했는지조차 모릅니다. 그저 "localhost의 5432"에 접속했을 뿐입니다. 컨테이너 운영의 편안함이 여기서 드러납니다. **접속하는 쪽은 평소와 똑같이** 하면 됩니다.

주의: 컨테이너를 띄운 직후 곧바로 접속하면 "connection refused" 또는 "데이터베이스 시스템이 시작 중" 같은 메시지가 날 수 있습니다. PostgreSQL이 컨테이너 안에서 **초기화를 마치고 준비되기까지 몇 초** 걸리기 때문입니다. 잠시 기다린 뒤 다시 시도하거나, `docker logs pg-lab`으로 `database system is ready to accept connections` 메시지가 떴는지 확인하면 됩니다(로그 확인은 7장에서 자세히 다룹니다).

절 요약

- 접속 정보 네 가지(**호스트·포트·사용자·데이터베이스**)는 네이티브 설치와 동일합니다.
- 호스트는 `localhost`, 포트는 매핑한 호스트 포트(5432)를 씁니다.
- **CRUD 교재 1장의 psql 접속·메타 명령(\l · \dt · \q)이 컨테이너에도 그대로 통합**합니다.

DBeaver(GUI)로 접속하기

도입

명령으로 접속하는 psql에 익숙해졌다면, 이번에는 마우스로 다루는 화면(GUI) 도구로도 같은 컨테이너에 연결해 봅니다. 결론부터 말하면, **PostgreSQL CRUD 교재에서 쓰던 DBeaver 사용**

법이 컨테이너에도 똑같이 적용됩니다. 입력칸에 넣는 값이 컨테이너냐 네이티브냐에 따라 달라지지 않기 때문입니다.

핵심 개념 설명

GUI 클라이언트와 연결 설정 (GUI client / connection setup)

GUI 클라이언트(GUI client) 는 PostgreSQL에 연결해 마우스로 다루는 화면 도구입니다. **DBeaver**가 대표적인 무료 도구입니다. psql과 역할은 같습니다. 둘 다 서버에 말을 거는 **손님(클라이언트)** 도구이고, psql은 명령으로, DBeaver는 화면으로 말을 건다는 차이뿐입니다.

연결을 만들 때 넣는 값들을 **연결 설정(connection setup)** 이라고 합니다. 앞 절에서 psql 옵션으로 적던 값(`-h` · `-p` · `-U`)을 DBeaver에서는 입력칸에 채워 넣을 뿐, 의미는 완전히 같습니다.

CRUD 교재의 DBeaver 사용법이 그대로 적용된다

PostgreSQL CRUD 교재에서 DBeaver로 연결할 때 Host·Port·Database·Username·Password 다섯 칸을 채웠습니다. 컨테이너 PostgreSQL도 **똑같은 다섯 칸**을 채우면 됩니다. 컨테이너라고 해서 별도의 연결 방식이나 드라이버가 필요하지 않습니다. 호스트에 `-p` 로 문을 뚫어 두었으므로, DBeaver 입장에서는 그냥 "localhost의 5432에 있는 PostgreSQL" 일 뿐입니다.

이렇게 연결합니다

DBeaver에서 새 연결을 추가하는 흐름입니다(화면의 버튼을 눌러 진행).

1. DBeaver 실행 후 상단 메뉴에서 새 연결 추가
(Database -> New Database Connection -> PostgreSQL 선택)
2. 연결 창에 아래 다섯 칸 입력
3. Test Connection(연결 시험) 클릭 -> 성공 확인
4. Finish 로 연결 저장

연결 창에 넣는 값은 psql 옵션과 정확히 대응합니다.

DBeaver 연결 창 입력값 (psql 옵션과 대응)

DBeaver 입력 칸	값	psql에서의 같은 옵션
Host	localhost	-h localhost
Port	5432	-p 5432
Database	postgres	끝에 적던 데이터베이스 이름
Username	postgres	-U postgres
Password	secret (컨테이너의 POSTGRES_PASSWORD)	접속 시 물어보던 비밀번호

값을 채운 뒤 **Test Connection** 버튼을 누르면 접속 가능 여부를 미리 확인할 수 있습니다. 성공하면 **Finish** 로 연결을 저장하고, 왼쪽 **Database Navigator**(데이터베이스 탐색기) 트리에서 `postgres > Schemas > public > Tables` 를 펼쳐 내부를 둘러봅니다(아직 테이블이 없으면 비어 있습니다).

쿼리는 **SQL Editor**(SQL 편집기)를 열어 입력하고 실행합니다. 같은 컨테이너이므로, psql에서 보던 것과 동일한 결과가 표(grid) 형태로 보입니다.

```
-- DBeaver의 SQL Editor에 입력한 뒤 Ctrl+Enter 로 실행합니다
SELECT datname FROM pg_database;
```

예상 결과(편집기 아래 결과 패널의 표):

```
+-----+
| datname |
+-----+
| postgres |
| template0 |
| template1 |
+-----+
```

주의: DBeaver로 접속하려면 당연히 **컨테이너가 실행 중**이어야 합니다(`docker ps` 로 확인). 컨테이너를 `docker stop` 으로 멈추면 호스트 5432 문 너머에 아무도 없으므로 "연결할 수 없음" 오류가 납니다. 또한 Password 칸에는 컨테이너를 띄울 때 `-e POSTGRES_PASSWORD` 로 지정한 값을 정확히 넣어야 합니다.

팁: psql과 DBeaver는 **같은 컨테이너에 동시에** 접속할 수 있습니다. 평소 둘러보기는 DBeaver로, 빠른 반복 작업은 psql로 처리하는 식으로 함께 쓰면 편리합니다. 어느 쪽으로 접속하든 컨테이너 안 데이터는 하나이므로, DBeaver에서 만든 테이블이 psql에서도 똑같이 보입니다.

절 요약

- DBeaver 연결은 **CRUD 교재와 동일한 다섯 칸**(Host·Port·Database·Username·Password)으로 끝납니다.
- 값은 psql 옵션과 일대일로 대응하며, 컨테이너라고 특별한 설정이 필요 없습니다.
- 연결하려면 컨테이너가 **실행 중**이어야 하고, Password는 `POSTGRES_PASSWORD` 값과 같아야 합니다.

포트 충돌 진단과 해결

도입

실습을 따라 하다 보면 누군가는 컨테이너를 띄우는 순간 빨간 오류를 만납니다. "port is already allocated" 같은 메시지입니다. 당황할 필요 없습니다. 이것은 컨테이너의 고장이 아니라, **호스트의 5432번 문을 이미 다른 프로그램이 쓰고 있다**는 흔하고 정상적인 신호입니다. 이 절에서는 그 오류를 읽고, 문 번호를 바꿔 깔끔하게 해결하는 법을 배웁니다.

핵심 개념 설명

포트 충돌 (port conflict)

포트 충돌(port conflict) 은 호스트의 한 포트를 이미 다른 프로그램이 쓰고 있어, 같은 포트로는 새 컨테이너를 매핑할 수 없는 상황입니다. 이미 차가 주차된 자리에 또 대려다 막히는 것과 같아서, **빈 자리(다른 포트)로 옮기면** 곧장 해결됩니다.

한 호스트 포트는 **한 번에 하나의 프로그램만** 차지할 수 있습니다. 그래서 다음 같은 경우 5432가 이미 점유돼 충돌이 납니다.

- PostgreSQL CRUD 교재 1장처럼 **PostgreSQL을 네이티브로 설치**해 두어 Windows 서비스가 이미 5432를 쓰는 경우.
- 앞서 띄운 **다른 PostgreSQL 컨테이너**가 이미 `-p 5432:5432` 로 5432를 잡고 있는 경우.

여기서 중요한 점은, **충돌은 호스트 포트(콜론 앞)에서만** 일어난다는 것입니다. 컨테이너 내부 포트(콜론 뒤)는 컨테이너마다 격리돼 있어 서로 부딪히지 않습니다.

포트 재지정 (-p 5433:5432)

해결책은 단순합니다. **호스트 쪽 번호만 비어 있는 번호로 바꿔** 주면 됩니다. 예를 들어 `-p 5433:5432` 는 "호스트 5433 → 컨테이너 5432" 로 잇습니다. 컨테이너 안 PostgreSQL은 여전히 5432를 듣고 있고(바꿀 필요 없음), 호스트에서는 5433 문으로 접속하게 됩니다.

이렇게 진단하고 해결합니다

먼저 충돌이 났을 때의 오류 화면을 읽어 봅니다. 5432가 이미 사용 중인데 또 `-p 5432:5432` 로 띄우면 다음과 같은 메시지가 납니다.

```
PS C:\Users\사용자> docker run -d --name pg-lab2 -e POSTGRES_PASSWORD=secret -p 5432:5432 postgr
```

예상 화면(오류):

```
docker: Error response from daemon: driver failed programming external connectivity
on endpoint pg-lab2: Bind for 0.0.0.0:5432 failed: port is already allocated.
```

`port is already allocated` (포트가 이미 할당됨)가 핵심 문구입니다. 컨테이너가 고장 난 것이 아니라 **호스트 5432가 이미 점유돼 있다**는 뜻입니다. 누가 쓰는지 PowerShell로 확인할 수 있습니다.

```
PS C:\Users\사용자> netstat -ano | findstr :5432
```

예상 화면:

```
TCP      0.0.0.0:5432    0.0.0.0:0      LISTENING      4812
```

`LISTENING` 으로 누군가 5432를 듣고 있음이 확인됩니다(맨 끝 숫자는 그 프로그램의 PID입니다). 이제 호스트 포트를 **5433** 으로 바꿔 다시 띄웁니다. 충돌 때문에 절반만 만들어진 컨테이너 이름이 남아 있을 수 있으니, 먼저 정리한 뒤 재실행합니다.

```
PS C:\Users\사용자> docker rm pg-lab2
PS C:\Users\사용자> docker run -d --name pg-lab2 -e POSTGRES_PASSWORD=secret -p 5433:5432 postgr
```

예상 화면:

```
a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2
```

성공했습니다. 이제 이 컨테이너에 접속할 때는 **호스트 포트 5433** 을 써야 합니다. psql도, DBeaver도 포트 칸만 5433으로 바꿉니다.

```
PS C:\Users\사용자> psql -h localhost -p 5433 -U postgres
```

진단·해결 명령 설명

입력	의미
<code>port is already allocated</code>	호스트 포트가 이미 다른 프로그램에 점유됐다는 오류 메시지
<code>netstat -ano findstr :5432</code>	5432 포트를 누가 쓰는지(PID 포함) 확인합니다
<code>docker rm pg-lab2</code>	충돌로 남은 생성 실패 컨테이너를 정리합니다
<code>-p 5433:5432</code>	호스트 포트만 5433으로 바꿔 충돌을 피합니다
<code>psql ... -p 5433</code>	접속 시에도 바뀐 호스트 포트(5433)를 사용합니다

팁: 어떤 호스트 포트로 매핑했는지 잊었다면 `docker ps` 의 `PORTS` 칸을 보면 됩니다.

`0.0.0.0:5433->5432/tcp` 라고 찍혀 있으면 "호스트 5433으로 접속" 이라는 뜻입니다. 접속이 안 될 때 가장 먼저 확인해야 할 곳입니다.

주의: 호스트 포트는 바뀌도 되지만, 콜론 뒤(**컨테이너 포트**) 는 PostgreSQL이 듣는 `5432` 로 두어야 합니다. 실수로 `-p 5433:5433` 처럼 적으면, 컨테이너 안에서 아무도 5433을 듣지 않으므로 매핑은 됐는데 접속이 안 되는 혼란스러운 상황이 됩니다.

절 요약

- `port is already allocated` 는 컨테이너 고장이 아니라 **호스트 포트 점유** 신호입니다.
- `netstat -ano | findstr :5432` 로 누가 쓰는지 확인할 수 있습니다.
- **호스트 포트만 5433 등으로 바꿔**(`-p 5433:5432`) 충돌을 피하고, 접속 시 그 포트를 사용합니다.

실습 과제

해보기: 포트 매핑한 컨테이너에 psql과 DBeaver로 접속하기

`-p` 로 포트를 매핑한 PostgreSQL 컨테이너를 띄우고, 호스트 psql과 DBeaver 양쪽으로 접속해 같은 데이터베이스가 보이는지 확인합니다. 이어 5432 충돌 상황을 만들어 5433으로 재매핑해 해결합니다(PostgreSQL CRUD 교재 1장 접속법을 그대로 재사용).

- **단계 1:** `docker run -d --name pg-lab -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres:16` 으로 컨테이너를 띄우고, `docker ps` 의 `PORTS` 칸에서 `0.0.0.0:5432->5432/tcp` 를 확인합니다.
- **단계 2:** `psql -h localhost -p 5432 -U postgres` 로 접속해 `\l` 로 데이터베이스 목록을, `\conninfo` 로 접속 정보를 확인하고 `\q` 로 나옵니다.
- **단계 3:** DBeaver에 Host `localhost`, Port `5432`, Database `postgres`, Username `postgres`, Password `secret` 을 넣어 Test Connection 을 성공시킵니다.
- **단계 4(충돌 만들기):** 같은 5432로 두 번째 컨테이너 `pg-lab2` 를 `-p 5432:5432` 로 띄워 `port is already allocated` 오류를 직접 봅니다.
- **단계 5(해결):** 실패한 컨테이너를 `docker rm pg-lab2` 로 지우고, `-p 5433:5432` 로 다시 띄운 뒤 `psql -h localhost -p 5433 -U postgres` 로 접속합니다.

점검표(직접 체크) - [] `docker ps` 의 `PORTS` 칸에서 매핑(`->`)을 읽을 수 있다. - [] 호스트 psql 로 접속해 `\conninfo` 에 `port "5432"` 가 보였다. - [] DBeaver Test Connection 이 성공했고, 같은 목록이 표로 보였다. - [] 충돌 오류 메시지에서 `port is already allocated` 를 확인했다. - [] 5433으로 재매핑한 컨테이너에 5433 포트로 접속했다.

FAQ

Q1. 컨테이너는 `docker ps` 에서 실행 중인데, psql로 접속하면 "connection refused" 라고 나옵니다. → A1. 두 가지를 확인합니다. 첫째, `docker ps` 의 `PORTS` 칸에 `0.0.0.0:5432->5432/tcp` 같은 매핑이 있는지 봅니다. 매핑(`->`)이 없으면 `-p` 옵션을 빠뜨린 것이므로 컨테이너를 다시 띄워야 합니다. 둘째, 컨테이너를 막 띄운 직후라면 PostgreSQL이 아직 초기화 중일 수 있으니 몇 초 기다린 뒤(또는 `docker logs pg-lab` 으로 `ready to accept connections` 확인 후) 다시 시도합니다.

Q2. `-p 5432:5432` 에서 앞뒤 숫자는 꼭 같아야 하나요? → A2. 아닙니다. 앞(호스트 포트)은 자유롭게 바꿀 수 있습니다. `-p 5433:5432`, `-p 15432:5432` 모두 가능합니다. 다만 뒤(컨테이너 포

트)는 **PostgreSQL이 듣는 5432**로 두어야 합니다. 호스트 포트를 바꿨다면, 접속할 때 psql/DBeaver의 포트 값도 그 번호로 맞춰야 합니다.

Q3. PostgreSQL CRUD 교재 1장과 접속 방법이 정말 똑같나요? 컨테이너용 특별한 명령이 따로 있는 건 아닌가요? → A3. 똑같습니다. 접속하는 쪽(psql·DBeaver)은 서버가 컨테이너에 있는지 네이티브 설치인지 알지 못합니다. `psql -h localhost -p 5432 -U postgres` 명령도, `\l · \dt · \q` 메타 명령도, DBeaver 연결 다섯 칸도 CRUD 교재 1장 그대로입니다. 컨테이너 쪽에서 `-p`로 문만 열어 두면 나머지는 평소와 동일합니다.

Q4. 네이티브로 설치한 PostgreSQL과 컨테이너를 동시에 쓰고 싶은데 5432가 겹칩니다. → A4. 호스트 포트를 다르게 주면 둘 다 쓸 수 있습니다. 네이티브 PostgreSQL이 5432를 쓰고 있으니, 컨테이너는 `-p 5433:5432`로 띄웁니다. 그러면 네이티브는 `localhost:5432`, 컨테이너는 `localhost:5433`으로 각각 접속됩니다. 포트로 둘을 구분하는 셈입니다.

장 요약

이번 장에서는 격리된 컨테이너 안의 PostgreSQL에 호스트에서 접속하는 방법을 배웠습니다. 컨테이너는 기본적으로 바깥과 단절돼 있어 **포트 매핑(-p 호스트:컨테이너)**으로 출입문을 뚫어줘야 하며, `docker ps`의 `PORTS` 칸에서 그 매핑을 확인할 수 있음을 보았습니다. 문을 열고 난 뒤에는 **PostgreSQL CRUD 교재 1장의 psql 접속·메타 명령과 DBeaver 연결법이 컨테이너에도 그대로** 적용된다는 것을 직접 확인했습니다. 마지막으로 `port is already allocated` 오류를 읽고, 호스트 포트를 5433 등으로 바꿔(`-p 5433:5432`) 충돌을 깔끔하게 해결하는 법을 익혔습니다. 이제 컨테이너 PostgreSQL을 평소 데이터베이스처럼 다룰 수 있습니다.

핵심 키워드

포트 매핑(-p), 호스트 포트, 컨테이너 포트, localhost 접속, psql 접속, DBeaver 연결, 포트 충돌, `-p 5433:5432`

다음 장 미리보기

다음 장에서는 컨테이너를 지워도 데이터가 사라지지 않게 지키는 **데이터 영속성(data persistence)**을 다룹니다. 컨테이너 휘발성의 함정을 실험으로 확인하고, **네임드 볼륨**으로 PostgreSQL 데이터를 안전하게 보존하는 방법을 배웁니다.

데이터 영속성: 컨테이너를 지워도 데이터를 지키기

학습 목표 - 컨테이너 삭제 시 데이터가 사라지는 휘발성 함정을 실험으로 보일 수 있습니다. - 네임드 볼륨(named volume)으로 PostgreSQL 데이터를 영속화할 수 있습니다. - 네임드 볼륨과 바인드 마운트(bind mount)의 차이와 쓰임을 비교할 수 있습니다. - `/var/lib/postgresql/data`가 데이터 저장 경로임을 이해하고 볼륨 연결을 확인할 수 있습니다.

앞 장까지 우리는 PostgreSQL 컨테이너를 띄우고, 포트를 열어 호스트에서 접속하는 데까지 익혔습니다. 그런데 지금 띄운 컨테이너에 회원 데이터를 잔뜩 넣어 두고, 무심코 컨테이너를 지웠다고 생각해 봅시다. 다시 컨테이너를 띄우면 그 데이터가 돌아올까요. 결론부터 말하면 **돌아오지 않습니다**.

이 장은 그 "사라짐"의 정체를 파헤치고, 데이터를 컨테이너 수명과 분리해 안전하게 지키는 방법을 다룹니다. 핵심은 **볼륨(volume)**입니다. 노트북을 새것으로 바꿔도 자료가 남도록 외장 디스크에 따로 보관해 두는 것과 같은 발상입니다.

이 책은 Windows 11의 **PowerShell(파워셸)**에서 `docker` 명령을 입력하는 상황을 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, 화면 앞의 `PS C:\Users\me>` 표시는 "지금 이 폴더에서 명령을 기다리는 중"이라는 안내입니다. 이 장의 명령은 모두 비실행 예시이므로, 실제로 따라 할 때는 컨테이너 이름이나 비밀번호를 본인 환경에 맞게 바꾸면 됩니다.

참고로 이 장에서 쓰는 `docker exec`는 7장에서 자세히 다루지만, 여기서는 "실행 중인 컨테이너 안으로 들어가 명령을 한 번 실행하는 도구" 정도로만 이해하면 충분합니다.

컨테이너는 사라진다: 데이터 휘발 함정

도입

컨테이너는 "쓰고 버리는" 것을 전제로 설계된 가벼운 실행 단위입니다. 가볍다는 장점의 이면에는, 컨테이너를 지우면 그 안에 쌓아 둔 데이터도 함께 사라진다는 함정이 숨어 있습니다. 이 절에서는 데이터를 직접 넣고 컨테이너를 지운 뒤 다시 띄워, 데이터가 정말 사라지는지 눈으로 확인합니다. 함정을 한 번 제대로 겪어 봐야, 다음 절에서 배울 볼륨의 가치가 분명해집니다.

핵심 개념 설명

컨테이너 휘발성 (ephemeral container)

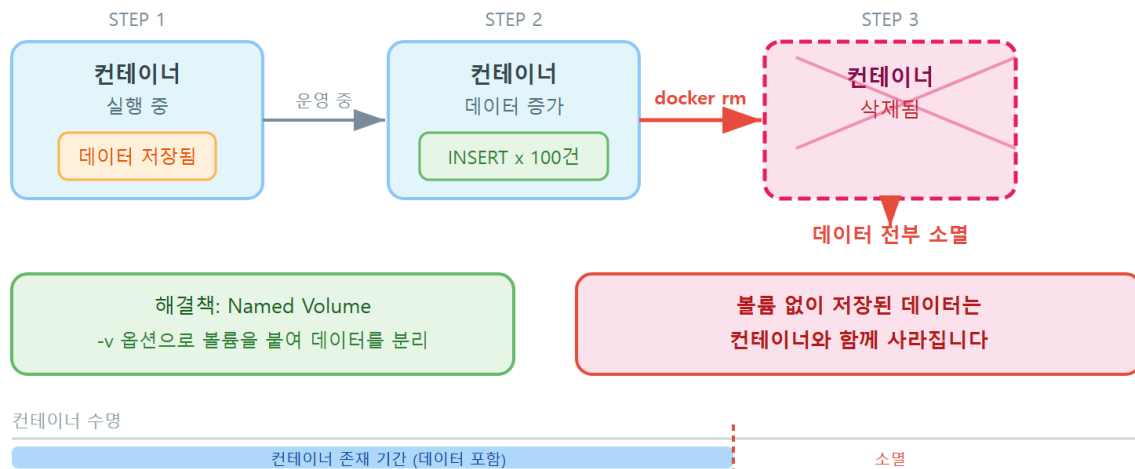
컨테이너 휘발성(ephemeral container)이란, 컨테이너 내부에 쓴 데이터가 그 컨테이너의 수명에 묶여 있어, 컨테이너를 삭제하면 데이터도 함께 사라지는 성질을 말합니다. 컨테이너는 이미지(설계도)에 "쓰기 가능한 얇은 층"을 한 겹 덮어 만들어집니다. 우리가 컨테이너 안에서 만드는 테이블이나 행은 모두 이 얇은 층에 쌓이는데, 컨테이너를 `docker rm`으로 지우면 이 층이 통째로 버려집니다.

- **왜(why) 이렇게 되나:** 컨테이너는 언제든지 지우고 다시 만들 수 있는 "일회용 실행 단위"로 설계되었습니다. 데이터를 컨테이너에 묶어 두면 이 일회용 성질과 충돌하므로, 도커는 데이터를 컨테이너 바깥에 두는 방법(볼륨)을 따로 제공합니다.
- **언제(when) 문제가 되나:** 컨테이너를 삭제(`docker rm`)하거나, 메이저 버전 업그레이드처럼 컨테이너를 새로 만들어 교체할 때 데이터가 통째로 날아갑니다.
- **무엇과 혼동하나:** `docker stop` (정지)만으로는 데이터가 사라지지 않습니다. 정지된 컨테이너를 다시 `docker start` 하면 데이터는 그대로입니다. 데이터를 날리는 것은 정지가 아니라 **삭제(rm)**입니다.

데이터 영속성 (data persistence)

데이터 영속성(data persistence)이란, 컨테이너가 사라져도 데이터가 보존되도록 컨테이너의 수명과 데이터의 수명을 분리하는 것입니다. 핵심 문장 하나로 정리하면 이렇습니다. "컨테이너는 일회용, 데이터는 영구 보관." 이 분리를 가능하게 해 주는 도구가 다음 절의 볼륨입니다.

컨테이너 삭제 시 데이터 휘발 — 경고



ch_05_s01 · 컨테이너 휘발성 — 볼륨 없이는 데이터도 사라진다

이렇게 실험합니다

먼저 볼륨을 붙이지 않은 컨테이너를 하나 띄웁니다. 이것이 함정의 출발점입니다.

```
# 볼륨 없이 PostgreSQL 컨테이너 띄우기 (데이터가 컨테이너에 묶임)
docker run -d --name pg-temp -e POSTGRES_PASSWORD=secret postgres:16
```

예상 출력:

```
a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2
```

이제 컨테이너 안으로 들어가 테이블을 만들고 데이터를 한 건 넣습니다. PostgreSQL CRUD 교재에서 익힌 SQL이 그대로 통합니다.

```
# 컨테이너 안 psql로 테이블 만들고 데이터 넣기
docker exec -it pg-temp psql -U postgres -c "CREATE TABLE memo (id serial, note text);"
docker exec -it pg-temp psql -U postgres -c "INSERT INTO memo (note) VALUES ('소중한 데이터');"
```

예상 출력:

```
CREATE TABLE
INSERT 0 1
```

데이터가 잘 들어갔는지 확인합니다.

```
docker exec -it pg-temp psql -U postgres -c "SELECT * FROM memo;"
```

예상 출력:

```
id |      note
----+-----
 1 | 소중한 데이터
(1 row)
```

여기까지는 평화롭습니다. 이제 컨테이너를 정지하고 **삭제**한 뒤, 같은 이름으로 새 컨테이너를 띄워 보겠습니다.

```
# 컨테이너를 정지하고 삭제
docker stop pg-temp
docker rm pg-temp

# 같은 이름으로 새 컨테이너를 다시 띄우기
docker run -d --name pg-temp -e POSTGRES_PASSWORD=secret postgres:16
```

다시 띄운 컨테이너에서 아까 만든 `memo` 테이블을 찾아봅니다.

```
docker exec -it pg-temp psql -U postgres -c "SELECT * FROM memo;"
```

예상 출력:

```
ERROR:  relation "memo" does not exist
LINE 1: SELECT * FROM memo;
                        ^
```

테이블 자체가 사라졌습니다. 같은 이미지, 같은 이름으로 띄웠지만 이전 컨테이너의 데이터는 함께 삭제되었기 때문입니다. 이것이 데이터 휘발 함정입니다.

명령 인자 설명

명령/인자	의미
<code>docker run -d</code>	컨테이너를 백그라운드(detached)로 실행합니다
<code>--name pg-temp</code>	컨테이너에 <code>pg-temp</code> 라는 이름을 붙입니다
<code>-e POSTGRES_PASSWORD=secret</code>	관리자 비밀번호를 환경변수로 지정합니다(필수)
<code>docker exec -it <이름> psql</code> ...	실행 중인 컨테이너 안에서 <code>psql</code> 명령을 한 번 실행합니다
<code>docker rm pg-temp</code>	정지된 컨테이너를 삭제합니다(이때 내부 데이터도 사라짐)

위험: 운영 중인 데이터베이스 컨테이너를 볼륨 없이 띄운 상태에서 `docker rm` 을 실행하면, 그 안의 모든 데이터가 복구 불가능하게 사라집니다. 실습이 아니라 실제 데이터가 들어 있는 컨테이너라면, 반드시 다음 절의 볼륨을 먼저 붙여 두어야 합니다.

팁: `docker stop` 과 `docker rm` 을 헷갈리지 않도록 기억하면 좋습니다. **stop은 전원을 끄는 것**(데이터 유지), **rm은 기기를 폐기하는 것**(데이터 소멸)입니다. 정지만 했다면 `docker start <이름>` 으로 데이터 그대로 되살릴 수 있습니다.

절 요약

- 컨테이너 내부에 쓴 데이터는 컨테이너의 "쓰기 가능한 층"에 쌓이며, 컨테이너 수명에 묶여 있습니다.
- `docker rm` 으로 컨테이너를 삭제하면 그 안의 데이터도 함께 사라집니다(`docker stop` 은 데이터를 유지).
- 데이터를 지키려면 컨테이너의 수명과 데이터의 수명을 분리해야 합니다(데이터 영속성).

네임드 볼륨(named volume)으로 데이터 영속화하기

도입

앞 절에서 데이터가 사라지는 함정을 직접 겪었습니다. 이제 해결책을 적용할 차례입니다. 컨테이너 바깥에 도커가 관리하는 별도의 저장 공간을 만들어 두고, 컨테이너의 데이터 저장 경로를

그곳에 연결하면, 컨테이너를 지웠다 다시 만들어도 데이터가 남습니다. 이 별도의 저장 공간이 **네임드 볼륨**입니다.

핵심 개념 설명

네임드 볼륨 (named volume)

네임드 볼륨(named volume)이란, 도커가 관리하는 영역에 이름을 붙여 만든 저장 공간으로, 컨테이너에 연결(마운트)해 데이터를 영속화하는 장치입니다. 라벨을 붙여 둔 전용 보관함과 같아서, 쓰던 기기(컨테이너)를 바꿔도 같은 보관함을 새 기기에 다시 꽂아 쓸 수 있습니다.

- **왜(why) 쓰나:** 볼륨은 컨테이너 바깥에 독립적으로 존재하므로, 컨테이너를 삭제해도 볼륨과 그 안의 데이터는 남습니다. 컨테이너는 일회용, 데이터는 볼륨에 영구 보관이라는 분리가 이루어집니다.
- **언제(when) 쓰나:** PostgreSQL처럼 데이터를 보존해야 하는 모든 컨테이너에 사실상 필수입니다. 데이터베이스를 띄울 때는 볼륨을 거의 항상 붙인다고 생각하면 됩니다.
- **어떻게(how) 쓰나:** `docker run`에 `-v 볼륨이름:컨테이너내부경로` 형태로 옵션을 붙입니다. PostgreSQL의 데이터 경로는 `/var/lib/postgresql/data`입니다(자세히는 마지막 절에서 다룹니다).

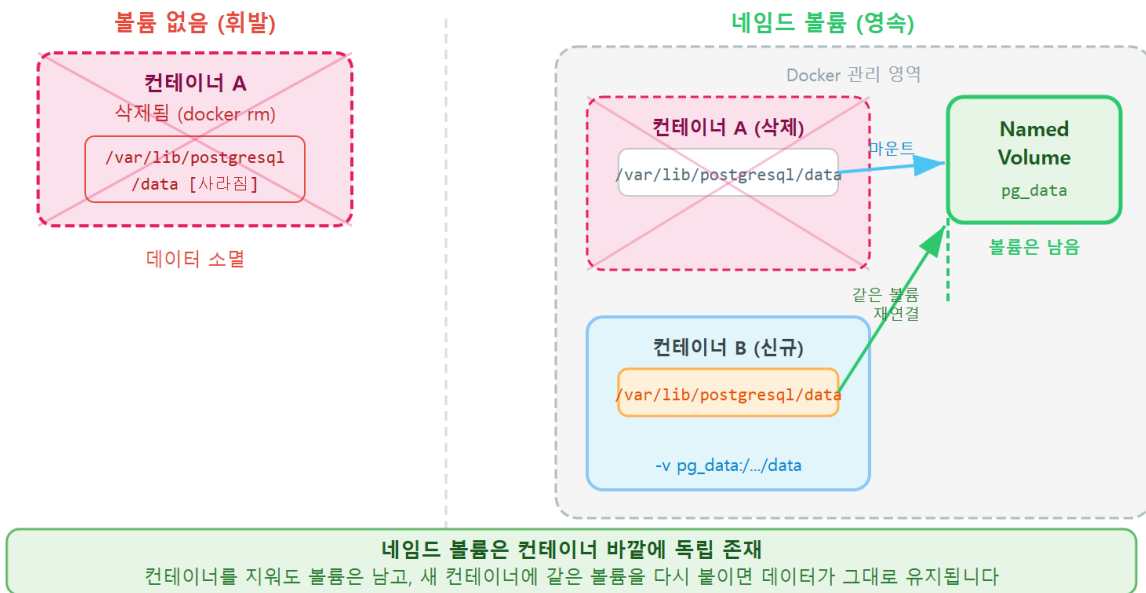
볼륨 마운트 (-v 옵션)

볼륨 마운트(-v 옵션)란, 컨테이너의 특정 내부 경로를 볼륨이나 호스트 폴더에 연결해, 그 경로에 쓰이는 데이터가 컨테이너 바깥에 저장되도록 하는 것입니다. `-v`

`pgdata:/var/lib/postgresql/data`라고 쓰면, "pgdata라는 이름의 볼륨을 컨테이너의 `/var/lib/postgresql/data` 경로에 연결하라"는 뜻입니다. 이렇게 하면 PostgreSQL이 그 경로에 저장하는 모든 데이터가 볼륨으로 흘러 들어갑니다.

볼륨은 미리 만들어 둘 필요가 없습니다. `-v pgdata:...`처럼 존재하지 않는 이름을 적으면 도커가 그 이름으로 볼륨을 **자동 생성**합니다.

네임드 볼륨으로 데이터 영속화



ch_05_s02 · 네임드 볼륨으로 데이터 영속화

이렇게 설정합니다

이번에는 `-v` 로 `pgdata` 볼륨을 붙여 컨테이너를 띄웁니다. 앞 절과 거의 같지만 `-v` 한 줄이 추가된 것이 핵심입니다.

pgdata 볼륨을 데이터 경로에 연결해 컨테이너 띄우기

```
docker run -d --name pg-keep -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data post
```

예상 출력:

```
f6e5d4c3b2a1f6e5d4c3b2a1f6e5d4c3b2a1f6e5d4c3b2a1f6e5d4c3b2a1f6e5
```

데이터를 한 건 넣습니다.

```
docker exec -it pg-keep psql -U postgres -c "CREATE TABLE memo (id serial, note text);"
```

```
docker exec -it pg-keep psql -U postgres -c "INSERT INTO memo (note) VALUES ('볼륨에 저장된 데이
```

예상 출력:

```
CREATE TABLE
INSERT 0 1
```

이제 앞 절차처럼 컨테이너를 **삭제하고 다시 만듭니다**. 단, 새로 띄울 때도 같은 `pgdata` 볼륨을 붙이는 것이 핵심입니다.

```
# 컨테이너 삭제 (볼륨은 -v를 명시적으로 지우지 않는 한 남는다)
docker stop pg-keep
docker rm pg-keep

# 같은 pgdata 볼륨을 붙여 새 컨테이너를 띄우기
docker run -d --name pg-keep -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data postgres
```

다시 띄운 컨테이너에서 `memo` 테이블을 찾아봅니다.

```
docker exec -it pg-keep psql -U postgres -c "SELECT * FROM memo;"
```

예상 출력:

```
id |      note
----+-----
  1 | 볼륨에 저장된 데이터
(1 row)
```

데이터가 그대로 살아 있습니다. 컨테이너는 새로 만들어졌지만, 데이터는 컨테이너 바깥의 `pgdata` 볼륨에 보관되어 있었기 때문입니다. 앞 절차 정확히 같은 절차를 밟았는데도 결과가 정반대인 이유는 오직 `-v` 옵션 하나입니다.

`-v` 옵션의 구조

부분	예시 값	의미
볼륨 이름	<code>pgdata</code>	도커가 관리하는 볼륨의 이름(없으면 자동 생성)
구분자	<code>:</code>	볼륨 이름과 컨테이너 경로를 나눕니다
컨테이너 내부 경로	<code>/var/lib/postgresql/data</code>	PostgreSQL이 실제 데이터를 쓰는 경로

주의: 컨테이너를 삭제해도 볼륨은 자동으로 사라지지 않습니다. 이는 데이터를 지키기 위한 안전장치이지만, 반대로 안 쓰는 볼륨이 디스크에 계속 쌓일 수 있다는 뜻이기도 합니다. 볼륨까지 함께 지우려면 `docker rm -v <컨테이너>` 처럼 `-v` 를 붙이거나, 8장에서 배울 `docker compose down` `-v` 를 써야 합니다. 데이터가 든 볼륨을 지울 때는 늘 한 번 더 확인합니다.

베스트 프랙티스: 데이터베이스 컨테이너는 처음 띄울 때부터 항상 네임드 볼륨을 붙이는 습관을 들이는 것이 좋습니다. "나중에 붙이지" 하고 미루다 데이터를 채운 뒤에는, 그 데이터를 안전하게 볼륨으로 옮기는 일이 번거로워집니다. 첫 `docker run` 부터 `-v` 를 함께 적습니다.

절 요약

- 네임드 볼륨은 도커가 컨테이너 바깥에 관리하는 저장 공간으로, 컨테이너를 지워도 남습니다.
- `-v 볼륨이름:/var/lib/postgresql/data` 로 PostgreSQL 데이터 경로를 볼륨에 연결합니다.
- 컨테이너를 교체할 때 같은 볼륨을 다시 붙이면 데이터가 그대로 이어집니다.

바인드 마운트(bind mount)와 비교하기

도입

볼륨으로 데이터를 지키는 법을 익혔습니다. 그런데 데이터를 컨테이너 바깥에 두는 방법이 하나 더 있습니다. 호스트(내 Windows PC)의 특정 폴더를 컨테이너에 직접 연결하는 **바인드 마운트**입니다. 둘은 비슷해 보이지만 저장 위치와 관리 방식이 다릅니다. 이 절에서 둘을 비교해, 언제 어느 것을 쓸지 감을 잡습니다.

핵심 개념 설명

바인드 마운트 (bind mount)

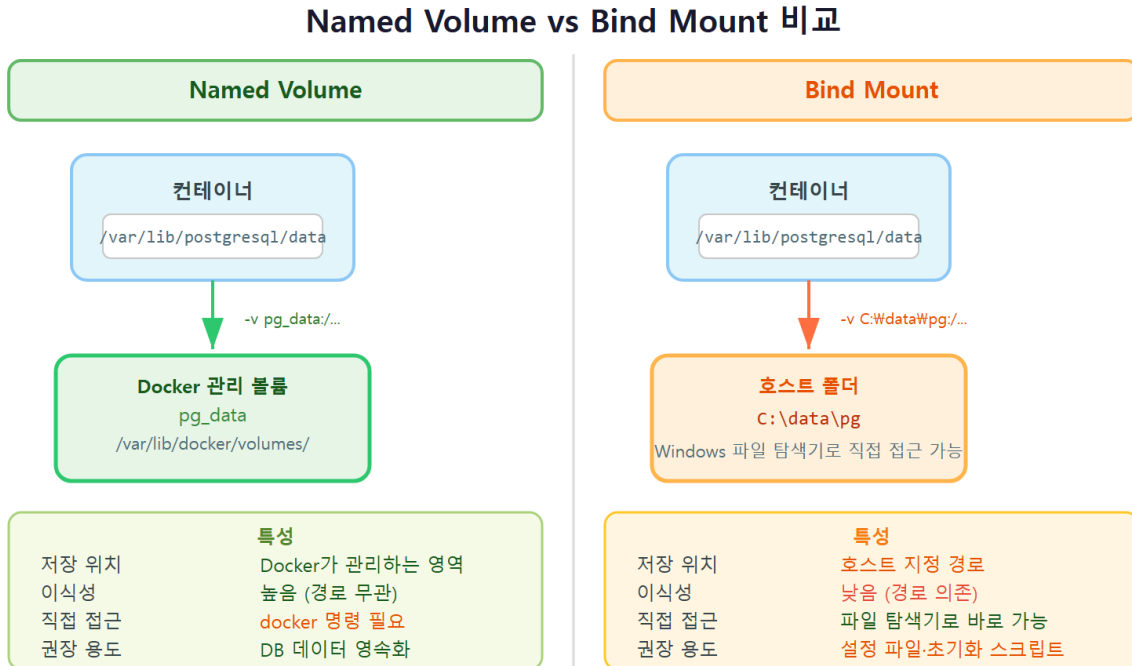
바인드 마운트(bind mount) 란, 호스트의 특정 폴더(예: `C:\docker\pgdata`)를 컨테이너 내부 경로에 직접 연결해 데이터를 공유하는 방식입니다. 내 책상 위 폴더를 작업실(컨테이너) 안에서 그대로 들여다보게 연결해 두는 것과 같습니다. 네임드 볼륨이 "도커가 관리하는 보이지 않는 보관함"이라면, 바인드 마운트는 "내가 위치를 정확히 아는 내 PC의 폴더"입니다.

- **왜(why) 쓰나:** 호스트에서 파일을 직접 보고 편집할 수 있습니다. 설정 파일이나 초기화 스크립트(6장)처럼 내가 직접 만든 파일을 컨테이너에 넣어 줄 때 특히 편리합니다.
- **언제(when) 쓰나:** 호스트의 파일을 컨테이너에 전달하거나, 컨테이너가 만든 파일을 호스트에서 곧바로 확인하고 싶을 때 적합합니다.
- **어떻게(how) 쓰나:** `-v` 옵션의 앞부분에 볼륨 이름 대신 **호스트 경로**를 적습니다.

Windows에서는 `-v C:\docker\pgdata:/var/lib/postgresql/data` 처럼 `C:\` 경로를 사용합니다.

볼륨 vs 바인드 마운트

둘 다 `-v` 로 쓰지만, 앞부분이 이름이면 네임드 볼륨, 경로(`C:\...` 또는 `/...`)면 바인드 마운트입니다. 이 한 곳 차이가 동작을 가릅니다.



ch_05_s03 · Named Volume vs Bind Mount — 저장 위치·이식성·접근 방식 비교

이렇게 설정합니다

다음은 호스트의 `C:\docker\pgdata` 폴더를 데이터 경로에 바인드 마운트하는 예시입니다.

```
# 호스트 폴더 C:\docker\pgdata 를 데이터 경로에 직접 연결
docker run -d --name pg-bind -e POSTGRES_PASSWORD=secret -v C:\docker\pgdata:/var/lib/postgresql
```

예상 출력:

```
0a9b8c7d6e5f0a9b8c7d6e5f0a9b8c7d6e5f0a9b8c7d6e5f0a9b8c7d6e5f0a9b
```

이렇게 띄우면 `C:\docker\pgdata` 폴더 안에 PostgreSQL의 데이터 파일들이 생깁니다. 탐색기로 그 폴더를 열면 파일이 보입니다. 반면 앞 절의 네임드 볼륨은 도커가 관리하는 내부 영역에 저장되어 탐색기에서 곧바로 보이지 않습니다.

두 방식의 차이를 표로 정리하면 다음과 같습니다.

네임드 볼륨 vs 바인드 마운트 비교

항목	네임드 볼륨	바인드 마운트
-v 표기	pgdata:/... (이름)	C:\docker\pgdata:/... (호스트 경로)
저장 위치	도커가 관리하는 영역	내가 지정한 호스트 폴더
호스트에서 파일 보기	곧바로는 어려움	탐색기에서 바로 보임
관리 명령	docker volume ls/inspect 로 관리	일반 폴더처럼 직접 관리
이식성	환경이 달라도 동일하게 동작	호스트 경로 구조에 의존
DB 데이터 권장도	권장(안정적)	비권장(권한·성능 이슈 가능)

주의: Windows 11에서 PostgreSQL의 데이터 디렉토리를 C:\ 폴더로 바인드 마운트하면, 파일 권한이나 파일 시스템 차이 때문에 컨테이너가 정상 기동하지 못하는 경우가 있습니다(권한 관련 오류로 시작에 실패하기도 합니다). 그래서 **데이터베이스의 데이터 저장에는 네임드 볼륨을 쓰는 것이 안전합니다**. 바인드 마운트는 초기화 스크립트나 설정 파일을 넣어 줄 때처럼 "내가 만든 파일을 전달"하는 용도에 더 잘 맞습니다.

팁: 바인드 마운트에 적은 호스트 경로(C:\docker\pgdata)가 없으면, 도커가 빈 폴더를 자동으로 만들어 연결합니다. 다만 경로 오타를 내면 엉뚱한 위치에 폴더가 생길 수 있으니, 경로는 정확히 적습니다.

절 요약

- 바인드 마운트는 호스트의 특정 폴더(C:\...)를 컨테이너 경로에 직접 연결합니다.
- -v 앞부분이 이름이면 네임드 볼륨, 경로면 바인드 마운트로 동작이 같습니다.
- DB 데이터 저장에는 네임드 볼륨이, 설정·스크립트 전달에는 바인드 마운트가 어울립니다.

데이터 디렉터리 /var/lib/postgresql/data 이해하기

도입

앞에서 볼륨을 붙일 때 매번 `/var/lib/postgresql/data` 라는 경로가 등장했습니다. 이 경로가 무엇이기엔 늘 여기에 볼륨을 붙이는 걸까요. 이 절에서는 PostgreSQL이 실제 데이터를 저장하는 이 경로의 의미를 이해하고, `docker volume` 명령으로 볼륨이 어디에 어떻게 연결되어 있는지 확인하는 법을 배웁니다.

핵심 개념 설명

데이터 디렉터리 (data directory)

데이터 디렉터리(data directory)란, PostgreSQL이 테이블·인덱스 등 모든 실제 데이터를 저장하는 디렉터리로, 공식 postgres 이미지에서는 `/var/lib/postgresql/data` 경로입니다. 참고에서 실제 물건이 쌓이는 지정된 적재 구역과 같습니다. 우리가 영속화를 위해 볼륨을 바로 이 경로에 마운트하는 이유는, 여기가 데이터가 실제로 쌓이는 자리이기 때문입니다.

- **왜(why) 이 경로인가:** 공식 postgres 이미지가 이 경로를 데이터 저장 위치로 정해 두었습니다. 따라서 이 경로에만 볼륨을 붙이면 데이터 전체가 영속화됩니다.
- **무엇을 오해하나:** "볼륨만 만들면 영속화된다"고 생각하기 쉽지만, 볼륨을 **이 경로에 마운트**해야 의미가 있습니다. 엉뚱한 경로에 붙이면 데이터는 여전히 컨테이너의 휘발 영역에 쌓입니다.

마운트 지점 (mount point)

마운트 지점(mount point)이란, 볼륨이나 호스트 폴더가 컨테이너 내부에서 연결되는 경로를 말합니다. PostgreSQL 컨테이너에서는 `/var/lib/postgresql/data` 가 그 지점입니다. 어떤 컨테이너의 마운트 지점과 연결된 볼륨이 무엇인지는 `docker inspect` 나 `docker volume inspect` 로 확인할 수 있습니다.

이렇게 확인합니다

지금 도커에 어떤 볼륨들이 있는지 목록으로 봅니다.

```
# 도커가 관리하는 볼륨 목록 보기
docker volume ls
```

예상 출력:

DRIVER	VOLUME NAME
local	pgdata

`pgdata` 볼륨이 보입니다. 이 볼륨의 상세 정보를 들여다봅니다.

```
# pgdata 볼륨의 상세 정보 보기
docker volume inspect pgdata
```

예상 출력:

```
[
  {
    "CreatedAt": "2026-06-17T09:12:33Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/pgdata/_data",
    "Name": "pgdata",
    "Options": null,
    "Scope": "local"
  }
]
```

Mountpoint 는 도커가 이 볼륨의 데이터를 실제로 저장하는 위치입니다. Windows에서는 WSL2 안의 리눅스 경로로 표시되며, 우리가 직접 그 폴더를 열 일은 거의 없습니다. 중요한 것은 "이 볼륨이 존재하고, 데이터가 여기에 보관된다"는 사실입니다.

이번에는 컨테이너 쪽에서 볼륨이 어느 경로에 연결되어 있는지 확인합니다.

```
# 컨테이너의 마운트 연결 정보만 추려 보기
docker inspect pg-keep --format "{{ range .Mounts }}{{ .Name }} -> {{ .Destination }}{{ end }}"
```

예상 출력:

```
pgdata -> /var/lib/postgresql/data
```

`pgdata` 볼륨이 컨테이너의 `/var/lib/postgresql/data` 에 연결되어 있음을 한 줄로 확인할 수 있습니다. 데이터 영속화가 제대로 걸려 있는지 점검할 때 이 명령이 유용합니다.

확인 명령 설명

명령	하는 일
<code>docker volume ls</code>	도커가 관리하는 볼륨 목록을 보여 줍니다
<code>docker volume inspect <이름></code>	특정 볼륨의 저장 위치·생성 시각 등 상세 정보를 보여 줍니다
<code>docker inspect <컨테이너></code>	컨테이너의 포트·볼륨·환경변수 등 전체 설정을 보여 줍니다
<code>--format "{{ ... }}"</code>	출력에서 원하는 항목만 추려 보여 줍니다

팁: 컨테이너가 데이터를 잃었다고 느껴지면, 가장 먼저 `docker inspect` 로 볼륨이 `/var/lib/postgresql/data` 에 제대로 연결되어 있는지 확인합니다. `-v` 옵션을 빼먹고 띄웠거나, 경로를 살짝 다르게 적은 경우가 흔한 원인입니다.

절 요약

- `/var/lib/postgresql/data` 는 PostgreSQL이 실제 데이터를 저장하는 경로이며, 볼륨은 이 경로에 마운트해야 합니다.
- `docker volume ls` 로 볼륨 목록을, `docker volume inspect` 로 상세 정보를 봅니다.
- `docker inspect <컨테이너>` 로 볼륨이 어느 경로에 연결됐는지 점검할 수 있습니다.

실습 과제

해보기: 볼륨으로 데이터 살아남기 실험하기

이 장의 핵심을 직접 손으로 확인합니다. 볼륨이 있을 때와 없을 때 데이터의 운명이 어떻게 갈리는지를 비교 실험합니다.

- 단계 1(휘발 확인): 볼륨 없이 `docker run -d --name pg-temp -e POSTGRES_PASSWORD=secret postgres:16` 으로 컨테이너를 띄우고, `docker exec` 로 테이블을 만들어 데이터를 한 건 넣습니다. 그런 다음 `docker stop` → `docker rm` → 같은 이름으로 다시 `docker run` 후 데이터를 조회해 **사라졌는지** 확인합니다.
- 단계 2(영속 확인): 이번에는 `-v pgdata:/var/lib/postgresql/data` 를 붙여 같은 과정을 반복합니다. 컨테이너를 지웠다 같은 볼륨으로 다시 띄운 뒤 데이터를 조회해 **남아 있는지** 확인합니다.

- 단계 3(점검): `docker volume ls` 와 `docker volume inspect pgdata` 로 볼륨이 존재하고 어디에 저장되는지 확인하고, `docker inspect` 로 볼륨이 `/var/lib/postgresql/data` 에 연결됐는지 점검합니다.
- 정리: 실험을 마치면 안 쓰는 컨테이너는 `docker rm` 으로 정리합니다. 단, `pgdata` 볼륨은 데이터가 남아 있으니 의도적으로 지울 때만 `docker volume rm pgdata` 를 실행합니다.

두 실험의 결과(단계 1은 사라짐, 단계 2는 유지)를 한 줄씩 기록해, "오직 `-v` 한 줄이 데이터의 운명을 갈랐다"를 본인 말로 정리해 봅니다.

FAQ

Q1. 컨테이너를 `docker stop` 만 해도 데이터가 사라지나요? → A1. 아닙니다. `docker stop` 은 컨테이너의 전원을 끄는 것일 뿐, 데이터는 그대로 남습니다. `docker start <이름>` 으로 다시 켜면 데이터를 그대로 쓸 수 있습니다. 데이터를 날리는 것은 정지가 아니라 **삭제**(`docker rm`) 입니다. 둘을 분명히 구분하면 불필요한 데이터 손실을 피할 수 있습니다.

Q2. 네임드 볼륨과 바인드 마운트 중 PostgreSQL에는 무엇을 써야 하나요? → A2. 데이터베이스의 데이터 저장에는 **네임드 볼륨**을 권합니다. 도커가 관리하므로 권한·성능 면에서 안정적입니다. 특히 Windows에서 `c:\` 폴더로 바인드 마운트하면 권한 문제로 컨테이너가 기동에 실패할 수 있습니다. 바인드 마운트는 초기화 스크립트나 설정 파일을 컨테이너에 전달하는 용도(6장)에 더 알맞습니다.

Q3. 컨테이너를 지웠는데 볼륨이 그대로 남아 디스크를 차지합니다. 어떻게 정리하나요? → A3. 컨테이너를 지워도 볼륨은 안전을 위해 자동으로 사라지지 않습니다. `docker volume ls` 로 목록을 본 뒤, 더 이상 필요 없는 볼륨만 골라 `docker volume rm <볼륨이름>` 으로 지웁니다. 단, 그 볼륨에 든 데이터는 함께 사라지므로 지우기 전에 정말 필요 없는지 반드시 확인합니다. 안 쓰는 볼륨을 한꺼번에 정리하는 방법은 10장에서 다룹니다.

Q4. 같은 볼륨을 두 개의 컨테이너에 동시에 붙여도 되나요? → A4. 기술적으로 가능하지만 PostgreSQL 데이터 디렉터리에는 권하지 않습니다. 두 PostgreSQL 인스턴스가 같은 데이터 파일을 동시에 쓰면 데이터가 손상될 수 있습니다. 데이터 볼륨 하나에는 한 번에 하나의 PostgreSQL 컨테이너만 연결하는 것을 원칙으로 삼습니다.

장 요약

이 장에서는 컨테이너가 "쓰고 버리는" 일회용 단위라서, 볼륨 없이 데이터를 넣으면 컨테이너 삭제와 함께 데이터가 사라진다는 휘발 함정을 실험으로 확인했습니다. 그 해결책으로 **네임드 볼륨**을 `-v 볼륨이름:/var/lib/postgresql/data` 로 연결해, 컨테이너를 지웠다 다시 만들어도 데이터가 살아남게 하는 법을 익혔습니다. 또한 호스트 폴더를 직접 잇는 **바인드 마운트**와 비교해 각자의 쓰임을 구분하고, `docker volume ls · inspect` 로 볼륨의 위치와 연결을 점검했습니다. 핵심 문장은 하나입니다. "컨테이너는 일회용, 데이터는 볼륨에 영구 보관."

핵심 키워드

데이터 영속성(data persistence), 컨테이너 휘발성(ephemeral container), 네임드 볼륨(named volume), `-v` 볼륨 마운트, 바인드 마운트(bind mount), 데이터 디렉터리(data directory), `/var/lib/postgresql/data`, `docker volume ls/inspect`

다음 장 미리보기

다음 장에서는 컨테이너를 띄울 때 환경변수로 비밀번호·사용자·기본 데이터베이스를 지정하고, `/docker-entrypoint-initdb.d` 에 초기화 스크립트를 넣어 기동과 동시에 테이블과 시드 데이터를 자동으로 준비하는 법을 배웁니다. 이때 이 장에서 익힌 볼륨이 "초기화가 최초 1회만 도는" 동작과 어떻게 엮히는지도 함께 살펴봅니다.

환경변수와 초기화: 컨테이너에 PostgreSQL 구성하기

학습 목표 - POSTGRES_PASSWORD·USER·DB 환경변수로 컨테이너를 구성할 수 있습니다. - /docker-entrypoint-initdb.d로 초기화 스크립트를 자동 적용할 수 있습니다. - 초기화 스크립트가 최초 1회만 실행되는 이유와 볼륨의 관계를 설명할 수 있습니다. - .env로 비밀번호를 분리해 다루는 습관을 적용할 수 있습니다.

지난 장까지 우리는 컨테이너를 띄우고, 접속하고, 볼륨으로 데이터를 영속하는 법을 익혔습니다. 그런데 매번 컨테이너를 만들 때마다 빈 PostgreSQL이 뜨고, 거기에 사람이 손으로 들어가 사용자 계정을 만들고 데이터베이스를 만들고 테이블을 만든다면 번거롭습니다. 새 PC, 새 동료, 새 환경마다 같은 작업을 반복해야 하니까요.

이 장은 그 반복을 없애는 장입니다. 컨테이너를 켜는 순간 **비밀번호·사용자·기본 데이터베이스가 정해지고, 테이블과 시작 데이터까지 자동으로 준비되도록** 만드는 법을 배웁니다. 핵심 도구는 두 가지입니다. 하나는 컨테이너를 켤 때 미리 값을 넣어 주는 **환경변수(environment variable)**, 다른 하나는 첫 기동 때 SQL을 자동 실행해 주는 **초기화 스크립트(init script)** 입니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 을 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, 프롬프트는 `PS C:\Users\me>` 처럼 생겼습니다. 아래 예시에서 `PS C:\Users\me>` 는 "여러분이 입력하는 자리"를 뜻하고, 그 아래 줄들은 Docker나 PostgreSQL 이 돌려주는 출력입니다.

용어 미리 보기: **환경변수(environment variable)** 는 프로그램을 실행할 때 바깥에서 미리 정해 주는 설정값입니다. 스마트폰을 처음 켤 때 정하는 언어·시간대처럼, 컨테이너도 켜질 때 "비밀번호는 이것, 사용자는 이것"이라고 미리 알려 줄 수 있습니다.

환경변수로 컨테이너 구성하기:

POSTGRES_PASSWORD/USER/DB

도입

새 노트북을 처음 켜면 언어를 고르고, 시간대를 정하고, 계정을 만드는 초기 설정 화면이 뜹니다. PostgreSQL 컨테이너에도 그런 "처음 켤 때 정하는 값"이 있습니다. 이 절에서는 컨테이너에 값을 전달하는 `-e` 옵션과, 공식 postgres 이미지가 알아보는 세 가지 핵심 값

`POSTGRES_PASSWORD` · `POSTGRES_USER` · `POSTGRES_DB` 를 배웁니다.

핵심 개념 설명

환경변수 (environment variable)와 `-e` 옵션

환경변수(environment variable) 는 컨테이너를 켤 때 바깥에서 미리 넣어 주는 설정값입니다. 공식 postgres 이미지는 정해진 이름의 환경변수를 읽고, 그 값에 맞춰 첫 기동 때 PostgreSQL 을 구성합니다.

- **왜(why) 필요한가:** 컨테이너 안에 들어가 손으로 계정과 데이터베이스를 만드는 대신, 컨테이너를 켜는 명령 한 줄에 설정을 함께 적어 두면 누가 언제 띄워도 같은 구성이 만들어 집니다.
- **언제(when) 쓰나:** 컨테이너를 처음 만들 때입니다. 뒤에서 강조하겠지만, 이 값들은 데이터베이스를 **처음 생성하는 순간**에만 영향을 줍니다.
- **어떻게(how) 쓰나:** `docker run` 에 `-e 이름=값` 형태로 붙입니다. `-e` 는 environment(환경)의 머리글자이며, 여러 개를 줄 수 있습니다.

공식 postgres 이미지가 알아보는 대표 환경변수는 세 가지입니다.

- `POSTGRES_PASSWORD` : 관리자 계정의 비밀번호입니다. **반드시 지정해야 하는 유일한 필수 값**입니다. 빠뜨리면 컨테이너가 뜨자마자 멈춥니다.
- `POSTGRES_USER` : 만들 관리자(슈퍼유저) 계정의 이름입니다. 생략하면 기본값 `postgres` 가 쓰입니다.
- `POSTGRES_DB` : 처음 만들어 둘 기본 데이터베이스의 이름입니다. 생략하면 `POSTGRES_USER` 와 같은 이름의 데이터베이스가 만들어 집니다.

이 값들이 "최초 생성"에만 영향을 준다는 점

여기서 입문자가 가장 많이 오해하는 부분을 미리 못 박아 두겠습니다. `POSTGRES_USER` 와 `POSTGRES_DB` 는 **데이터 디렉터리가 비어 있는 첫 기동에만** 작동합니다. 즉 컨테이너가 사용할 데이터가 아직 하나도 없을 때, 그 값으로 계정과 데이터베이스를 새로 만들어 줍니다.

이미 데이터가 들어 있는 볼륨을 다시 붙여 띄우면, 이 값들을 바꿔 적어도 아무 일도 일어나지 않습니다. 예를 들어 처음에 `POSTGRES_DB=shop` 으로 만든 뒤, 나중에 `POSTGRES_DB=blog` 로 바꿔 같은 볼륨으로 다시 띄워도 `blog` 데이터베이스는 생기지 않습니다. 이 함정은 다음 절에서 자세히 다룹니다. 지금은 "이 값들은 처음 한 번만 통한다"는 점만 기억하면 됩니다.

이렇게 설정/실행합니다

다음은 비밀번호·사용자·기본 데이터베이스를 한꺼번에 지정해 컨테이너를 띄우는 명령입니다.

```
PS C:\Users\me> docker run -d --name pg16 `
-e POSTGRES_PASSWORD=mysecret `
-e POSTGRES_USER=appuser `
-e POSTGRES_DB=shop `
-p 5432:5432 postgres:16
```

PowerShell에서 줄 끝의 백틱(`)은 "명령이 다음 줄로 이어진다"는 표시입니다. 한 줄로 길게 쓰는 것이 어렵다면 위처럼 옵션마다 줄을 나눠도 됩니다.

예상 화면:

```
7c2f1a9b3d4e8f0a1c2b3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a
```

길고 어지러운 글자 한 줄은 방금 만들어진 컨테이너의 고유 ID입니다. 이 줄이 보이면 정상적으로 만들어졌다는 뜻입니다. 정말로 `appuser` 계정과 `shop` 데이터베이스가 만들어졌는지 컨테이너 안의 `psql`로 확인해 봅시다.

```
PS C:\Users\me> docker exec -it pg16 psql -U appuser -d shop -c "\conninfo"
```

예상 화면:

```
You are connected to database "shop" as user "appuser" via socket in "/var/run/postgresql" at po
```

`database "shop" as user "appuser"` 라는 문구가 보이면, 환경변수로 준 값대로 데이터베이스와 계정이 만들어진 것입니다.

`docker run -e POSTGRES_PASSWORD=값 [-e POSTGRES_USER=값] [-e POSTGRES_DB=값] 이미지:태그`

환경변수 설명

환경변수	의미	생략하면
<code>POSTGRES_PASSWORD</code>	관리자 계정 비밀번호(필수)	컨테이너가 곧바로 멈춤
<code>POSTGRES_USER</code>	만들 관리자 계정 이름	기본값 <code>postgres</code> 사용
<code>POSTGRES_DB</code>	처음 만들 기본 데이터베이스 이름	<code>POSTGRES_USER</code> 와 같은 이름으로 생성
<code>-e 이름=값</code>	컨테이너에 환경변수를 전달하는 옵션	(값을 안 주는 셈)

주의: `POSTGRES_USER` 와 `POSTGRES_DB` 는 데이터가 비어 있는 **첫 기동에만** 적용됩니다. 이미 데이터가 든 볼륨으로 다시 띄우면서 이 값을 바꿔도 새 계정이나 새 데이터베이스가 생기지 않습니다. 변경을 반영하려면 SQL로 직접 만들거나, 데이터를 비우고 처음부터 다시 만들어야 합니다 (다음 절 참고).

팁: `POSTGRES_USER` 만 지정하고 `POSTGRES_DB` 를 빼면, 그 사용자와 같은 이름의 데이터베이스가 자동으로 하나 만들어집니다. 예를 들어 `POSTGRES_USER=appuser` 만 주면 `appuser` 라는 데이터베이스가 생깁니다. 사용자와 데이터베이스 이름을 다르게 하고 싶을 때만 `POSTGRES_DB` 를 따로 적으면 됩니다.

절 요약

- `-e 이름=값` 으로 컨테이너에 환경변수를 전달합니다.
- `POSTGRES_PASSWORD` 는 필수, `POSTGRES_USER` · `POSTGRES_DB` 는 선택이며 생략 시 기본값이 쓰입니다.
- 이 값들은 데이터가 비어 있는 첫 기동에만 적용됩니다.
- 만들어진 결과는 `docker exec ... psql` 로 접속해 확인할 수 있습니다.

초기화 스크립트: `/docker-entrypoint-initdb.d`

도입

환경변수로는 계정과 데이터베이스까지 만들 수 있었습니다. 그런데 테이블을 만들고, 시작용 데이터를 몇 줄 넣어 두는 일은 어떻게 할까요? 컨테이너를 켤 때마다 손으로 SQL을 치기는 번거롭습니다. 공식 postgres 이미지는 이를 위해 **첫 기동 때 자동으로 SQL을 실행해 주는 특별한 폴더**를 제공합니다. 이 절에서는 그 폴더 `/docker-entrypoint-initdb.d` 를 배웁니다.

핵심 개념 설명

초기화 스크립트 (init script)와 엔트리포인트(entrypoint)

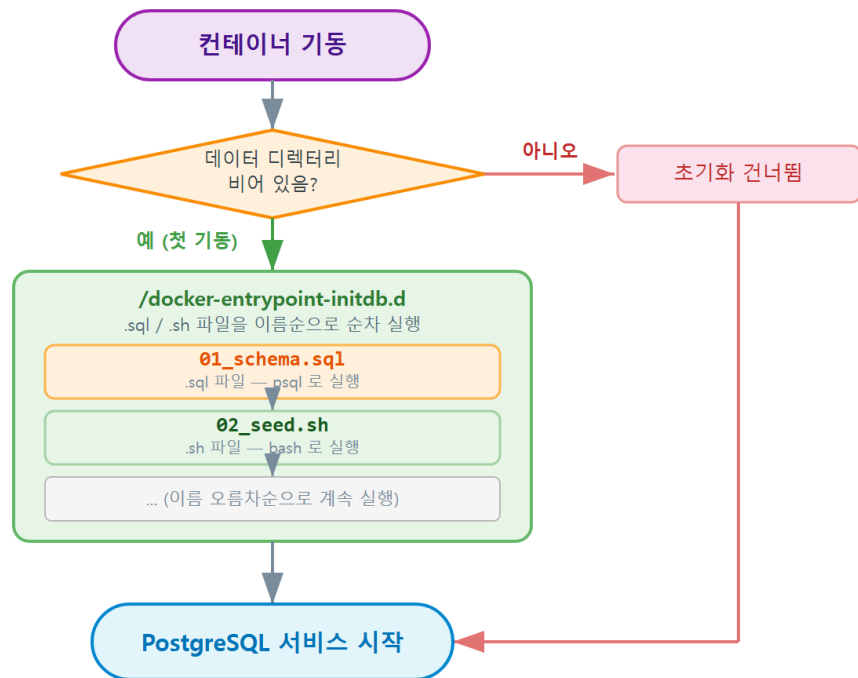
공식 postgres 이미지 안에는 컨테이너가 켜질 때 제일 먼저 실행되는 시작 프로그램이 들어 있습니다. 이 시작 프로그램을 **엔트리포인트(entrypoint)** 라고 부릅니다. "건물에 들어설 때 반드시 지나는 정문"과 같은 역할입니다.

이 엔트리포인트는 데이터베이스를 처음 준비할 때 `/docker-entrypoint-initdb.d` 라는 폴더를 들여다봅니다. 그 안에 들어 있는 `.sql` 이나 `.sh` (셸 스크립트) 파일을 **이름 순서대로 자동 실행**합니다. 이렇게 첫 기동 때 자동 실행되는 파일을 **초기화 스크립트(init script)** 라고 합니다. 새 기기를 처음 켤 때 한 번 도는 초기 설정 마법사와 같습니다.

- **왜(why) 필요한가:** 테이블 생성·시드 데이터 입력을 컨테이너 기동과 한 묶음으로 자동화 하면, 띄우기만 해도 바로 쓸 수 있는 데이터베이스가 준비됩니다.
- **언제(when) 쓰나:** 실습·개발 환경에서 "준비된 상태"의 DB가 매번 필요할 때, 또는 동료에게 같은 초기 데이터를 나눠 줄 때 씁니다.
- **어떻게(how) 쓰나:** 호스트(내 PC)에 만든 SQL 파일이 든 폴더를, 컨테이너의 `/docker-entrypoint-initdb.d` 경로에 마운트(연결)해 주면 됩니다.

여기서 마운트는 5장에서 배운 그 개념입니다. 내 PC의 폴더를 컨테이너 안 특정 경로에 그대로 비춰 주는 것입니다. 초기화 스크립트는 호스트 폴더를 컨테이너의 init 폴더에 **바인드 마운트(bind mount)** 해서 넣습니다.

초기화 스크립트 실행 흐름 — /docker-entrypoint-initdb.d



데이터 디렉터리가 비어 있을 때만(최초 기동 시) 초기화 단계가 실행됩니다

ch_06_s02 · 초기화 스크립트: /docker-entrypoint-initdb.d

이렇게 설정/실행합니다

먼저 내 PC에 초기화용 폴더와 SQL 파일을 하나 만듭니다. 예시로 C:\docker\initdb 폴더에 01_schema.sql 을 둡니다.

```

-- C:\docker\initdb\01_schema.sql
-- 첫 기동 때 자동 실행될 초기화 스크립트입니다.
CREATE TABLE products (
  id    SERIAL PRIMARY KEY,
  name  TEXT NOT NULL,
  price INTEGER NOT NULL
);

INSERT INTO products (name, price) VALUES
  ('keyboard', 30000),
  ('mouse', 15000);
    
```

이제 이 폴더를 컨테이너의 /docker-entrypoint-initdb.d 에 연결해 컨테이너를 띄웁니다.

```
PS C:\Users\me> docker run -d --name pg16 `
-e POSTGRES_PASSWORD=mysecret `
-e POSTGRES_USER=appuser `
-e POSTGRES_DB=shop `
-v C:\docker\initdb:/docker-entrypoint-initdb.d `
-p 5432:5432 postgres:16
```

컨테이너가 첫 기동하면서 폴더 안 SQL을 실행했는지는 로그로 확인합니다.

```
PS C:\Users\me> docker logs pg16
```

예상 화면:

```
/usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/01_schema.sql
CREATE TABLE
INSERT 0 2
PostgreSQL init process complete; ready for start up.
LOG:  database system is ready to accept connections
```

running /docker-entrypoint-initdb.d/01_schema.sql 과 CREATE TABLE, INSERT 0 2 가 보이면 스크립트가 자동 실행된 것입니다. 정말 테이블과 데이터가 들어갔는지 확인해 봅시다.

```
PS C:\Users\me> docker exec -it pg16 psql -U appuser -d shop -c "SELECT * FROM products;"
```

예상 화면:

```
id | name   | price
---+-----+-----
 1 | keyboard | 30000
 2 | mouse   | 15000
(2 rows)
```

띄우기만 했는데 테이블과 데이터가 준비되어 있습니다.

초기화 관련 항목 설명

항목	의미
<code>/docker-entrypoint-initdb.d</code>	첫 기동 때 자동 실행되는 스크립트를 두는 컨테이너 안 폴더
<code>-v C:\docker\initdb:/docker-entrypoint-initdb.d</code>	내 PC 폴더를 init 폴더에 연결(바인드 마운트)
<code>.sql</code> 파일	psql로 실행되는 SQL 스크립트
<code>.sh</code> 파일	셸 명령으로 실행되는 스크립트(더 복잡한 준비 작업용)
실행 순서	파일 이름 순(<code>01_</code> , <code>02_</code> 처럼 번호를 붙여 순서 제어)

팁: 여러 스크립트를 정해진 순서로 돌리고 싶다면 `01_schema.sql`, `02_seed.sql` 처럼 앞에 번호를 붙입니다. 엔트리포인트가 파일 이름을 사전 순으로 정렬해 실행하므로, 번호가 곧 실행 순서가 됩니다. 테이블을 먼저 만들고 데이터를 나중에 넣는 식의 의존 순서를 지킬 때 유용합니다.

주의: 초기화 스크립트는 **데이터 디렉터리가 비어 있는 첫 기동에만** 실행됩니다. 이미 데이터가 있는 볼륨을 붙이면 이 폴더의 스크립트는 통째로 무시됩니다. 이 동작이 바로 다음 절에서 다룰 "최초 1회만" 함정의 핵심입니다.

절 요약

- 엔트리포인트는 첫 기동 때 `/docker-entrypoint-initdb.d` 안의 `.sql`·`.sh` 를 이름 순으로 자동 실행합니다.
- 호스트 폴더를 이 경로에 마운트해 테이블 생성·시드 데이터를 자동화합니다.
- 파일 이름 앞에 번호를 붙여 실행 순서를 제어합니다.
- 초기화 스크립트는 데이터가 비어 있는 첫 기동에만 작동합니다.

"초기화는 최초 1회만" 함정과 볼륨의 관계

도입

앞 두 절에서 거듭 "첫 기동에만 적용된다"는 경고를 봤습니다. 이 절은 그 경고의 정체를 정면으로 다룹니다. 입문자가 가장 많이 막히는 지점이 바로 여기입니다. "초기화 스크립트를 고쳤는데 아무리 다시 띄워도 반영이 안 된다"는 상황을 일부러 재현해 보고, 왜 그런지와 어떻게 푸는지를 배웁니다.

핵심 개념 설명

최초 1회 실행 (first-run only)

최초 1회 실행(first-run only) 이란, 초기화 스크립트와 `POSTGRES_USER` · `POSTGRES_DB` 같은 값이 데이터 디렉터리가 비어 있을 때만 적용되고, 데이터가 이미 차 있으면 건너뛰는 동작입니다. 새 공책에만 표지를 인쇄하고, 이미 글을 쓴 공책에는 다시 인쇄하지 않는 것과 같습니다.

판단 기준은 컨테이너의 데이터 저장 경로인 `/var/lib/postgresql/data` (5장에서 배운 데이터 디렉터리)가 **비어 있는**가입니다.

- 비어 있으면: 엔트리포인트가 "처음 만드는 데이터베이스구나" 하고 환경변수로 계정·DB를 만들고, `init` 폴더의 스크립트를 실행합니다.
- 이미 차 있으면: "이미 만들어진 데이터베이스구나" 하고 초기화 단계를 **통째로 건너뛰고** 곧바로 PostgreSQL을 켵니다.

볼륨과의 관계 — 왜 반영이 안 되는가

여기서 5장의 볼륨이 다시 등장합니다. **네임드 볼륨(named volume)** 을 붙여 컨테이너를 띄우면, 데이터는 컨테이너 바깥의 볼륨에 남습니다. 그래서 컨테이너를 `docker rm` 으로 지우고 새로 만들어도, **같은 볼륨을 다시 붙이는 한 데이터 디렉터리는 비어 있지 않습니다.**

결과적으로 이런 흐름이 됩니다. 초기화 스크립트를 고쳐도 → 볼륨에 옛 데이터가 남아 있어 → 데이터 디렉터리가 비어 있지 않다고 판단 → 초기화 단계를 건너뛰 → 고친 스크립트가 실행되지 않음. "분명 SQL을 바꿨는데 새 테이블이 안 생긴다"는 호소의 정체가 바로 이것입니다.

이렇게 설정/실행합니다

함정을 직접 재현해 봅시다. 앞 절에서 만든 `01_schema.sql` 에 테이블을 하나 더 추가했다고 합시다.

```
-- C:\docker\initdb\01_schema.sql (수정함: orders 테이블 추가)
CREATE TABLE products (
  id    SERIAL PRIMARY KEY,
  name  TEXT NOT NULL,
  price INTEGER NOT NULL
);

CREATE TABLE orders (
  id          SERIAL PRIMARY KEY,
  product_id  INTEGER REFERENCES products(id)
);
```

이미 볼륨을 쓰고 있던 컨테이너를 지우고, 같은 볼륨으로 다시 띄웁니다.

```
PS C:\Users\me> docker rm -f pg16
PS C:\Users\me> docker run -d --name pg16 `
-e POSTGRES_PASSWORD=mysecret -e POSTGRES_USER=appuser -e POSTGRES_DB=shop `
-v pgdata:/var/lib/postgresql/data `
-v C:\docker\initdb:/docker-entrypoint-initdb.d `
-p 5432:5432 postgres:16
```

새로 추가한 `orders` 테이블이 생겼는지 확인합니다.

```
PS C:\Users\me> docker exec -it pg16 psql -U appuser -d shop -c "\dt"
```

예상 화면:

```

      List of relations
 Schema | Name      | Type  | Owner
-----+-----+-----+-----
 public | products | table | appuser
(1 row)
```

`orders` 가 없습니다. 로그를 보면 이유가 분명히 적혀 있습니다.

```
PostgreSQL Database directory appears to contain a database; Skipping initialization
LOG:  database system is ready to accept connections
```

Skipping initialization, 즉 "초기화를 건너뛴"이라는 줄이 핵심입니다. 볼륨에 옛 데이터가 남아 있어 초기화 단계 전체를 지나친 것입니다.

해결: 볼륨을 초기화한다

고친 스크립트를 반영하려면 데이터 디렉토리를 진짜로 비워야 합니다. 즉 **볼륨을 지우고 처음부터 다시** 만듭니다.

```
PS C:\Users\me> docker rm -f pg16
PS C:\Users\me> docker volume rm pgdata
PS C:\Users\me> docker run -d --name pg16 `
-e POSTGRES_PASSWORD=mysecret -e POSTGRES_USER=appuser -e POSTGRES_DB=shop `
-v pgdata:/var/lib/postgresql/data `
-v C:\docker\initdb:/docker-entrypoint-initdb.d `
-p 5432:5432 postgres:16
```

이제 볼륨이 비어 있으므로 초기화가 다시 돌고, **orders** 테이블까지 생깁니다.

```

      List of relations
 Schema |   Name   | Type  | Owner
-----+-----+-----+-----
 public | orders   | table | appuser
 public | products | table | appuser
(2 rows)
```

상황과 처리 설명

상황/명령	의미
Skipping initialization 로그	데이터가 이미 있어 초기화를 건너뛰었다는 신호
docker volume rm pgdata	볼륨을 삭제해 데이터 디렉토리를 진짜로 비움
볼륨 삭제 후 재기동	디렉터리가 비어 다시 초기화가 실행됨
docker volume ls	남아 있는 볼륨 목록을 확인

위험: `docker volume rm` 은 그 볼륨에 든 **모든 데이터를 영구히 지웁니다**. 실습용 데이터라면 괜찮지만, 실제로 지키고 싶은 데이터가 든 볼륨에는 절대 쓰면 안 됩니다. 운영 중인 데이터베이스에서 스키마를 바꿔야 한다면 볼륨을 지우는 대신, psql로 `ALTER TABLE` · `CREATE TABLE` 같은 SQL을 직접 실행해 변경합니다. 초기화 스크립트 재실행은 어디까지나 "처음부터 다시 만들어도 되는" 환경에서만 쓰는 방법입니다.

팁: 실습 중 초기화를 자주 다시 돌려야 한다면, 컨테이너 이름과 볼륨 이름을 정해 두고 "지우고 다시 만들기"를 한 묶음으로 메모해 두면 편합니다. 8장에서 배울 `docker compose down -v` 는 컨테이너와 볼륨을 한 번에 정리해, 이 "초기화 다시 돌리기"를 훨씬 간단하게 만들어 줍니다.

절 요약

- 초기화는 데이터 디렉터리가 비어 있는 최초 1회에만 실행됩니다.
- 볼륨에 옛 데이터가 남아 있으면 스크립트를 고쳐도 `Skipping initialization` 으로 건너뛸
니다.
- 고친 스크립트를 반영하려면 볼륨을 지우고 처음부터 다시 만들어야 합니다.
- `docker volume rm` 은 데이터를 영구 삭제하므로 실습 환경에서만 신중히 씁니다.

.env 파일과 비밀번호 다루기 첫걸음

도입

지금까지 우리는 비밀번호를 `-e POSTGRES_PASSWORD=mysecret` 처럼 명령에 그대로 적었습니다. 학습할 때는 편하지만, 실제로는 위험한 습관입니다. 명령 기록에 비밀번호가 통째로 남고, 그 명령을 공유하면 비밀번호까지 함께 새어 나가기 때문입니다. 이 절에서는 비밀번호를 명령에서 떼어 내 `.env` 파일에 분리하는 첫걸음을 배웁니다.

핵심 개념 설명

.env 파일 (dotenv)과 비밀번호 분리(secret separation)

`.env` 파일(dotenv) 은 비밀번호 같은 설정값을 명령·코드와 분리해 따로 담아 두는 파일입니다. 비밀번호를 소스 코드에 적지 않고 따로 잠금 메모지에 보관하는 것과 같습니다. 파일 이름은 점(.)으로 시작하는 그대로 `.env` 이며, 한 줄에 `이름=값` 형태로 적습니다.

- **왜(why) 필요한가:** 비밀번호가 명령·코드·기록에 흩어져 남지 않게 한곳에 모으면, 관리와 보호가 쉬워집니다. 값이 바뀌어도 `.env` 한 곳만 고치면 됩니다.
- **언제(when) 쓰나:** 비밀번호·접속 정보처럼 외부에 노출되면 안 되는 값을 다룰 때 씁니다.
- **어떻게(how) 쓰나:** `.env` 에 값을 적고, `docker run` 에 `--env-file` 옵션으로 그 파일을 가리키게 합니다.

.env는 자동으로 읽히지 않는다

입문자가 흔히 오해하는 점이 있습니다. `.env` 파일을 만들어 두기만 하면 `docker run` 이 알아서 읽어 줄 것이라 기대하지만, 그렇지 않습니다. `docker run` 은 `--env-file` 로 명시해 줘야 `.env` 를

읽습니다. (8장에서 배울 `docker compose` 는 같은 폴더의 `.env` 를 자동으로 읽어 주는데, 이는 `compose`만의 동작입니다. 둘을 헷갈리지 않도록 지금 구분해 둡니다.)

또 하나 중요한 원칙은 `.env` 를 **git에 올리지 않는 것**입니다. git은 코드 변경 이력을 저장·공유하는 도구인데, 비밀번호가 든 `.env` 를 함께 올리면 비밀번호가 저장소에 영구히 남아 누구나 볼 수 있게 됩니다. 그래서 `.gitignore` 라는 파일에 `.env` 를 적어 "이 파일은 git이 추적하지 말라"고 알려 줍니다.

이렇게 설정/실행합니다

먼저 프로젝트 폴더에 `.env` 파일을 만들어 값을 적습니다.

```
# C:\docker\.env
POSTGRES_PASSWORD=mysecret
POSTGRES_USER=appuser
POSTGRES_DB=shop
```

이제 비밀번호를 명령에 직접 쓰지 않고, `--env-file` 로 이 파일을 가리켜 컨테이너를 띄웁니다.

```
PS C:\docker> docker run -d --name pg16 `
  --env-file .env `
  -v pgdata:/var/lib/postgresql/data `
  -p 5432:5432 postgres:16
```

명령 어디에도 비밀번호가 보이지 않습니다. 값은 `.env` 에서 읽어 들어갑니다. 잘 적용됐는지 확인합니다.

```
PS C:\docker> docker exec -it pg16 psql -U appuser -d shop -c "\conninfo"
```

예상 화면:

```
You are connected to database "shop" as user "appuser" via socket in "/var/run/postgresql" at po
```

다음으로, `.env` 가 실수로 git에 올라가지 않도록 같은 폴더에 `.gitignore` 파일을 만들어 한 줄 적어 둡니다.

```
# C:\docker\.gitignore
.env
```

파일과 옵션 설명

항목	의미
<code>.env</code>	비밀번호 등 설정값을 이름=값 으로 모아 둔 파일
<code>--env-file .env</code>	그 파일의 값을 환경변수로 컨테이너에 전달
<code>.gitignore</code>	git이 추적하지 말 파일 목록
<code>.gitignore</code> 에 적은 <code>.env</code>	<code>.env</code> 를 저장소에 올리지 않도록 제외

위험: `.env` 를 한 번이라도 git에 올렸다면, 그 뒤에 `.gitignore` 에 추가해도 이미 올라간 기록에는 비밀번호가 남아 있습니다. git은 과거 이력을 보존하기 때문입니다. 따라서 새 프로젝트라면 처음부터 `.gitignore` 에 `.env` 를 넣어 두는 것이 가장 안전합니다. 비밀번호가 실수로 올라갔다면 그 비밀번호는 노출된 것으로 보고 새 값으로 바꿔야 합니다.

팁: `.env` 는 올리지 않더라도, 동료가 무슨 값을 채워야 할지 알 수 있게 `.env.example` 처럼 값은 비운 견본 파일을 함께 두는 방식이 흔히 쓰입니다. 예를 들어 `POSTGRES_PASSWORD=` 만 적어 두면, 받는 사람이 이를 복사해 `.env` 로 만든 뒤 자기 값을 채워 넣을 수 있습니다. 이 견본 파일은 비밀이 없으므로 git에 올려도 됩니다.

절 요약

- `.env` 는 비밀번호 등 설정값을 명령에서 분리해 모아 두는 파일입니다.
- `docker run` 은 `--env-file` 로 명시해야 `.env` 를 읽습니다(자동 아님).
- `.gitignore` 에 `.env` 를 적어 비밀번호가 저장소에 올라가지 않게 합니다.
- 한 번 올라간 비밀번호는 기록에 남으므로 처음부터 제외하는 것이 안전합니다.

실습 과제

해보기: 환경변수와 초기화 스크립트로 자동 구성되는 컨테이너 만들기

비밀번호는 `.env` 로 분리하고, 초기화 스크립트로 테이블과 시드 데이터가 자동 준비되는 컨테이너를 만들어 봅니다. 이어 "초기화는 최초 1회만" 함정도 직접 재현하고 풀어 봅니다.

- 단계 1: `C:\docker` 폴더를 만들고 그 안에 `.env` (`POSTGRES_PASSWORD·USER·DB`)와 `.gitignore` (`.env` 한 줄)를 만듭니다.
- 단계 2: `C:\docker\initdb\01_schema.sql` 에 테이블 하나와 시드 데이터 두 줄을 적습니다.

- 단계 3: `--env-file .env`, `-v pgdata:/var/lib/postgresql/data`, `-v C:\docker\initdb:/docker-entrypoint-initdb.d` 를 모두 붙여 컨테이너를 띄웁니다.
- 단계 4: `docker logs` 에서 `running ...01_schema.sql` 을 찾고, `docker exec ... psql -c "\dt"` 로 테이블이 생겼는지 확인합니다.
- 단계 5: `01_schema.sql` 에 테이블을 하나 더 추가한 뒤, 컨테이너만 지우고 같은 볼륨으로 다시 띄워 봅니다. 새 테이블이 안 생기고 로그에 `Skipping initialization` 이 뜨는 함정을 확인합니다.
- 단계 6: `docker volume rm pgdata` 로 볼륨을 비운 뒤 다시 띄워, 이번에는 새 테이블까지 생기는지 확인합니다.
- 점검: 비밀번호가 어느 명령에도 직접 적히지 않았는지, 그리고 단계 5와 단계 6에서 테이블 목록이 어떻게 달라졌는지 한 줄로 정리해 봅니다.

FAQ

Q1. `POSTGRES_DB` 를 바꿔서 다시 띄웠는데 새 데이터베이스가 안 생깁니다. 왜 그런가요? → A1. 환경변수 `POSTGRES_USER` · `POSTGRES_DB` 는 데이터 디렉터리가 비어 있는 첫 기동에만 적용되기 때 문입니다. 이미 데이터가 든 볼륨을 붙이면 이 값들은 무시됩니다. 새 데이터베이스가 필요하면 `psql`로 `CREATE DATABASE` 를 직접 실행하거나, 실습 환경이라면 볼륨을 비우고 처음부터 다시 만 듭니다.

Q2. 초기화 스크립트를 고쳤는데 반영이 안 됩니다. 어떻게 하나요? → A2. 같은 이유입니다. 초 기화 스크립트도 데이터 디렉터리가 비어 있을 때만 실행됩니다. 볼륨에 옛 데이터가 남아 있으 면 로그에 `Skipping initialization` 이 뜨며 건너뜀니다. 반영하려면 `docker rm -f` 로 컨테이너를 지우고 `docker volume rm` 으로 볼륨까지 비운 뒤 다시 띄웁니다. 단, 볼륨 삭제는 데이터를 영구 히 지우므로 실습 환경에서만 합니다.

Q3. `.env` 파일을 만들었는데 `docker run` 이 안 읽는 것 같습니다. → A3. `docker run` 은 `.env` 를 자동으로 읽지 않습니다. `--env-file .env` 처럼 옵션으로 명시해야 합니다. 같은 폴더의 `.env` 를 자동으로 읽는 것은 8장에서 배울 `docker compose` 의 동작이라 서로 다릅니다. 또한 `--env-file` 경로가 현재 폴더 기준으로 맞는지도 확인하세요.

Q4. 초기화 스크립트로는 테이블을 만들 수 있는데, 운영 중 스키마 변경도 이렇게 하면 되나 요? → A4. 아닙니다. 초기화 스크립트는 "처음부터 다시 만들어도 되는" 환경에서만 쓰는 방법 입니다. 운영 중인 데이터베이스는 데이터를 지키면서 바꿔야 하므로, 볼륨을 비우는 대신 `psql` 로 `ALTER TABLE` · `CREATE TABLE` 같은 SQL을 직접 실행해 변경합니다.

장 요약

이 장에서는 컨테이너를 켜는 것만으로 준비된 PostgreSQL이 뜨도록 구성하는 법을 배웠습니다. `-e` 로 전달하는 `POSTGRES_PASSWORD` · `POSTGRES_USER` · `POSTGRES_DB` 환경변수로 비밀번호·관리자 계정·기본 데이터베이스를 정했고, `/docker-entrypoint-initdb.d` 에 SQL을 넣어 테이블과 시드 데이터까지 첫 기동에 자동 적용했습니다. 무엇보다 "초기화는 최초 1회만"이라는 핵심 함정을 직접 재현하며, 볼륨에 옛 데이터가 남으면 `Skipping initialization` 으로 초기화를 건너뛰는 점과 볼륨을 비워야 다시 적용된다는 점을 익혔습니다. 끝으로 비밀번호를 `.env` 로 분리하고 `.gitignore` 로 보호하는 안전 습관의 첫걸음을 뒀습니다. 다음 장에서는 이렇게 구성한 컨테이너 안으로 직접 들어가, 운영에 필요한 명령들을 다뤄 봅니다.

핵심 키워드

환경변수(environment variable), `-e`, `POSTGRES_PASSWORD`, `POSTGRES_USER`, `POSTGRES_DB`, 초기화 스크립트(init script), `/docker-entrypoint-initdb.d`, 엔트리포인트(entrypoint), 최초 1회 실행(first-run only), `Skipping initialization`, `.env` 파일(dotenv), `--env-file`, `.gitignore`

다음 장 미리보기

다음 장에서는 운영 중인 컨테이너를 안에서 다루고 살펴보는 명령을 배웁니다. `docker exec` 로 컨테이너 안 `psql`과 셸에 들어가고, `docker logs` 로 기동·오류 메시지를 추적하며, `docker inspect` · `stats` 와 헬스체크로 컨테이너 상태와 자원을 점검하는 운영 점검 루틴을 익힙니다.

컨테이너 운영 명령: 안으로 들어가 다루고 살펴보기

학습 목표 - docker exec로 컨테이너 안의 psql을 사용해 SQL을 실행할 수 있습니다. - 대화형 셸로 컨테이너 내부 파일 시스템을 둘러볼 수 있습니다. - docker logs로 기동·오류 메시지를 추적할 수 있습니다. - docker inspect·stats와 헬스체크로 컨테이너 상태를 점검할 수 있습니다.

지금까지 우리는 컨테이너를 띄우고(3장), 호스트에서 접속하고(4장), 데이터를 영속화하고(5장), 환경변수로 구성하는(6장) 법을 배웠습니다. 컨테이너는 잘 떠 있습니다. 그런데 막상 운영을 시작하면 새로운 질문이 생깁니다. "안에서 SQL을 한 번 돌려 보고 싶은데?", "PostgreSQL이 왜 안 뜨지? 무슨 메시지를 남겼을까?", "이 컨테이너가 지금 포트를 어떻게 열고 있더라?"

이 장은 그런 질문에 답하는 **운영 도구 상자**입니다. 컨테이너 안으로 들어가 다루는 `docker exec`, 작동 일지를 읽는 `docker logs`, 설정과 자원을 들여다보는 `docker inspect`와 `docker stats`, 그리고 컨테이너가 정말 준비됐는지 알려 주는 **헬스체크(healthcheck)** 까지 차례로 익힙니다. 코드를 새로 만드는 장이 아니라, 이미 떠 있는 컨테이너를 **살펴보고 손보는** 장입니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 을 기준으로 합니다. 아래 예시에서 `PS C:\Users\me>` 는 "여러분이 직접 입력하는 자리"를 뜻하고, 그 아래 줄들은 Docker나 PostgreSQL이 돌려주는 출력입니다. 이 장의 실습은 3장에서 띄운 `pg16` 컨테이너(또는 6장에서 구성한 컨테이너)가 **실행 중**이라는 전제에서 진행합니다. `docker ps` 로 `Up ...` 상태인지 먼저 확인해 주세요.

용어 미리 보기: **셸(shell)** 은 글자로 명령을 주고받는 창구입니다. 리눅스 컨테이너 안에서 흔히 쓰는 셸이 `bash` (배시)입니다. **표준 출력(standard output)** 은 프로그램이 화면에 내보내는 글자 줄들을 가리키는데, 컨테이너는 이 출력을 모아 로그로 보관합니다.

docker exec로 컨테이너 안 psql 사용하기

도입

PostgreSQL 컨테이너에 데이터가 잘 들어갔는지 SQL로 직접 확인하고 싶을 때가 있습니다. 4장에서는 호스트에 설치한 `psql`이나 `DBeaver`로 바깥에서 접속했습니다. 그런데 호스트에 `psql`을 따로 깔지 않았다면 어떻게 할까요? 사실 그럴 필요가 없습니다. **컨테이너 안에는 이미 `psql`이 들어 있기 때문입니다.** 이 절에서는 실행 중인 컨테이너 안에서 명령을 돌리는 `docker exec`로, 컨테이너 안의 `psql`을 꺼내 쓰는 법을 배웁니다.

핵심 개념 설명

`docker exec` (실행 중인 컨테이너 안에서 명령 실행하기)

`docker exec`는 이미 실행 중인 컨테이너 안에서 새 명령을 실행하게 해 주는 명령입니다. 이미 돌아가고 있는 작업실의 문을 열고 들어가, 그 안에서 일을 시키는 것과 같습니다.

- **왜(why) 필요한가:** 컨테이너는 호스트와 격리되어 있어, 바깥 PowerShell에서 친 명령은 컨테이너 안에 닿지 않습니다. 컨테이너 안의 `psql`이나 파일을 다루려면 "안으로 들어가는" 통로가 필요한데, 그 통로가 `docker exec`입니다.
- **언제(when) 쓰나:** 컨테이너 안 `psql`로 SQL을 돌려 볼 때, 안의 파일을 확인할 때, 안에서만 쓸 수 있는 도구(`pg_dump` 등)를 실행할 때 씁니다.
- **어떻게(how) 쓰나:** `docker exec` [옵션] 컨테이너이름 실행할명령 형태입니다. 예를 들어 `docker exec -it pg16 psql -U postgres`입니다.

여기서 입문자가 자주 헷갈리는 점이 두 가지 있습니다. 첫째, `docker exec`는 **새 컨테이너를 띄우지 않습니다.** 이미 떠 있는 컨테이너 안에서 명령만 더 돌립니다(새로 띄우는 것은 `docker run`입니다). 둘째, **컨테이너가 실행 중(up)이어야 합니다.** 정지된 컨테이너에는 들어갈 수 없으므로, 먼저 `docker start`로 켜 두어야 합니다.

`-it` 옵션 (대화형으로 들어가기)

`docker exec` 뒤의 `-it`는 두 옵션을 붙여 쓴 것입니다. `-i`는 "내가 키보드로 입력하는 것을 컨테이너 안 프로그램에 계속 전달하라", `-t`는 "사람이 보기 좋은 터미널 화면을 붙여 달라"는 뜻입니다. `psql`처럼 명령을 주고받으며 **대화하듯** 쓰는 프로그램에는 이 `-it`가 거의 항상 필요합니다. `-it` 없이 실행하면 `psql` 프롬프트가 떠도 입력이 전달되지 않아 곧 끊깁니다.

컨테이너 안 `psql`과 PostgreSQL 교재의 연결

여기서 중요한 사실 하나를 짚겠습니다. **호스트(내 PC)에 psql을 한 번도 깔지 않았어도, 컨테이너 안의 psql로 PostgreSQL을 그대로 다룰 수 있습니다.** 공식 postgres 이미지에는 PostgreSQL 서버뿐 아니라 psql 클라이언트도 함께 들어 있기 때문입니다.

그래서 별도 교재인 PostgreSQL CRUD 교재에서 익힌 **메타 명령(meta-command)** 이 컨테이너 안에서도 똑같이 통합니다. 메타 명령이란 psql 안에서 백슬래시(\)로 시작하는 편의 명령으로, 데이터베이스 목록을 보는 \l (엘), 테이블 목록을 보는 \dt, 도움말 \? 등이 있습니다. 컨테이너라고 해서 새로운 접속법이나 새로운 명령을 배울 필요가 없다는 뜻입니다. "Docker는 PostgreSQL을 담는 상자일 뿐, 그 안의 PostgreSQL을 다루는 법은 똑같다"고 기억하면 됩니다.

이렇게 설정/실행합니다

다음은 실행 중인 pg16 컨테이너 안의 psql 로 들어가 PostgreSQL에 접속하는 명령입니다.

```
PS C:\Users\me> docker exec -it pg16 psql -U postgres
```

예상 화면:

```
psql (16.x (Debian 16.x-1.pgdg120+1))
Type "help" for help.

postgres=#
```

프롬프트가 postgres=# 으로 바뀌었습니다. 지금 우리는 컨테이너 안 PostgreSQL에 관리자 (postgres)로 접속한 상태입니다. 여기서 PostgreSQL CRUD 교재에서 쓰던 메타 명령을 그대로 써 봅니다.

```
postgres=# \l
                                List of databases
   Name   | Owner   | Encoding | Collate  | Ctype    | Access privileges
-----+-----+-----+-----+-----+-----
 postgres | postgres | UTF8     | en_US.utf8 | en_US.utf8 |
 template0 | postgres | UTF8     | en_US.utf8 | en_US.utf8 | =c/postgres +
 template1 | postgres | UTF8     | en_US.utf8 | en_US.utf8 | =c/postgres +
(3 rows)

postgres=# \dt
Did not find any relations.

postgres=# SELECT version();
                                version
-----
PostgreSQL 16.x (Debian 16.x-1.pgdg120+1) on x86_64-pc-linux-gnu, compiled by gcc ...
(1 row)
```

\l 로 데이터베이스 목록이, \dt 로 (아직 테이블이 없으니) "관계를 찾지 못함"이 그대로 나옵니다. 호스트에 psql을 깔지 않고도 컨테이너 안에서 PostgreSQL을 똑같이 다루고 있는 것입니다. 다 살펴봤으면 \q 로 psql을 빠져나옵니다. 이때 빠져나오는 것은 **psql일 뿐**이고, 컨테이너 자체는 계속 실행 중으로 남습니다.

```
postgres=# \q
PS C:\Users\me>
```

특정 데이터베이스에 바로 붙고 싶으면 -d 옵션으로 데이터베이스 이름을 더해 줍니다. 한 줄 짜리 SQL만 빠르게 돌리고 싶다면 대화형으로 들어가지 않고 -c 옵션으로 곧장 실행할 수도 있습니다.

```
PS C:\Users\me> docker exec -it pg16 psql -U postgres -c "SELECT now();"

```

예상 화면:

```

                                now
-----
2026-06-17 02:14:33.512+00
(1 row)
```

docker exec [-it] 컨테이너이름 psql -U 사용자 [-d 데이터베이스] [-c "SQL"]

명령과 옵션 설명

항목	의미
<code>docker exec</code>	실행 중인 컨테이너 안에서 새 명령을 실행합니다
<code>-it</code>	대화형 터미널을 붙여 입력을 주고받게 합니다(<code>-i</code> + <code>-t</code>)
<code>pg16</code>	명령을 실행할 대상 컨테이너 이름입니다
<code>psql -U postgres</code>	컨테이너 안 <code>psql</code> 로 <code>postgres</code> 사용자로 접속합니다
<code>-d 이름</code>	접속할 데이터베이스를 지정합니다(생략 시 사용자와 같은 이름)
<code>-c "SQL"</code>	대화형으로 들어가지 않고 한 줄 SQL만 실행하고 끝냅니다

팁: `psql` 안의 메타 명령은 PostgreSQL CRUD 교재에서 익힌 것이 그대로 통합니다. 데이터베이스 목록은 `\l`, 현재 DB의 테이블 목록은 `\dt`, 특정 테이블 구조는 `\d 테이블명`, 도움말은 `\?` 입니다. Docker는 환경을 격리해 줄 뿐, SQL과 `psql` 사용법은 네이티브 설치와 똑같습니다.

주의: `docker exec` 는 컨테이너가 실행 중(up)일 때만 됩니다. 정지된 컨테이너에 시도하면 `Error response from daemon: container ... is not running` 오류가 납니다. 이때는 `docker start pg16` 으로 먼저 켜 뒤 다시 시도합니다. 또한 `docker exec` 는 `docker run` 과 달리 **새 컨테이너를 만들지 않는다**는 점을 기억하세요.

절 요약

- `docker exec` 는 실행 중인 컨테이너 안에서 새 명령(`psql` 등)을 실행하는 통로입니다.
- 대화형으로 `psql`을 쓰려면 `-it` 를 붙이고, 한 줄 SQL만 돌리려면 `-c "SQL"` 을 씁니다.
- 호스트에 `psql`을 깔지 않아도 컨테이너 안 `psql`로 PostgreSQL을 다룰 수 있습니다.
- `\l` · `\dt` 같은 메타 명령은 PostgreSQL CRUD 교재에서 익힌 그대로 통합니다.

컨테이너 셀 진입과 내부 둘러보기

도입

`psql` 로 데이터를 다루는 것과 별개로, 컨테이너라는 "작은 리눅스 상자" 자체가 어떻게 생겼는지 궁금할 때가 있습니다. 설정 파일은 어디에 있고, 데이터는 실제로 어느 폴더에 쌓이며, 환경

변수는 안에서 어떻게 보일까요? 이 절에서는 `docker exec` 로 `psql` 이 아니라 **셸(bash)** 을 실행해 컨테이너 안으로 들어가, 내부 파일 시스템을 직접 둘러보는 법을 배웁니다.

핵심 개념 설명

대화형 셸 진입 (`docker exec -it` 컨테이너 `bash`)

앞 절에서 `docker exec -it pg16 psql ...` 로 `psql` 을 실행했습니다. 같은 방식으로 실행할 명령을 `psql` 대신 `bash` 로 바꾸면, `psql` 이 아니라 **리눅스 셸** 이 열립니다. 그러면 컨테이너 안에서 `ls` (목록 보기), `cat` (파일 내용 보기), `cd` (폴더 이동) 같은 리눅스 명령을 직접 칠 수 있습니다.

- **왜(why) 필요한가:** 데이터 디렉터리·설정 파일의 실제 위치를 눈으로 확인하거나, 초기화 스크립트가 잘 들어갔는지 점검할 때, 컨테이너 내부를 직접 봐야 할 때가 있습니다.
- **언제(when) 쓰나:** 6장에서 다룬 `/docker-entrypoint-initdb.d` 에 스크립트가 들어갔는지 확인하거나, 5장의 데이터 디렉터리 `/var/lib/postgresql/data` 를 들여다볼 때 유용합니다.
- **어떻게(how) 쓰나:** `docker exec -it pg16 bash` 로 들어가고, 다 봤으면 `exit` 로 빠져나옵니다.

컨테이너 파일 시스템 (격리된 안쪽 세계)

컨테이너 안은 **호스트(Windows)**와 **완전히 분리된 작은 리눅스 파일 시스템**입니다. 그래서 컨테이너 안의 경로는 Windows의 `C:\Users\...` 가 아니라 리눅스식 `/var/lib/postgresql/data` 처럼 슬래시(/)로 시작합니다. 컨테이너 안에서 무엇을 보든, 그것은 호스트의 파일이 아니라 격리된 컨테이너 내부의 모습이라는 점을 기억해야 합니다.

여기서 입문자가 자주 하는 오해가 있습니다. "컨테이너 안에서 파일을 막 고쳐도 되겠지?" 하는 것입니다. 컨테이너 안에서 직접 고친 변경은 그 컨테이너에만 남고, 컨테이너를 지우면(`docker rm`) 함께 사라집니다(볼륨에 올라간 데이터 디렉터리는 예외입니다). 즉, 셸 진입은 **둘러보고 점검하는 용도**로 쓰고, 영구적인 구성은 환경변수·초기화 스크립트·볼륨(앞 장들)으로 잡는 것이 원칙입니다.

이렇게 설정/실행합니다

다음은 `pg16` 컨테이너 안의 `bash` 셸로 들어가는 명령입니다.

```
PS C:\Users\me> docker exec -it pg16 bash
```

예상 화면:

```
root@3f8a1c5d9e2b:/#
```

프롬프트가 `root@3f8a1c5d9e2b:/#` 으로 바뀌었습니다. 이제 우리는 컨테이너 안 리눅스의 최상위 (`/`) 폴더에 `root` 사용자로 들어와 있습니다. `3f8a1c5d9e2b` 는 컨테이너의 짧은 ID입니다. 데이터 디렉터리와 설정 파일을 둘러봅니다.

```
root@3f8a1c5d9e2b:/# ls /var/lib/postgresql/data
base      pg_commit_ts  pg_hba.conf  pg_stat      postgresql.conf
global    pg_dynshmem   pg_ident.conf pg_subtrans  postmaster.opts
...

root@3f8a1c5d9e2b:/# cat /var/lib/postgresql/data/postgresql.conf | head -n 3
# -----
# PostgreSQL configuration file
# -----
```

`/var/lib/postgresql/data` 가 5장에서 배운 **데이터 디렉터리**입니다. 실제 데이터(`base`)와 설정 파일(`postgresql.conf`, `pg_hba.conf`)이 여기에 모여 있는 것을 눈으로 확인할 수 있습니다. 6장에서 초기화 스크립트를 넣었다면 `/docker-entrypoint-initdb.d` 도 들여다볼 수 있습니다.

```
root@3f8a1c5d9e2b:/# ls /docker-entrypoint-initdb.d
01_schema.sql

root@3f8a1c5d9e2b:/# env | grep POSTGRES
POSTGRES_PASSWORD=mysecret
POSTGRES_DB=appdb
```

`env` 로 컨테이너 안 환경변수도 확인됩니다. 다 둘러봤으면 `exit` 로 셸을 빠져나옵니다.

```
root@3f8a1c5d9e2b:/# exit
exit
PS C:\Users\me>
```

`exit` 를 쳐도 컨테이너는 멈추지 않습니다. 우리가 빠져나온 것은 방금 `docker exec` 로 연 셸 세션일 뿐이고, PostgreSQL 서버는 계속 돌고 있습니다.

셸 안에서 쓰는 리눅스 명령

명령	의미
<code>ls</code> 경로	해당 폴더의 파일·폴더 목록을 봅니다
<code>cat</code> 파일	파일의 내용을 출력합니다
<code>env</code>	컨테이너 안에 설정된 환경변수를 모두 보여 줍니다
<code>exit</code>	셸 세션을 빠져나옵니다(컨테이너는 계속 실행)

주의: 컨테이너 안에서 설정 파일을 직접 고치는 것은 임시방편입니다. 그 변경은 해당 컨테이너에만 남고, 컨테이너를 새로 만들면 사라집니다. 영구적인 구성은 환경변수(6장)·초기화 스크립트(6장)·볼륨(5장)으로 잡아야 재현성이 지켜집니다. 셸 진입은 "둘러보고 점검하는 창"으로 쓰는 것이 좋습니다.

팁: 일부 가벼운 이미지(예: `alpine` 계열)에는 `bash` 가 없을 수 있습니다. 그럴 때는 `docker exec -it pg16 sh` 처럼 더 기본적인 셸 `sh` 로 들어갑니다. 공식 `postgres:16` 이미지는 데비안 기반이라 `bash` 가 들어 있습니다.

절 요약

- `docker exec -it` 컨테이너 `bash` 로 컨테이너 안 리눅스 셸에 들어갑니다.
- 컨테이너 안은 호스트와 격리된 리눅스 파일 시스템이며, 경로는 `/` 로 시작합니다.
- 데이터 디렉터리(`/var/lib/postgresql/data`)·설정 파일·환경변수를 직접 확인할 수 있습니다.
- 셸 진입은 점검용입니다. 영구 구성은 환경변수·스크립트·볼륨으로 잡고, `exit` 해도 컨테이너는 계속 돛니다.

로그 확인: docker logs

도입

컨테이너가 안 뜨거나, 났는데 곧바로 멈출 때 가장 먼저 봐야 할 것이 **로그(log)** 입니다. 자동차에 경고등이 들어오면 계기판부터 보듯, 컨테이너에 문제가 생기면 로그부터 봅니다. 이 절에서는 컨테이너가 남긴 메시지를 읽는 `docker logs` 와, 실시간으로 따라 보는 `-f`, 최근 줄만 보는 `-tail` 옵션을 배웁니다.

핵심 개념 설명

docker logs (표준 출력 로그 읽기)

docker logs 는 컨테이너가 **표준 출력(standard output)** 으로 내보낸 메시지를 모아 보여 주는 명령입니다. 기기가 작동하며 남기는 작동 일지를 들여다보는 것과 같습니다. PostgreSQL 컨테이너는 기동 과정, 접속 준비 완료, 오류 같은 메시지를 표준 출력으로 내보내고, Docker가 이를 차곡차곡 모아 둡니다.

- **왜(why) 필요한가:** 컨테이너가 `docker ps` 에 안 보이거나(곧바로 종료), 접속이 안 될 때 원인은 거의 항상 로그에 적혀 있습니다. 로그를 읽을 줄 알면 문제 해결 속도가 크게 빨라집니다.
- **언제(when) 쓰나:** 컨테이너가 뜨자마자 멈췄을 때, PostgreSQL이 "접속 준비 완료"를 찍었는지 확인할 때, 운영 중 오류를 추적할 때 씁니다.
- **어떻게(how) 쓰나:** `docker logs` 컨테이너이름 입니다. 컨테이너 안에 들어갈 필요 없이, 바깥 PowerShell에서 바로 봅니다.

여기서 입문자가 자주 하는 오해가 있습니다. "로그를 보려면 컨테이너에 들어가 로그 파일을 열어야 한다"고 생각하기 쉽지만, 그렇지 않습니다. `docker logs` 는 바깥에서 컨테이너 안으로 들어가지 않고도 곧장 메시지를 보여 줍니다. 또 하나, `docker logs` 는 **컨테이너가 화면에 내보낸 메시지**를 보여 주는 것이지, PostgreSQL 안의 테이블 데이터를 보여 주는 것이 아닙니다. 데이터는 `psql` (앞 절)로 봅니다.

-f 와 --tail (실시간 추적과 최근 줄만 보기)

로그가 길어지면 전부 보기 부담스럽습니다. 두 옵션이 도움이 됩니다.

- **-f** 는 "follow(따라가기)"의 약자로, 로그를 화면에 띄워 둔 채 **새 메시지가 생길 때마다 실시간으로 이어서** 보여 줍니다. 컨테이너가 지금 무슨 일을 하는지 지켜볼 때 씁니다. 보기를 멈추려면 `ctrl + c` 를 누릅니다. 이때 멈추는 것은 **보기**일 뿐, 컨테이너는 계속 둡니다.
- **--tail 줄수** 는 처음부터 다 보지 않고 **마지막 몇 줄만** 보여 줍니다. `docker logs --tail 20 pg16` 이면 최근 20줄만 나옵니다. 방금 무슨 일이 있었는지 빠르게 확인할 때 편합니다.

이렇게 설정/실행합니다

다음은 `pg16` 컨테이너의 로그를 보는 명령입니다.

```
PS C:\Users\me> docker logs pg16
```

예상 화면:

```
PostgreSQL Database directory appears to contain a database; Skipping initialization

LOG:  starting PostgreSQL 16.x on x86_64-pc-linux-gnu, compiled by gcc ...
LOG:  listening on IPv4 address "0.0.0.0", port 5432
LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
LOG:  database system was shut down at 2026-06-17 02:00:11 UTC
LOG:  database system is ready to accept connections
```

마지막 줄 `database system is ready to accept connections`가 보이면, PostgreSQL이 접속을 받을 준비를 마쳤다는 신호입니다. 컨테이너가 났는데 접속이 안 된다면, 먼저 이 줄이 로그에 있는지부터 확인합니다.

최근 줄만 보고 싶으면 `--tail`을 씁니다.

```
PS C:\Users\me> docker logs --tail 5 pg16
```

예상 화면:

```
LOG:  listening on IPv4 address "0.0.0.0", port 5432
LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
LOG:  database system was shut down at 2026-06-17 02:00:11 UTC
LOG:  database system is ready to accept connections
```

실시간으로 따라 보려면 `-f`를 붙입니다. 이 상태에서 다른 창으로 접속을 시도하면, 그에 따른 로그가 즉시 이어서 찍히는 것을 볼 수 있습니다.

```
PS C:\Users\me> docker logs -f pg16
```

예상 화면:

```
LOG:  database system is ready to accept connections
... (새 메시지가 생기면 여기에 계속 이어집니다. 보기를 멈추려면 Ctrl + C)
```

비밀번호 환경변수를 빠뜨려 컨테이너가 곧바로 멈춘 경우라면, 로그에 그 이유가 분명히 적혀 있습니다.

Error: Database is uninitialized and superuser password is not specified.
You must specify POSTGRES_PASSWORD to a non-empty value for the superuser.

이 메시지를 읽을 줄 알면, 3장에서 배운 "컨테이너가 안 보이는 가장 흔한 원인"을 스스로 진단할 수 있습니다.

`docker logs [-f] [--tail 줄수] 컨테이너이름`

옵션 설명

옵션	의미
<code>docker logs pg16</code>	컨테이너가 남긴 메시지를 처음부터 모두 보여 줍니다
<code>-f</code>	실시간으로 새 메시지를 이어서 보여 줍니다(<code>Ctrl + c</code> 로 보기 종료)
<code>--tail 20</code>	마지막 20줄만 보여 줍니다
<code>-t</code>	각 줄 앞에 타임스탬프(시각)를 함께 표시합니다

팁: `docker logs -f`로 로그를 지켜보면서 다른 PowerShell 창을 하나 더 열어 접속을 시도하면, 접속·종료 메시지가 실시간으로 찍히는 것을 볼 수 있습니다. 문제 재현과 원인 추적에 아주 유용한 조합입니다. 보기를 멈추는 `Ctrl + c`는 로그 보기를 끝낼 뿐 컨테이너를 멈추지 않습니다.

주의: `docker logs`가 보여 주는 것은 컨테이너가 표준 출력으로 내보낸 메시지입니다. PostgreSQL 안의 테이블이나 데이터를 보여 주는 것이 아닙니다. 데이터를 조회하려면 앞 절의 `docker exec ... psql`을 써야 합니다. 둘의 역할을 혼동하지 마세요.

절 요약

- `docker logs` 컨테이너는 컨테이너가 남긴 기동·오류 메시지를 바깥에서 바로 보여 줍니다.
- `database system is ready to accept connections`가 정상 기동의 신호입니다.
- `-f`는 실시간 추적(`Ctrl + c`로 종료), `--tail` 줄수는 최근 줄만 보기입니다.
- 로그는 화면 출력 메시지일 뿐, 테이블 데이터는 `psql`로 봅니다.

상태·자원 점검: `docker inspect / stats / 헬스체크`

도입

컨테이너가 돌고 있다는 것까지는 알았습니다. 그런데 운영을 하다 보면 더 자세한 질문이 생깁니다. "이 컨테이너가 어떤 포트를 열고, 어떤 볼륨을 붙이고, 어떤 환경변수로 뒀더라?", "메모리를 얼마나 쓰고 있지?", "지금 접속을 받을 준비가 정말 됐나?" 이 절에서는 설정을 통째로 들여다보는 `docker inspect`, 자원 사용량을 보는 `docker stats`, 그리고 준비 상태를 알려 주는 **헬스 체크**를 배웁니다. 이 절은 이 장의 도구들을 한데 모아 **운영 점검 루틴**으로 묶는 마무리이기도 합니다.

핵심 개념 설명

docker inspect (설정 통째로 들여다보기)

docker inspect 는 컨테이너의 모든 설정을 자세히 보여 주는 명령입니다. 컨테이너의 신분증과 설계 명세서를 한 번에 펼쳐 보는 것과 같습니다. 포트 매핑, 붙은 볼륨, 환경변수, IP 주소, 시작 명령 등 컨테이너에 관한 거의 모든 정보가 들어 있습니다.

- **왜(why) 필요한가:** "내가 이 컨테이너를 어떤 옵션으로 띄웠더라?"가 기억나지 않을 때, `docker inspect` 가 실제 적용된 설정을 그대로 알려 줍니다. 포트나 볼륨 연결이 의심될 때 진단의 출발점입니다.
- **언제(when) 쓰나:** 포트 매핑이 제대로 됐는지, 볼륨이 데이터 디렉터리에 잘 붙었는지(5장), 환경변수가 의도대로 들어갔는지(6장) 확인할 때 씁니다.
- **어떻게(how) 쓰나:** `docker inspect` 컨테이너이름 이면 전체가 나옵니다. 양이 매우 많으므로, 보통 `--format` 으로 원하는 항목만 뽑아 봅니다.

docker stats (자원 사용량 실시간 보기)

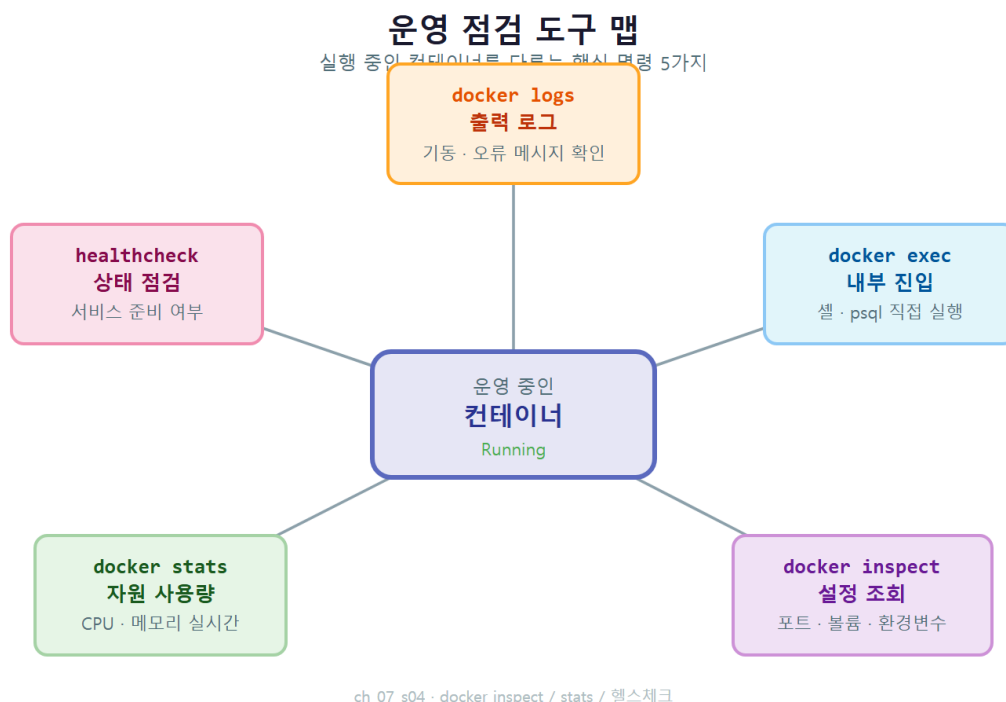
docker stats 는 컨테이너가 지금 CPU·메모리·네트워크를 얼마나 쓰는지 실시간으로 보여 주는 명령입니다. 작업 관리자(Task Manager)의 컨테이너 버전이라고 생각하면 됩니다. 컨테이너가 메모리를 너무 많이 먹는지, 부하가 걸렸는지 한눈에 확인할 때 씁니다. 보기를 멈추려면 `Ctrl + c` 를 누릅니다.

헬스체크 (healthcheck, 준비 상태 점검)

여기서 입문자가 가장 자주 빠지는 함정을 짚겠습니다. **컨테이너가 running (실행 중)이라고 해서, 그 안의 PostgreSQL이 곧바로 접속을 받을 준비가 된 것은 아닙니다.** 컨테이너는 뒀지만 PostgreSQL이 아직 기동 중일 수 있습니다. 이 짧은 시간 차 때문에 "컨테이너는 뒀는데 접속이 거부된다"는 일이 생깁니다.

헬스체크(healthcheck) 는 이 문제를 해결합니다. 컨테이너 안 서비스가 정말로 요청을 받을 준비가 됐는지 **주기적으로 점검해 상태를 보고**하는 장치입니다. 기계가 정상 작동 중인지 알려 주는 초록불 표시등과 같습니다. PostgreSQL은 `pg_isready` 라는 작은 점검 명령을 제공하는데, 이를 헬스체크로 등록하면 컨테이너 상태가 `running` 외에 `(healthy)` (건강함)까지 함께 표시됩니다.

헬스체크는 컨테이너를 띄울 때 옵션으로 등록합니다. `compose` 파일(8장)에서는 더 깔끔하게 선언할 수 있는데, 여기서는 `docker run` 에서의 형태를 미리 맞춥니다.



이렇게 설정/실행합니다

먼저 `docker inspect` 로 포트·볼륨·환경변수 설정을 확인합니다. 전체를 다 보면 양이 많으므로, `--format` 으로 포트 매핑만 뽑아 봅니다.

```
PS C:\Users\me> docker inspect --format '{{json .NetworkSettings.Ports}}' pg16
```

예상 화면:

```
{"5432/tcp":[{"HostIp":"0.0.0.0","HostPort":"5432"}]}
```

컨테이너 내부 `5432` 포트가 호스트 `5432` 포트로 매핑되어 있음을 한 줄로 확인했습니다(4장의 포트 매핑). 환경변수와 마운트(볼륨)도 같은 방식으로 뽑아 봅니다.

```
PS C:\Users\me> docker inspect --format '{{.Config.Env}}' pg16
```

예상 화면:

```
[POSTGRES_PASSWORD=mysecret POSTGRES_DB=appdb PG_MAJOR=16 ... ]
```

이제 `docker stats` 로 자원 사용량을 봅니다.

```
PS C:\Users\me> docker stats pg16
```

예상 화면:

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
3f8a1c5d9e2b	pg16	0.05%	38.2MiB / 7.6GiB	0.49%	1.2kB/820B	0B/0B	7

pg16 이 메모리 약 38MiB를 쓰며 한가하게(CPU 0.05%) 떠 있는 것을 볼 수 있습니다. 한 번만 찍고 끝내려면 `--no-stream` 을 붙이고, 실시간으로 계속 보려면 그냥 두었다가 `ctrl + c` 로 멈춥니다.

다음은 헬스체크를 등록해 컨테이너를 띄우는 예입니다. PostgreSQL이 준비됐는지 `pg_isready` 로 10초마다 점검하게 합니다(기존 pg16 이 있다면 `docker rm -f pg16` 으로 정리 후 실행하거나 다른 이름을 씁니다).

```
PS C:\Users\me> docker run -d --name pg16 `
-e POSTGRES_PASSWORD=mysecret `
--health-cmd="pg_isready -U postgres" `
--health-interval=10s `
--health-timeout=3s `
--health-retries=5 `
postgres:16
```

위에서 줄 끝의 백틱(`)은 PowerShell에서 한 명령을 여러 줄로 나눠 쓰는 줄바꿈 기호입니다. 한 줄로 길게 이어 써도 됩니다.

띄운 직후 `docker ps` 로 상태를 보면, PostgreSQL이 기동하는 동안에는 (health: starting) 으로, 준비가 끝나면 (healthy) 로 바뀝니다.

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
7b2c4e6a8d0f	postgres:16	Up 8 seconds (health: starting)	5432/tcp	pg16

잠시 뒤 다시 확인하면:

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
7b2c4e6a8d0f	postgres:16	Up 25 seconds (healthy)	5432/tcp	pg16

(healthy) 가 보이면 PostgreSQL이 접속을 받을 준비가 됐다는 확실한 신호입니다. `running` 만 보고 접속을 시도하다 거부당하던 문제를, 헬스체크로 깔끔하게 가릴 수 있습니다.

`docker inspect [--format '{{...}}']` 컨테이너이름 `docker stats [--no-stream]` 컨테이너이름

운영 점검 명령 정리

명령	무엇을 보여 주나
<code>docker inspect 이름</code>	포트·볼륨·환경변수 등 컨테이너 설정 전체
<code>docker inspect --format '...'</code>	설정 중 원하는 항목만 골라 출력
<code>docker stats 이름</code>	CPU·메모리·네트워크 실시간 사용량
<code>docker stats --no-stream</code>	사용량을 한 번만 찍고 종료
<code>--health-cmd="pg_isready -U postgres"</code>	준비 상태를 점검할 명령 등록
<code>--health-interval=10s</code>	점검을 얼마나 자주 할지 간격 지정

여기까지 이 장에서 배운 운영 도구를 한 컨테이너에 모아 보면 다음과 같습니다. `docker logs` 로 무슨 일이 있었는지 읽고, `docker exec` 로 안에 들어가 데이터를 확인하고, `docker inspect` 로 설정을 들여다보고, `docker stats` 로 자원을 보고, 헬스체크로 준비 상태를 가립니다. 위 다이어그램이 이 다섯 도구가 각각 어떤 정보를 보여 주는지 한눈에 정리합니다.

팁: `docker inspect` 전체 출력은 길고 복잡합니다. 처음에는 `--format` 으로 `.NetworkSettings.Ports` (포트), `.Mounts` (볼륨), `.Config.Env` (환경변수)처럼 한 항목씩 뽑아 보는 연습을 권합니다. Docker Desktop 대시보드의 컨테이너 화면에서도 같은 정보를 클릭으로 볼 수 있어, 명령과 화면을 함께 익히면 좋습니다.

주의: 컨테이너가 `Up (running)`이라고 해서 PostgreSQL이 곧바로 접속을 받는 것은 아닙니다. 기동에는 약간의 시간이 걸립니다. 자동화·스크립트에서 컨테이너를 띄우자마자 접속하면 거부될

수 있으니, 헬스체크의 (healthy) 를 확인한 뒤 접속하는 습관을 들이면 안전합니다. 다만 헬스체크가 없다고 컨테이너가 안 뜨는 것은 아닙니다. 헬스체크는 "상태를 더 정확히 알려 주는" 선택 장치입니다.

절 요약

- `docker inspect` 는 포트·볼륨·환경변수 등 컨테이너 설정 전체를 보여 주며, `--format` 으로 원하는 항목만 뽑습니다.
- `docker stats` 는 CPU·메모리 등 자원 사용량을 실시간으로 보여 줍니다(`Ctrl + C` 로 종료).
- 헬스체크는 서비스가 정말 준비됐는지 점검해 (healthy) 로 표시합니다.
- `running` 과 접속 가능은 다릅니다. 헬스체크로 준비 상태를 정확히 가릴 수 있습니다.

실습 과제

해보기: 운영 점검 루틴 만들기

실행 중인 PostgreSQL 컨테이너 하나를 두고, 이 장에서 배운 운영 도구를 순서대로 적용하며 나만의 점검 체크리스트를 채워 봅니다(PostgreSQL CRUD 교재에서 익힌 psql 사용법을 그대로 재활용합니다).

- 단계 1: `docker ps` 로 점검할 컨테이너가 `up` 상태인지 확인합니다(정지 상태면 `docker start` 로 켵니다).
- 단계 2: `docker exec -it pg16 psql -U postgres` 로 들어가 `\l` 로 데이터베이스 목록, `\dt` 로 테이블 목록을 확인하고 `\q` 로 나옵니다. 호스트에 psql을 깔지 않고도 됐다는 점을 메모합니다.
- 단계 3: `docker exec -it pg16 bash` 로 셸에 들어가 `ls /var/lib/postgresql/data` 로 데이터 디렉터리를 둘러본 뒤 `exit` 로 나옵니다.
- 단계 4: `docker logs --tail 20 pg16` 으로 최근 로그를 보고 `database system is ready to accept connections` 줄을 찾습니다.
- 단계 5: `docker inspect --format '{{json .NetworkSettings.Ports}}' pg16` 으로 포트 매핑을, `docker stats --no-stream pg16` 으로 메모리 사용량을 확인해 적습니다.
- 점검: 위 다섯 단계의 결과(접속 성공 여부, 데이터 디렉터리 위치, 기동 메시지 유무, 매핑된 포트, 메모리 사용량)를 한 줄씩 적으면 운영 점검표 한 장이 완성됩니다. 다음에 컨테이너에 문제가 생기면 이 순서대로 짚어 봅니다.

FAQ

Q1. `docker exec` 와 `docker run` 은 무엇이 다른가요? → A1. `docker run` 은 이미지로부터 새 컨테이너를 만들어 실행하는 명령이고, `docker exec` 는 이미 실행 중인 컨테이너 안에서 명령만 더 돌리는 명령입니다. `psql`을 한 번 써 보려고 `docker run` 을 또 하면 불필요한 컨테이너가 새로 생깁니다. 떠 있는 컨테이너 안을 다룰 때는 항상 `docker exec` 를 씁니다. 또한 `docker exec` 는 컨테이너가 실행 중일 때만 됩니다.

Q2. 호스트(내 PC)에 `psql`을 설치하지 않았는데 컨테이너 안 `psql`을 써도 되나요? → A2. 네, 그것이 이 장의 핵심입니다. 공식 `postgres` 이미지에는 PostgreSQL 서버와 함께 `psql` 클라이언트도 들어 있습니다. `docker exec -it 컨테이너 psql -U postgres` 로 컨테이너 안 `psql`을 바로 쓸 수 있고, PostgreSQL CRUD 교재에서 익힌 `\l · \dt` 같은 메타 명령과 SQL이 그대로 통합니다. 호스트를 깨끗하게 유지하면서 DB를 다룰 수 있다는 것이 Docker 운영의 큰 장점입니다.

Q3. `docker logs` 에 아무것도 안 나오거나, 컨테이너가 자꾸 멈춥니다. 어떻게 진단하나요? → A3. 먼저 `docker ps -a` 로 컨테이너가 `Exited` 상태인지 봅니다. 그다음 `docker logs 컨테이너이름` 으로 마지막 메시지를 읽으면 원인이 대부분 적혀 있습니다. 예를 들어 비밀번호 환경변수를 빠뜨리면 `You must specify POSTGRES_PASSWORD ...` 가 찍힙니다. 실시간으로 보고 싶으면 `docker logs -f` 로 따라가며 다른 창에서 동작을 재현해 보세요.

Q4. 컨테이너가 `up` 인데 접속이 거부됩니다. 왜 그런가요? → A4. 컨테이너가 실행 중이라는 것과, 안의 PostgreSQL이 접속을 받을 준비가 끝났다는 것은 별개입니다. 기동에는 잠깐 시간이 걸립니다. `docker logs` 에서 `database system is ready to accept connections` 줄이 떴는지 확인하거나, 헬스체크를 등록해 `(healthy)` 표시를 보고 접속하면 이 문제를 피할 수 있습니다.

장 요약

이 장에서는 이미 떠 있는 컨테이너를 안에서 다루고 바깥에서 살펴보는 운영 도구를 익혔습니다. `docker exec -it` 로 컨테이너 안 `psql`과 `bash` 셸에 들어가, 호스트에 `psql`을 깔지 않고도 PostgreSQL CRUD 교재의 메타 명령(`\l · \dt`)을 그대로 쓰고 데이터 디렉터리·설정 파일을 돌려봤습니다. `docker logs` 로 기동·오류 메시지를 읽고 `-f · --tail` 로 실시간·최근 줄만 추적하는 법을 배웠으며, `docker inspect` 로 포트·볼륨·환경변수 설정을, `docker stats` 로 자원 사용량을 확인했습니다. 끝으로 헬스체크로 "실행 중"과 "접속 준비 완료"를 구분하는 법까지 다뤘습니다. 이제 컨테이너에 문제가 생겨도 들어가서 보고, 로그를 읽고, 설정을 확인하는 점검 루틴을 스스로 돌릴 수 있습니다.

핵심 키워드

`docker exec`, `-it`, 컨테이너 내부 `psql`, 메타 명령(`\l`, `\dt`), `bash` 셸, `docker logs`, `-f`, `--tail`, `docker inspect`, `docker stats`, 헬스체크(`healthcheck`), `pg_isready`

다음 장 미리보기

다음 장에서는 지금까지 `docker run`에 길게 붙이던 포트·볼륨·환경변수 옵션을 **`docker-compose.yml` 한 파일로 옮겨 선언하는 법**을 배웁니다. 명령을 매번 외워 입력하는 대신 파일로 고정해 두면, 동료와 같은 환경을 그대로 재현하고 `docker compose up` 한 줄로 전체를 띄울 수 있습니다.

docker compose 기초: 한 파일로 PostgreSQL 선언하기

학습 목표 - 긴 docker run 대신 compose로 선언하는 이점을 설명할 수 있습니다. - docker-compose.yml에 PostgreSQL db 서비스를 작성할 수 있습니다. - docker compose up-down으로 환경을 통째로 띄우고 내릴 수 있습니다. - 볼륨·환경변수.env를 compose로 통합하고 팀과 재현 가능하게 공유할 수 있습니다.

앞 장들에서 우리는 `docker run` 에 `-p` 로 포트를 잇고, `-v` 로 볼륨을 붙이고, `-e` 로 환경변수를 넣으며 PostgreSQL 컨테이너를 띄웠습니다. 명령 하나로 많은 일을 할 수 있다는 것은 배웠지만, 그만큼 명령 한 줄이 점점 길고 외우기 어려워졌습니다.

이 장에서는 그 긴 명령을 **파일 한 장으로 대신하는** 방법을 배웁니다. 도구 이름은 **Docker Compose(도커 컴포즈)** 입니다. 컨테이너에 줄 옵션들을 `docker-compose.yml` 이라는 파일에 미리 적어 두면, `docker compose up` 한 번으로 그 구성대로 컨테이너가 떠오릅니다. 명령을 매번 입력하는 대신, **원하는 상태를 파일로 적어 두는** 방식으로 발상을 바꾸는 것입니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 을 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, 프롬프트는 `PS C:\Users\me>` 처럼 생겼습니다. 아래 예시에서 `PS C:\Users\me>` 는 "여러분이 입력하는 자리"를 뜻하고, 그 아래 줄들은 Docker가 돌려주는 출력입니다.

용어 미리 보기: **YAML(야믈)** 은 사람이 읽기 쉽게 만든 설정 파일 형식입니다. 들여쓰기(빈칸)로 항목의 포함 관계를 나타냅니다. **선언(declaration)** 이란 "이렇게 만들라"고 절차를 일일이 시키는 대신 "최종 모습은 이렇다"고 적어 두는 방식을 말합니다.

왜 compose인가: 길어지는 명령과 재현성 문제

도입

공연 무대를 떠올려 봅시다. 매 공연마다 조명·음향·소품을 기억에 의존해 손으로 다시 세팅하면, 한 번이라도 빠뜨리거나 순서가 틀리기 쉽습니다. 그래서 무대 감독은 **큐시트(cue sheet)** 한 장에 무엇을 어떻게 둘지 적어 둡니다. compose는 컨테이너 세계의 큐시트입니다. 이 절에서는 왜 긴 `docker run` 을 파일로 옮기는 편이 나은지, 그 동기를 먼저 이해합니다.

핵심 개념 설명

Docker Compose (docker compose)

Docker Compose(도커 컴포즈) 는 컨테이너의 구성을 YAML 파일 하나에 선언해 두고, 그 파일대로 컨테이너를 한 번에 띄우고 내리는 도구입니다. Docker Desktop을 설치하면 함께 들어오므로 따로 설치할 필요는 없습니다.

- **왜(why) 필요한가:** `docker run` 은 명령을 입력하는 그 순간에만 효력이 있습니다. 어떤 옵션을 줬는지는 명령 기록에만 남아, 시간이 지나면 잊어버리기 쉽습니다. compose는 구성을 **파일로 남겨** 언제든지 똑같이 재현하게 해 줍니다.
- **언제(when) 쓰나:** 옵션이 두세 개를 넘어가거나, 같은 환경을 여러 번 띄우거나, 그 환경을 동료와 나눠야 할 때 씁니다.
- **어떻게(how) 쓰나:** `docker-compose.yml` 파일을 한 번 작성해 두고, 그 파일이 있는 폴더에서 `docker compose up / docker compose down` 을 실행합니다.

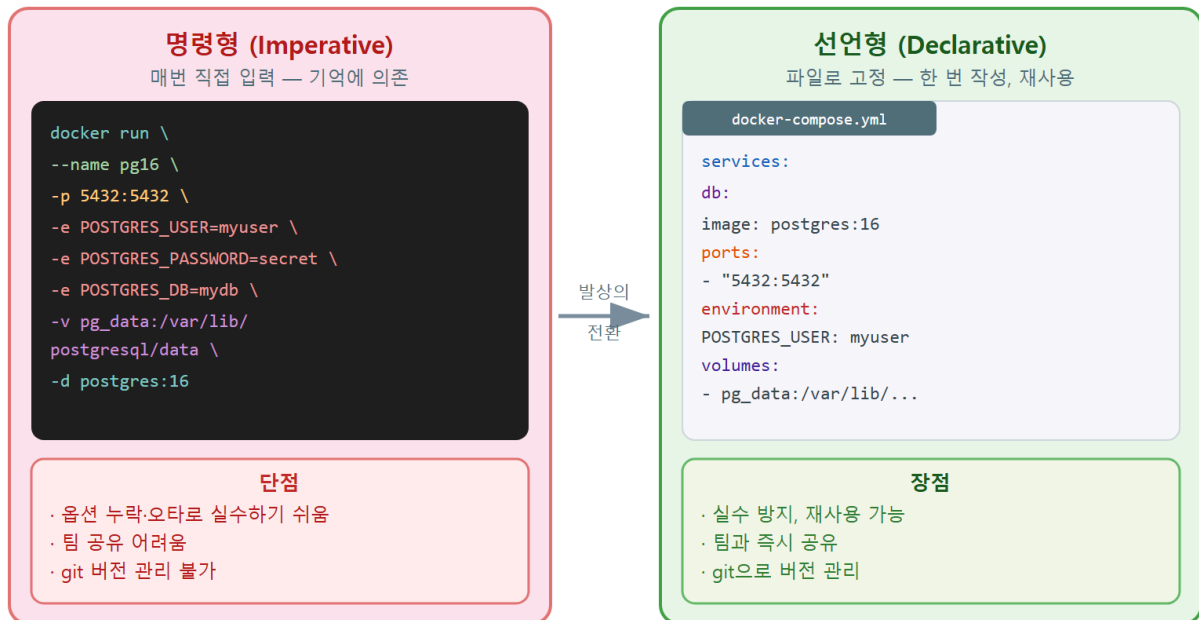
여기서 입문자가 자주 하는 오해를 두 가지 짚겠습니다. 첫째, "compose는 컨테이너가 여러 개일 때만 쓴다"는 생각입니다. 컨테이너가 하나뿐이어도 옵션이 길면 compose가 훨씬 편합니다. 둘째, "compose는 `docker run` 과 완전히 다른 별개의 엔진"이라는 생각입니다. compose는 결국 우리가 배운 `docker run` 과 같은 일을 파일을 읽어 대신 해 주는 **편의 도구**일 뿐입니다.

선언적 구성 (declarative configuration)

선언적 구성(declarative configuration) 이란, "이 단계를 실행하고 다음 단계를 실행하라"고 명령을 나열하는 대신, "**완성된 모습은 이렇다**" 고 결과를 적어 두는 방식입니다. compose 파일에는 "어떤 이미지로, 어떤 포트를, 어떤 볼륨을 가진 컨테이너가 떠 있어야 한다"는 최종 상태만 적습니다. 그 상태를 만드는 일은 Docker가 알아서 합니다.

요리에 비유하면, 명령형은 "냄비를 올려라, 물을 부어라, 3분 끓여라"처럼 동작을 하나씩 시키는 것이고, 선언형은 "완성 접시는 이런 모습"이라고 사진을 보여 주는 것에 가깝습니다. 우리가 적은 결과대로 환경이 맞춰지므로, 같은 파일은 어느 PC에서든 같은 환경을 만들어 냅니다. 이것이 5장에서 짚었던 **재현성(reproducibility)** 으로 이어집니다.

명령형 vs 선언형: docker run 대 docker compose



명령어를 매번 입력하는 대신 YAML 파일 하나에 선언해 두고 재사용한다

ch_08_s01 · 왜 compose인가: 명령형 대 선언형

이렇게 보면 차이가 분명합니다

먼저 앞 장들에서 띄우던 방식, 즉 옵션이 잔뜩 붙은 `docker run` 한 줄입니다.

```
PS C:\Users\me> docker run -d --name pg16 -p 5432:5432 -e POSTGRES_PASSWORD=mysecret -e POSTGRES
```

한 줄이 화면을 넘길 만큼 길어졌습니다. 옵션 하나를 빠뜨리거나 오타가 나면 컨테이너가 엉뚱하게 뜨거나 아예 멈춥니다. 게다가 이 줄을 동료에게 전달하려면 메신저로 통째로 복사해 보내야 하고, 받은 사람은 어느 값이 무엇인지 한눈에 알기 어렵습니다.

같은 구성을 compose 파일로 적으면 다음과 같은 모습이 됩니다(자세한 작성법은 다음 절에서 다룹니다).

```
# docker-compose.yml (비실행 - 참고용 구성 파일)
```

```
services:
```

```
  db:
```

```
    image: postgres:16
```

```
    ports:
```

```
      - "5432:5432"
```

```
    environment:
```

```
      POSTGRES_PASSWORD: mysecret
```

```
      POSTGRES_USER: appuser
```

```
      POSTGRES_DB: appdb
```

```
    volumes:
```

```
      - pgdata:/var/lib/postgresql/data
```

```
volumes:
```

```
  pgdata:
```

같은 정보지만 항목별로 줄이 나뉘어, 무엇이 포트이고 무엇이 환경변수인지 한눈에 들어옵니다. 이 파일은 한 번 적어 두면 명령을 외울 필요 없이 `docker compose up` 으로 그대로 살아납니다.

명령형 vs 선언형 비교

구분	긴 <code>docker run</code>	<code>docker-compose.yml</code>
형태	명령 한 줄(명령형)	파일에 항목별 선언(선언형)
효력	입력한 순간뿐	파일로 남아 언제나 재실행
가독성	한 줄이 길어 파악 어려움	항목이 줄별로 정리됨
공유	명령 줄을 통째로 전달	파일을 그대로 전달
실수	오타·옵션 누락 잦음	파일을 검토·수정 가능

팁: 옛 자료에서는 `docker-compose up` (중간에 붙임표)처럼 별도 명령으로 쓰는 경우가 보입니다. 요즘 Docker(버전 27 기준)에서는 `docker compose up` (띄어쓰기)이 표준입니다. 둘 다 거의 같게 동작하지만, 이 책에서는 최신 표기인 `docker compose` (띄어쓰기)를 씁니다.

절 요약

- `compose`는 컨테이너 구성을 YAML 파일에 선언해 한 번에 띄우고 내리는 도구입니다.
- 긴 `docker run` 은 명령을 입력한 순간에만 효력이 있고 공유·재현이 어렵습니다.
- 선언적 구성은 "완성된 모습"을 적어 두는 방식이라 같은 파일이 같은 환경을 재현합니다.

- 컨테이너가 하나여도 옵션이 길면 compose가 편리합니다.

docker-compose.yml로 PostgreSQL 선언하기

도입

앞 절에서 compose 파일의 전체 모습을 잠깐 봤습니다. 이제 그 파일을 한 줄씩 직접 써 봅니다. 이 절에서는 compose 파일의 기본 단위인 **서비스(service)**가 무엇인지 이해하고, PostgreSQL을 띄우는 `db` 서비스에 `image` · `ports` · `environment` · `volumes`를 채워 넣습니다. 더불어 YAML에서 가장 자주 사고가 나는 **들여쓰기** 규칙도 짚습니다.

핵심 개념 설명

서비스 (service)

서비스(service)는 compose 파일에서 컨테이너 하나의 구성을 가리키는 단위입니다. 쿼시트에 적힌 "조명" 한 항목처럼, 전체 구성 안에서 한 역할을 맡는 묶음입니다. 우리는 PostgreSQL을 담당하는 서비스에 `db`라는 이름을 붙일 것입니다.

- **왜(why) 필요한가:** 한 환경에 컨테이너가 여러 개일 수 있습니다(예: 데이터베이스, 웹 앱). 각 컨테이너의 구성을 서비스라는 단위로 나눠 적으면 정리가 됩니다.
- **언제(when) 쓰나:** compose 파일을 쓸 때 항상, 컨테이너마다 하나의 서비스를 정의합니다.
- **어떻게(how) 쓰나:** 최상위 `services:` 아래에 `db:`처럼 원하는 서비스 이름을 두고, 그 안에 이미지·포트 등 세부 설정을 들여 씁니다.

여기서 두 가지 오해를 짚겠습니다. 첫째, 서비스 이름(`db`)은 이미지 이름(`postgres`)과 다릅니다. `db`는 내가 붙이는 별명이고, 어떤 이미지를 쓸지는 그 안의 `image:` 항목으로 따로 정합니다. 둘째, 한 compose 파일에 서비스를 하나만 둘 수 있는 것은 아닙니다. 필요하다면 `db`, `app`처럼 여러 서비스를 나란히 둘 수 있습니다(10장에서 다룹니다).

compose 파일 구조 (image / ports / environment / volumes)

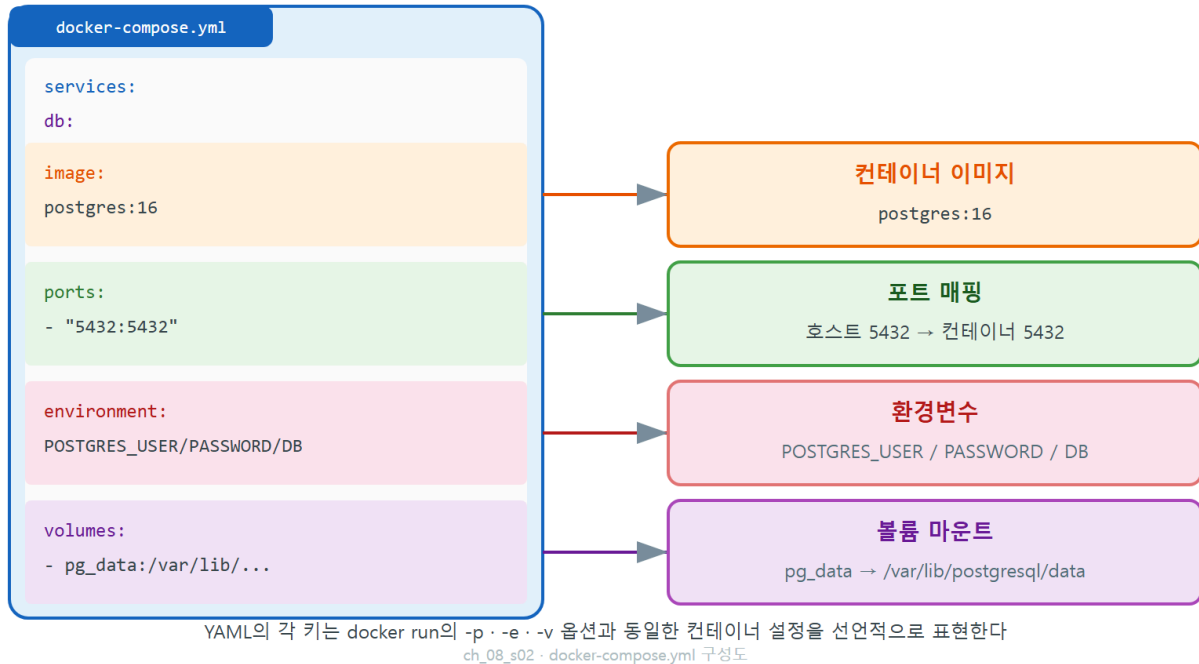
`db` 서비스 안에 들어가는 네 가지 핵심 항목은, 사실 앞 장에서 배운 `docker run` 옵션과 하나씩 짝이 맞습니다.

- `image`: 어떤 이미지를 쓸지(↔ `docker run`의 이미지 인자). 예: `postgres:16`

- `ports`: 호스트 포트와 컨테이너 포트 연결($\leftrightarrow$ `-p 5432:5432`)
- `environment`: 환경변수($\leftrightarrow$ `-e POSTGRES_PASSWORD=...`)
- `volumes`: 볼륨 마운트($\leftrightarrow$ `-v pgdata:/var/lib/postgresql/data`)

즉 새로운 개념을 외우는 것이 아니라, 이미 아는 옵션이 파일의 어느 줄로 옮겨지는지만 익히면 됩니다.

docker-compose.yml 키 → 컨테이너 설정 매핑



이렇게 작성합니다

실습 폴더(예: `C:\Users\me\pg-compose`)를 하나 만들고, 그 안에 `docker-compose.yml` 이라는 이름의 파일을 만들어 다음과 같이 작성합니다.

docker-compose.yml (비실행 - 참고용 구성 파일)

```
services:
  db:
    image: postgres:16
    container_name: pg16
    ports:
      - "5432:5432"
    environment:
      POSTGRES_PASSWORD: mysecret
      POSTGRES_USER: appuser
      POSTGRES_DB: appdb
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

각 항목이 무엇을 뜻하는지 아래 표로 정리합니다.

compose 파일 키 설명

키	의미	docker run 대응
services	컨테이너 구성들의 묶음(최상위)입니다	-
db	PostgreSQL을 담당하는 서비스 이름(별명)입니다	-
image: postgres:16	사용할 이미지와 태그입니다	이미지 인자
container_name: pg16	실제 컨테이너에 붙일 이름입니다	--name pg16
ports: - "5432:5432"	호스트 5432를 컨테이너 5432에 연결합니다	-p 5432:5432
environment	컨테이너에 줄 환경변수 목록입니다	-e KEY=VALUE
volumes (서비스 안)	볼륨을 컨테이너 경로에 마운트합니다	-v pgdata:/var/lib/...
volumes (최상위)	사용할 네임드 볼륨을 선언합니다	-

여기서 가장 중요한 규칙은 **YAML 들여쓰기**입니다. YAML은 들여쓰기로 포함 관계를 나타내므로, 빈칸 개수가 틀리면 파일을 읽지 못합니다. 두 가지만 지키면 됩니다. 첫째, **들여쓰기는 빈칸(스페이스)으로만** 합니다. 탭(Tab) 키로 들여쓰면 오류가 납니다. 둘째, 같은 단계의 항목은 **빈칸 개수를 똑같이** 맞춥니다(보통 2칸씩 늘립니다).

들여쓰기가 틀리면 다음과 비슷한 오류를 만납니다.

```
yaml: line 5: did not find expected key
```

```
services.db Additional property port is not allowed
```

위 첫 줄은 들여쓰기가 어긋났을 때, 둘째 줄은 키 이름을 `ports`가 아니라 `port`로 잘못 적었을 때 나오는 메시지입니다. 메시지에 나온 줄 번호와 키 이름을 보고 그 부분을 고치면 됩니다.

주의: 메모장(Notepad)으로 작성하면 들여쓰기 빈칸이 눈에 보이지 않아 실수하기 쉽습니다. VS Code(브이에스 코드) 같은 편집기를 쓰면 탭과 빈칸을 구분해 보여 주고, 들여쓰기가 어긋난 줄을 표시해 줘 훨씬 안전합니다. 또한 파일 이름은 정확히 `docker-compose.yml` (또는 `compose.yml`) 이어야 `compose`가 자동으로 찾습니다.

팁: `ports` 값을 "5432:5432" 처럼 따옴표로 감싼 이유가 있습니다. YAML은 숫자:숫자 형태를 시각(시:분)으로 오해할 수 있어, 포트 매핑은 문자열로 적는 것이 안전합니다. `container_name`은 생략해도 됩니다. 생략하면 `compose`가 `폴더이름-db-1` 처럼 자동으로 이름을 붙입니다.

절 요약

- 서비스(service)는 `compose` 파일에서 컨테이너 하나의 구성을 가리키는 단위입니다.
- `image` · `ports` · `environment` · `volumes` 는 각각 `docker run`의 이미지 · `-p` · `-e` · `-v` 와 짝이 맞습니다.
- YAML은 들여쓰기로 포함 관계를 나타내므로 탭이 아닌 빈칸으로, 단계마다 일정하게 들여씁니다.
- 파일 이름은 `docker-compose.yml` 이어야 `compose`가 자동으로 인식합니다.

docker compose up / down

도입

파일을 다 적었으니, 이제 그 파일대로 환경을 띄울 차례입니다. 무대 큐시트를 적었다면 "시작" 신호로 모든 준비가 한 번에 돌아가야겠지요. `compose`에서 그 신호가 `docker compose up` 이고, 공연이 끝나 무대를 정리하는 신호가 `docker compose down` 입니다. 이 절에서는 이 두 명령으로 환경을 통째로 띄우고 내리는 법을 배우고, `down` 에 `-v` 를 붙일 때 데이터까지 지워지는 차이를 분명히 짚습니다.

핵심 개념 설명

`docker compose up` (선언대로 띄우기)

`docker compose up` 은 현재 폴더의 `compose` 파일을 읽어, 거기 선언된 네트워크·볼륨·컨테이너를 한 번에 만들어 실행하는 명령입니다.

- **왜(why) 필요한가:** 파일에 적어 둔 구성을 실제로 살리려면 이 명령이 필요합니다. 여러 컨테이너가 있어도 한 번에 떠오릅니다.
- **언제(when) 쓰나:** 환경을 처음 띄울 때, 또는 파일을 고친 뒤 다시 적용할 때 씁니다.
- **어떻게(how) 쓰나:** `docker-compose.yml` 이 있는 폴더에서 `docker compose up -d` 를 실행합니다. `-d` 는 앞 장에서 배운 것과 같은 **백그라운드 실행**입니다. `-d` 를 빼면 로그가 화면에 계속 흐르고 그 창을 점유합니다.

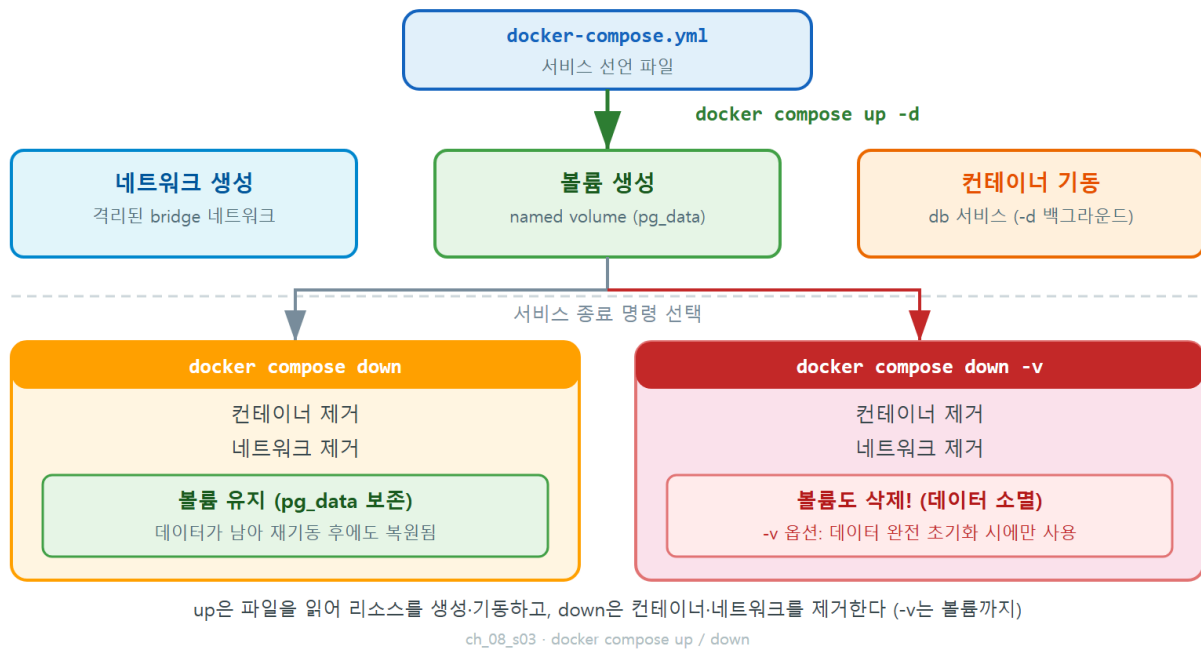
`docker compose down` (정리하기)와 `-v`의 차이

`docker compose down` 은 `up` 으로 만든 컨테이너와 네트워크를 멈추고 제거하는 명령입니다. 여기서 입문자가 반드시 알아야 할 중요한 차이가 있습니다.

- `docker compose down` : 컨테이너와 네트워크를 지웁니다. 그러나 **볼륨(데이터)**은 남깁니다. 그래서 다시 `up` 하면 이전 데이터가 그대로 살아납니다.
- `docker compose down -v` : 여기에 더해 **볼륨까지 함께 삭제**합니다(`-v` 는 volume). 데이터가 영구히 사라지므로, 다시 `up` 하면 빈 데이터베이스로 새로 시작합니다.

`down` 과 `down -v` 의 차이는 5장에서 배운 데이터 영속성과 직결됩니다. 평소 환경을 잠시 내릴 때는 `down` 을, 데이터까지 깨끗이 초기화하고 싶을 때만 `down -v` 를 씁니다.

docker compose up / down 양방향 흐름



이렇게 실행합니다

docker-compose.yml 이 있는 폴더로 이동한 뒤 환경을 띄웁니다.

```
PS C:\Users\me\pg-compose> docker compose up -d
```

예상 화면:

```
[+] Running 3/3
- Network pg-compose_default Created
- Volume "pg-compose_pgdata" Created
- Container pg16 Started
```

세 줄이 차례로 만들어진 것이 보입니다. compose가 파일을 읽어 **네트워크 → 볼륨 → 컨테이너** 순서로 알아서 준비했다는 뜻입니다. 우리가 일일이 docker network 나 docker volume 명령을 치지 않았는데도 필요한 것들이 함께 생겼습니다. 잘 댄지는 compose 전용 명령으로 확인합니다.

```
PS C:\Users\me\pg-compose> docker compose ps
```

예상 화면:

NAME	IMAGE	STATUS	PORTS
pg16	postgres:16	Up 10 seconds	0.0.0.0:5432->5432/tcp

이제 환경을 내려 봅니다. 데이터는 남기고 컨테이너만 정리하는 경우입니다.

```
PS C:\Users\me\pg-compose> docker compose down
```

예상 화면:

```
[+] Running 2/2
- Container pg16          Removed
- Network pg-compose_default Removed
```

Container 와 Network 는 Removed 로 지워졌지만, Volume 은 목록에 없습니다. 즉 데이터 볼륨 pg-compose_pgdata 는 남아 있습니다. 다시 `docker compose up -d` 를 하면 같은 볼륨이 다시 연결되어 이전 데이터가 그대로 돌아옵니다.

`docker compose up [-d] | docker compose down [-v]`

compose 명령 설명

명령	의미
<code>docker compose up -d</code>	파일대로 컨테이너·네트워크·볼륨을 만들어 백그라운드로 띄웁니다
<code>docker compose ps</code>	compose가 띄운 컨테이너의 상태를 봅니다
<code>docker compose logs -f</code>	compose 환경의 로그를 실시간으로 봅니다
<code>docker compose down</code>	컨테이너·네트워크를 지우되 볼륨(데이터)은 남깁니다
<code>docker compose down -v</code>	위에 더해 볼륨까지 삭제해 데이터를 초기화합니다

위험: `docker compose down -v` 의 `-v` 는 볼륨을 지웁니다. 그 안의 PostgreSQL 데이터가 **영구히 사라집니다**. "환경을 깔끔히 내리려고" 습관처럼 `-v` 를 붙이면 중요한 데이터를 통째로 날릴 수 있습니다. 데이터를 보존하려면 `-v` 없이 `docker compose down` 만 실행합니다. 정말 초기화가 필요할 때만, 백업(9장)을 떠 둔 뒤 `-v` 를 씁니다.

팁: compose 파일을 고친 뒤 변경을 적용하려면 다시 `docker compose up -d` 를 실행하면 됩니다. compose가 현재 상태와 파일을 비교해 바뀐 부분만 다시 만듭니다. 컨테이너를 완전히 새로 만들고 싶을 때는 `docker compose up -d --force-recreate` 를 쓸 수 있습니다.

절 요약

- `docker compose up -d` 는 파일대로 네트워크·볼륨·컨테이너를 한 번에 만들어 띄웁니다.
- `docker compose ps / logs` 로 compose가 띄운 환경의 상태와 로그를 봅니다.
- `docker compose down` 은 컨테이너·네트워크만 지우고 볼륨(데이터)은 남깁니다.
- `docker compose down -v` 는 볼륨까지 지워 데이터를 초기화하므로 신중히 씁니다.

볼륨·환경변수·.env를 compose로 통합하기

도입

지금까지의 compose 파일에는 비밀번호가 `mysecret` 처럼 그대로 적혀 있습니다. 6장에서 배운 것처럼, 비밀번호를 파일에 박아 두는 것은 좋지 않은 습관입니다. 이 절에서는 볼륨을 compose 안에서 제대로 선언하는 법과, 6장에서 다룬 `.env` 파일을 compose에서 참조해 비밀번호를 분리하는 법을 배웁니다.

핵심 개념 설명

compose 볼륨 정의 (named volume)

5장에서 우리는 `-v pgdata:/var/lib/postgresql/data` 로 네임드 볼륨을 붙였습니다. compose에서는 이를 두 곳에 나눠 적습니다.

- 서비스 안의 `volumes:` : 볼륨을 컨테이너의 어느 경로에 붙일지 적습니다(예: `pgdata:/var/lib/postgresql/data`).
- 최상위 `volumes:` : 사용할 볼륨 이름을 한 번 선언합니다(예: `pgdata:`).

최상위에 이름을 선언해 두는 이유는, 그 볼륨이 도커가 관리하는 **네임드 볼륨**임을 compose에 알려 주기 위해서입니다. 이렇게 적으면 `compose down (-v 없이)`으로 환경을 내려도 데이터가 안전하게 남습니다.

compose의 .env 연동 (변수 참조)

compose 파일은 같은 폴더의 **.env 파일**을 자동으로 읽어, **\${변수이름}** 형태로 값을 끼워 넣을 수 있습니다. 6장에서 비밀번호를 **.env**로 분리했던 발상이 compose에서도 그대로 통합니다.

- **왜(why) 필요한가:** 비밀번호 같은 민감한 값을 compose 파일에 직접 쓰면, 그 파일을 공유하거나 git에 올릴 때 비밀번호가 그대로 노출됩니다. 값을 **.env**로 빼 두면 compose 파일에는 **\${POSTGRES_PASSWORD}**라는 자리만 남습니다.
- **언제(when) 쓰나:** 비밀번호·사용자명처럼 PC마다 다르거나 숨겨야 하는 값을 다룰 때 씁니다.
- **어떻게(how) 쓰나:** 같은 폴더에 **.env** 파일을 만들어 **이름=값**으로 적고, compose 파일에서는 **\${이름}**으로 참조합니다.

이렇게 작성합니다

먼저 같은 폴더에 **.env** 파일을 만들고 민감한 값을 적습니다.

```
# .env (비실행 - 환경변수 정의 파일, git에 올리지 않음)
POSTGRES_PASSWORD=mysecret
POSTGRES_USER=appuser
POSTGRES_DB=appdb
```

그다음 **docker-compose.yml**에서 그 값을 **\${...}**로 참조합니다.

```
# docker-compose.yml (비실행 - .env 값을 참조하는 구성)
services:
  db:
    image: postgres:16
    container_name: pg16
    ports:
      - "5432:5432"
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

이제 compose 파일 어디에도 실제 비밀번호가 보이지 않습니다. 값이 제대로 끼워지는지는 **docker compose config**로 확인할 수 있습니다. 이 명령은 **.env**를 반영한 최종 구성을 보여 줍니다.

```
PS C:\Users\me\pg-compose> docker compose config
```

예상 화면:

```
services:
  db:
    image: postgres:16
    container_name: pg16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_PASSWORD: mysecret
      POSTGRES_USER: appuser
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
```

`${POSTGRES_PASSWORD}` 자리에 `.env` 의 실제 값(`mysecret`)이 끼워진 것이 보입니다. `compose`가 파일을 읽을 때 `.env` 의 값을 가져와 메웠다는 뜻입니다.

볼륨`.env` 관련 키 설명

항목	의미
서비스 안 <code>volumes</code>	네임드 볼륨을 컨테이너 경로에 마운트합니다
최상위 <code>volumes: pgdata:</code>	사용할 네임드 볼륨 이름을 선언합니다
<code>\${POSTGRES_PASSWORD}</code>	같은 폴더 <code>.env</code> 의 같은 이름 값을 끼워 넣습니다
<code>.env</code> 파일	이름=값 으로 민감한 값을 분리해 보관합니다
<code>docker compose config</code>	<code>.env</code> 를 반영한 최종 구성을 미리 확인합니다

주의: `.env` 파일은 `compose` 파일과 **같은 폴더**에 있어야 자동으로 읽힙니다. 또한 `.env` 에 적은 변수 이름과 `compose`의 `${...}` 안 이름이 정확히 같아야 합니다. 이름이 틀리거나 `.env` 에 값이 없으면, 그 자리는 빈 값으로 채워져 PostgreSQL이 비밀번호 없이 떠 멈출 수 있습니다. `docker compose config` 로 값이 제대로 들어갔는지 먼저 확인하는 습관을 권합니다.

절 요약

- 네임드 볼륨은 서비스 안 `volumes` (마운트)와 최상위 `volumes` (선언) 두 곳에 적습니다.

- compose는 같은 폴더의 `.env` 를 자동으로 읽어 `${이름}` 자리에 값을 끼워 넣습니다.
- 비밀번호 같은 민감한 값은 `.env` 로 분리해 compose 파일에는 자리만 남깁니다.
- `docker compose config` 로 `.env` 가 반영된 최종 구성을 미리 확인합니다.

팀 공유와 재현성: compose 파일을 git에 올리기

도입

compose의 가장 큰 보람은 "내 PC에서 되던 환경이 동료 PC에서도 똑같이 된다"는 데 있습니다. 그러려면 파일을 안전하게 나눠야 합니다. 이 절에서는 어떤 파일은 올리고 어떤 파일은 올리지 말아야 하는지(`.gitignore`), 그리고 동료가 같은 파일로 동일한 PostgreSQL 환경을 재현하는 흐름을 정리합니다.

핵심 개념 설명

버전 관리 (version control)와 `.gitignore`

버전 관리(version control) 는 파일의 변경 이력을 기록하고 동료와 나누는 방식으로, 대표 도구가 **git(깃)** 입니다. compose 파일을 git 저장소에 올려 두면, 환경 구성이 코드처럼 이력으로 남고 누구나 같은 파일을 받아 쓸 수 있습니다.

여기서 핵심 원칙이 하나 있습니다. `docker-compose.yml` 은 올리고, `.env` 는 올리지 않습니다. compose 파일에는 비밀번호가 빠져 있으니 공유해도 안전하지만, `.env` 에는 실제 비밀번호가 들어 있어 공유하면 위험합니다. 올리지 않을 파일은 `.gitignore` 라는 파일에 이름을 적어 git이 무시하게 합니다.

재현성 (reproducibility)

재현성(reproducibility) 은 "어느 PC에서든 같은 환경이 다시 만들어지는 성질"입니다. 이 책이 처음부터 강조해 온 가치입니다. compose 파일은 이미지 태그(`postgres:16`), 포트, 볼륨, 환경변수를 모두 고정해 두므로, 그 파일을 받은 사람은 `docker compose up -d` 한 번으로 작성자와 똑같은 환경을 얻습니다. "제 PC에서는 되는데요"라는 단골 문제를 줄여 주는 것이 compose의 가장 실용적인 이점입니다.

이렇게 구성합니다

저장소 폴더에 `.gitignore` 파일을 만들어, 공유하면 안 되는 파일 이름을 적습니다.

```
# .gitignore (비실행 - git이 무시할 파일 목록)
.env
```

그러면 git에 올라가는 것과 올라가지 않는 것이 다음처럼 나뉩니다.

git 공유 대상 정리

파일	git에 올리나	이유
<code>docker-compose.yml</code>	올림	비밀번호가 빠진 구성이라 안전, 환경을 공유
<code>.env</code>	올리지 않음	실제 비밀번호가 들어 있어 노출 위험
<code>.gitignore</code>	올림	무엇을 제외할지 동료와 함께 공유
<code>.env.example</code>	올림	값을 비운 견본으로, 동료가 보고 자기 <code>.env</code> 를 만들

동료가 이 저장소를 받아 환경을 재현하는 흐름은 다음과 같습니다.

1. git으로 저장소를 내려받는다 (`docker-compose.yml`, `.env.example` 포함)
2. `.env.example`을 복사해 `.env`를 만들고 자기 비밀번호를 채운다
3. `docker compose up -d` 를 실행한다
4. 작성자와 동일한 `postgres:16` 환경이 그대로 떠오른다

`.env` 만 각자 채우면, 나머지 구성은 파일에 고정되어 있어 누구의 PC에서든 같은 PostgreSQL 이 떠오릅니다. 이것이 compose가 주는 재현성의 실체입니다.

팁: `.env` 는 올리지 않되, 어떤 변수가 필요한지는 동료가 알아야 합니다. 그래서 값을 비운 견본 `.env.example` (예: `POSTGRES_PASSWORD=`)을 함께 올리는 것이 관례입니다. 동료는 이 견본을 복사해 `.env` 로 이름을 바꾸고 자기 값을 채우면 됩니다.

주의: `.env` 를 한 번이라도 git에 올린 적이 있다면, 나중에 `.gitignore` 에 추가해도 이미 올라간 기록에는 비밀번호가 남습니다. 처음부터 `.gitignore` 에 `.env` 를 넣고 시작하는 것이 가장 안전합니다. 실수로 올렸다면 비밀번호를 곧바로 바꾸는 것이 원칙입니다. 비밀 관리의 더 자세한 원칙은 10장에서 다룹니다.

절 요약

- `docker-compose.yml` 은 git에 올리고 `.env` 는 `.gitignore` 로 제외합니다.

- compose 파일은 이미지·포트·볼륨·환경변수를 고정해 재현성을 보장합니다.
- 동료는 저장소를 받아 자기 `.env` 만 채우고 `docker compose up -d` 로 같은 환경을 얻습니다.
- 값을 비운 `.env.example` 을 함께 올려 필요한 변수를 동료에게 안내합니다.

실습 과제

해보기: PostgreSQL 환경을 compose 한 파일로 재구성하기

앞 장들에서 `docker run` 옵션으로 띄우던 PostgreSQL을 compose 파일로 옮겨, `up / down` 으로 다루고 git 공유까지 구성해 봅니다.

- 단계 1: 실습 폴더(예: `C:\Users\me\pg-compose`)를 만들고 그 안에 `docker-compose.yml` 을 작성합니다. `image` · `ports` · `environment` · `volumes` 를 갖춘 `db` 서비스와 최상위 `volumes:` `pgdata:` 를 넣습니다.
- 단계 2: 같은 폴더에 `.env` 를 만들어 `POSTGRES_PASSWORD` · `POSTGRES_USER` · `POSTGRES_DB` 를 적고, compose 파일의 값들을 `${...}` 참조로 바꿉니다.
- 단계 3: `docker compose config` 로 `.env` 값이 제대로 끼워졌는지 먼저 확인합니다.
- 단계 4: `docker compose up -d` 로 환경을 띄우고 `docker compose ps` 로 `up` 상태와 포트 매핑을 확인합니다. 4장에서 익힌 `psql`이나 `DBeaver`로 접속이 되는지도 확인합니다.
- 단계 5: 테이블을 하나 만들어 데이터를 넣은 뒤 `docker compose down (-v 없이)`으로 내리고, 다시 `docker compose up -d` 를 했을 때 데이터가 남아 있는지 확인합니다.
- 단계 6: `.gitignore` 에 `.env` 를 적고, 값을 비운 `.env.example` 을 만들어 둡니다. 어떤 파일이 git에 올라가고 어떤 파일이 제외되는지 한 줄로 정리합니다.
- 점검: `docker compose down` 뒤에도 데이터가 살아남고, compose 파일만으로 같은 환경이 다시 떠오르면 성공입니다. `down` 과 `down -v` 의 차이를 직접 실험해 한 문장으로 메모해 봅니다.

FAQ

Q1. compose는 컨테이너가 여러 개일 때만 쓰는 것 아닌가요? → **A1.** 아닙니다. 컨테이너가 하나뿐이어도 옵션이 길거나 그 환경을 반복해 띄우고 공유해야 한다면 compose가 훨씬 편합니다. 이 장에서도 PostgreSQL 컨테이너 하나만으로 compose를 익혔습니다. 여러 컨테이너를 한 네트워크로 묶는 본격적인 사용은 10장에서 다룹니다.

Q2. `docker compose down` 을 하면 데이터가 사라지나요? → A2. 기본 `docker compose down` 은 컨테이너와 네트워크만 지우고 볼륨(데이터)은 남깁니다. 그래서 다시 `up` 하면 이전 데이터가 그대로 돌아옵니다. 데이터까지 지우는 것은 `-v` 를 붙인 `docker compose down -v` 이며, 이때는 볼륨이 삭제되어 데이터가 영구히 사라집니다. 둘을 분명히 구분해 써야 합니다.

Q3. YAML 파일을 저장했는데 `did not find expected key` 같은 오류가 납니다. → A3. 대부분 들여쓰기 문제입니다. YAML은 탭이 아니라 빈칸으로 들여써야 하고, 같은 단계의 항목은 빈칸 개수가 같아야 합니다. 오류 메시지에 나온 줄 번호를 찾아 그 줄과 위아래의 들여쓰기를 점검합니다. 메모장 대신 VS Code 같은 편집기를 쓰면 들여쓰기 오류를 미리 잡기 쉽습니다.

Q4. `.env` 에 비밀번호를 적었는데, 이 파일을 동료에게 줘도 되나요? → A4. 주지 않는 것이 원칙입니다. `.env` 에는 실제 비밀번호가 들어 있으므로 git이나 메신저로 공유하면 노출됩니다. 대신 `docker-compose.yml` 과 값을 비운 `.env.example` 만 공유하고, 동료는 그 견본을 복사해 자기 `.env` 를 직접 채웁니다. `.gitignore` 에 `.env` 를 넣어 실수로 올라가지 않게 막아 둡니다.

장 요약

이 장에서는 길고 외우기 어려운 `docker run` 명령을 **`docker-compose.yml` 한 파일로 옮기는** 법을 배웠습니다. compose는 컨테이너 구성을 "완성된 모습"으로 선언해 두는 선언적 도구로, `services` 아래 `db` 서비스에 `image`·`ports`·`environment`·`volumes` 를 적으면 그것이 그대로 `docker run` 옵션을 대신합니다. `docker compose up -d` 로 네트워크·볼륨·컨테이너를 한 번에 띄우고, `docker compose down` 으로 정리하되 `-v` 를 붙일 때만 볼륨까지 지워진다는 차이를 확인했습니다. 또한 `.env` 로 비밀번호를 분리해 `${...}` 로 참조하고, compose 파일은 git에 올리되 `.env` 는 `.gitignore` 로 제외해 팀이 동일한 환경을 재현하는 흐름까지 익혔습니다. 이제 환경 하나를 파일 한 장으로 다룰 수 있게 되었습니다.

핵심 키워드

`docker compose`, 선언적 구성(declarative configuration), 서비스(service), `docker-compose.yml`, `image` / `ports` / `environment` / `volumes`, `docker compose up`, `docker compose down`, `-v`, `.env`, `${변수}`, `.gitignore`, 재현성(reproducibility)

다음 장 미리보기

다음 장에서는 볼륨만으로는 충분하지 않은 이유와, 데이터를 안전하게 옮기는 **백업·복원·업그레이드**를 배웁니다. `pg_dump` 로 컨테이너 DB를 호스트 파일로 백업하고, 그 덤프를 새 컨테이너

에 복원하며, 메이저 버전 업그레이드를 dump/restore 절차로 수행하는 방법까지 다룹니다.

백업·복원·업그레이드: 데이터를 안전하게 옮기기

학습 목표 - 볼륨 보존과 별개로 논리 백업이 필요한 이유를 설명할 수 있습니다. - docker exec 와 pg_dump로 컨테이너 DB를 호스트 파일로 백업할 수 있습니다. - 덤프 파일을 새 컨테이너에 복원하고 데이터 보존을 검증할 수 있습니다. - 메이저 버전 업그레이드를 dump/restore 절차로 수행하는 이유와 방법을 이해합니다.

5장에서 우리는 **볼륨(volume)** 으로 데이터를 컨테이너 바깥에 따로 보관하는 법을 배웠습니다. 컨테이너를 지웠다 다시 만들어도 볼륨에 담긴 데이터는 살아남았습니다. 그렇다면 "볼륨만 잘 쓰면 데이터는 안전한 것 아닌가?"라는 의문이 자연스럽게 듭니다.

이 장은 그 의문에 답하면서 시작합니다. 결론부터 말하면, 볼륨은 데이터를 **지금 그 자리에** 유지해 줄 뿐, 실수로 지운 데이터를 되돌려 주거나 다른 PC·다른 버전으로 데이터를 옮겨 주지는 못합니다. 그래서 우리에게서는 별도의 **백업(backup)** 이 필요합니다. 이 장에서는 PostgreSQL의 표준 백업 도구 `pg_dump` 로 컨테이너 안 데이터를 호스트 파일로 떼내고(백업), 그 파일로 새 컨테이너에 데이터를 되살리며(복원), 마지막으로 버전을 16에서 17로 올리는 **메이저 버전 업그레이드(major upgrade)** 까지 한 흐름으로 다룹니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 을 기준으로 합니다. 아래 예시에서 `PS C:\Users\me>` 는 "여러분이 입력하는 자리"를 뜻하고, 그 아래 줄들은 명령이 돌려주는 출력입니다.

용어 미리 보기: **덤프(dump)** 는 데이터베이스 내용을 통째로 한 파일에 쏟아 낸 결과물을 뜻합니다. 이 책에서는 `.sql` 확장자를 가진 텍스트 파일로 만듭니다. **논리 백업(logical backup)** 은 데이터를 "CREATE TABLE", "INSERT" 같은 SQL 명령문 형태로 풀어서 저장하는 방식을 말합니다.

왜 백업이 필요한가: 볼륨만으로 충분치 않은 이유

도입

집 안 금고에 통장을 넣어 두면 도둑으로부터는 안전합니다. 하지만 불이 나면 금고째 타 버립니다. 그래서 사람들은 통장 사본을 따로 은행 대여 금고에도 둡니다. 볼륨은 "집 안 금고"이고, 백업은 "은행 대여 금고"입니다. 이 절에서는 볼륨이 있어도 왜 별도 백업이 필요한지, 그리고 버전이 달라져도 통하는 **논리 백업(dump)**의 가치를 짚습니다.

핵심 개념 설명

백업 (backup)

백업(backup)은 데이터를 원본과 분리된 별도 사본으로 떠 두는 일입니다. 원본에 무슨 일이 생겨도 사본으로 되돌릴 수 있게 하는 안전장치입니다.

- **왜(why) 필요한가:** 볼륨은 데이터를 그 자리에 보존할 뿐, 사람의 실수나 데이터 손상까지 막아 주지는 못합니다. 잘못된 `DELETE` 한 줄로 테이블을 비워 버리면, 볼륨 안의 데이터도 그대로 비워집니다. 볼륨은 "지금 상태"를 지킬 뿐 "예전 상태"를 보관하지 않습니다.
- **언제(when) 쓰나:** 정기적으로(예: 매일), 그리고 위험한 작업(대량 삭제, 스키마 변경, 버전 업그레이드) 직전에 백업을 씁니다.
- **어떻게(how) 쓰나:** PostgreSQL에서는 `pg_dump`로 데이터베이스 내용을 한 파일로 떠내는 것이 가장 기본적인 방법입니다. 다음 절에서 직접 실행합니다.

볼륨과 백업이 막아 주는 위험은 서로 다릅니다. 두 가지 상황을 보겠습니다.

- **상황 1(컨테이너 사고):** 컨테이너를 실수로 `docker rm` 했습니다. 볼륨이 있으면 같은 볼륨을 새 컨테이너에 다시 붙여 그대로 복구됩니다. 이때는 볼륨이 답입니다.
- **상황 2(데이터 사고):** `WHERE` 절을 빠뜨린 `DELETE`로 주문 테이블이 통째로 비었습니다. 볼륨에는 이미 "비어 버린 상태"가 저장되어 있어 도움이 안 됩니다. 어제 떠 둔 백업이 있어야만 되돌릴 수 있습니다.

논리 백업 vs 볼륨 보존

볼륨 보존은 데이터를 **파일(데이터 디렉터리)** 형태 그대로 묶어 두는 방식입니다. 반면 **논리 백업(logical backup)**은 같은 데이터를 SQL 명령문으로 풀어서 저장합니다. 이 차이가 결정적인 이점을 만듭니다.

논리 백업으로 만든 `.sql` 파일은 "테이블을 이렇게 만들고, 이런 행들을 넣어라"라는 지시서입니다. 지시서이기 때문에 **다른 PC, 다른 볼륨, 심지어 상위 버전의 PostgreSQL에서도 그대로 실행**

행할 수 있습니다. 반면 볼륨 안의 데이터 디렉터리는 특정 버전이 쓰던 내부 형식에 묶여 있어, 그대로 다른 버전에 붙이면 동작하지 않습니다(s04에서 자세히 다룹니다).

베스트 프랙티스: 볼륨과 백업은 둘 중 하나를 고르는 것이 아니라 함께 씁니다. 볼륨으로 평소 운영의 안정성을 확보하고, 정기적인 `pg_dump` 백업으로 실수·손상·이전·업그레이드에 대비합니다. "운영은 볼륨, 보험은 백업"으로 기억하면 좋습니다.

주의: 백업 파일을 같은 PC, 같은 디스크에만 두면 디스크가 고장 날 때 원본과 사본이 함께 사라집니다. 백업의 핵심은 "원본과 다른 곳"에 두는 것입니다. 학습 단계에서는 같은 PC에 저장하더라도, 실무에서는 다른 디스크나 클라우드 저장소로 옮겨 두는 습관이 필요합니다.

절 요약

- 볼륨은 데이터를 그 자리에 보존할 뿐, 실수·손상·이전·업그레이드까지 막아 주지는 못합니다.
- 백업은 원본과 분리된 사본으로, 위험 작업 전과 정기적으로 떠 둡니다.
- 논리 백업(`pg_dump`)은 데이터를 SQL 명령문으로 저장해 다른 환경·상위 버전에서도 통합니다.
- 볼륨과 백업은 함께 씁니다("운영은 볼륨, 보험은 백업").

pg_dump로 백업하기

도입

장부 전체를 복사본 노트에 그대로 옮겨 적어 따로 보관하면, 원본을 잃어도 노트만 있으면 다시 만들 수 있습니다. `pg_dump` 가 바로 그 "복사본 노트"를 만들어 주는 도구입니다. 이 절에서는 컨테이너 안에서 `docker exec` 로 `pg_dump` 를 실행해, 데이터베이스 내용을 호스트(내 PC)의 `.sql` 파일로 떠내는 방법을 배웁니다.

핵심 개념 설명

pg_dump (논리 백업 도구)

pg_dump 는 PostgreSQL 데이터베이스의 내용을 SQL 스크립트 형태로 추출해 백업하는 표준 도구입니다. PostgreSQL을 설치하면 함께 들어오며, 공식 postgres 이미지 안에도 이미 포함되

어 있습니다. 따라서 호스트에 PostgreSQL을 따로 깔지 않아도 컨테이너 안의 `pg_dump` 를 그대로 빌려 쓸 수 있습니다.

- **왜(why) 필요한가:** 데이터베이스 전체를 안전하게 한 파일로 떠 두기 위해서입니다. 이 파일 하나면 나중에 같은 데이터를 어디서든 되살릴 수 있습니다.
- **언제(when) 쓰나:** 정기 백업, 위험 작업 직전, 그리고 다른 PC나 상위 버전으로 데이터를 옮길 때 씁니다.
- **어떻게(how) 쓰나:** `pg_dump -U 사용자 데이터베이스이름` 형태로 실행하면, 그 데이터베이스를 되살릴 수 있는 SQL 명령문이 화면(표준 출력)으로 쏟아집니다. 이 출력을 파일로 모아 두면 백업 파일이 됩니다.

연결 메모: PostgreSQL CRUD 교재 12장에서 배운 `pg_dump` 백업이 컨테이너에서도 그대로 통합니다. 차이는 단 하나, 명령 앞에 `docker exec` 컨테이너이름 을 붙여 "컨테이너 안에서" 실행한다는 점뿐입니다. 백업의 본질과 옵션은 네이티브 설치 때와 동일합니다.

docker exec를 통한 백업

7장에서 배운 `docker exec` 는 실행 중인 컨테이너 안에서 명령을 실행하는 도구입니다. 백업은 이 `docker exec` 로 컨테이너 안의 `pg_dump` 를 실행하고, 그 출력을 호스트의 파일로 흘려보내는 방식으로 이루어집니다.

여기서 핵심은 **방향**입니다. `pg_dump` 는 컨테이너 안에서 돌지만, 그 결과물인 `.sql` 파일은 호스트(내 PC)에 저장됩니다. PowerShell의 `>` 기호가 이 "컨테이너의 출력을 호스트 파일로 모으는" 역할을 합니다. 그래서 컨테이너를 나중에 지워도 백업 파일은 호스트에 안전하게 남습니다.

pg_dump — 컨테이너 DB를 호스트 파일로 백업하기



pg_dump 출력을 리다이렉션으로 호스트 파일에 저장 — 컨테이너 삭제 후에도 백업 유지

ch_09_s02 : pg_dump와 docker exec를 통한 백업

이렇게 설정/실행합니다

먼저 백업할 컨테이너가 떠 있어야 합니다. 5장·6장에서 띄운 `pg16` 컨테이너가 `mydb` 데이터베이스를 가지고 있다고 가정하겠습니다. 다음은 그 `mydb` 를 호스트의 `backup.sql` 파일로 백업하는 명령입니다.

```
PS C:\Users\me> docker exec pg16 pg_dump -U postgres mydb > backup.sql
```

이 명령은 화면에 아무것도 출력하지 않고 조용히 끝나는 것이 정상입니다. `pg_dump` 가 내보낸 SQL이 전부 `backup.sql` 파일로 들어갔기 때문입니다. 파일이 잘 만들어졌는지는 다음으로 확인합니다.

```
PS C:\Users\me> Get-Content backup.sql -TotalCount 15
```

예상 화면:


```
--
-- PostgreSQL database dump
--

-- Dumped from database version 16.x
-- Dumped by pg_dump version 16.x

SET statement_timeout = 0;
SET client_encoding = 'UTF8';

CREATE TABLE public.orders (
    id integer NOT NULL,
    customer text,
    amount integer
);
```

CREATE TABLE 과 SET ... 같은 SQL 명령문이 보입니다. 이것이 바로 "데이터베이스를 되살리는 지시서"입니다. 파일 아래쪽에는 각 행을 넣는 COPY 나 INSERT 구문이 이어집니다.

docker exec [컨테이너] pg_dump -U [사용자] [DB이름] > [호스트파일.sql]

명령 구성 요소 설명

요소	의미
docker exec pg16	실행 중인 pg16 컨테이너 안에서 명령을 실행합니다
pg_dump	컨테이너 안에 포함된 PostgreSQL 백업 도구입니다
-U postgres	백업을 수행할 데이터베이스 사용자(여기서는 관리자)입니다
mydb	백업 대상 데이터베이스 이름입니다
>	컨테이너의 출력을 호스트의 파일로 모으는 PowerShell 기호입니다
backup.sql	호스트(내 PC)에 저장되는 백업 파일입니다

팁: 한 컨테이너 안의 모든 데이터베이스를 한꺼번에 백업하려면 pg_dump 대신 pg_dumpall 을 씁니다(docker exec pg16 pg_dumpall -U postgres > all.sql). pg_dumpall 은 사용자·권한 같은 클러스터 전체 정보까지 함께 떼내므로, 서버를 통째로 옮길 때 유용합니다. 특정 데이터베이스 하나만 다룰 때는 pg_dump 가 더 간단합니다.

주의: 파일 이름에 날짜를 넣어 두면 어느 시점의 백업인지 헷갈리지 않습니다. PowerShell에서는 docker exec pg16 pg_dump -U postgres mydb > "backup_\$(Get-Date -Format yyyyMMdd).sql" 처럼

날짜를 붙일 수 있습니다. 같은 이름(`backup.sql`)으로 계속 덮어쓰면, 어제 백업이 오늘 백업에 가려져 사라집니다.

절 요약

- `pg_dump` 는 데이터베이스 내용을 SQL 명령문으로 떠내는 PostgreSQL 표준 백업 도구입니다.
- `docker exec` 컨테이너 `pg_dump -U 사용자 DB > 파일.sql` 로 컨테이너 DB를 호스트 파일로 백업합니다.
- 컨테이너 안에서 실행되지만 결과 파일은 호스트에 남아, 컨테이너를 지워도 백업은 안전합니다.
- PostgreSQL CRUD 교재 12장의 `pg_dump` 가 `docker exec` 경유로 컨테이너에도 동일하게 적용됩니다.

복원하기: 덤프로 데이터 되돌리기

도입

복사해 둔 노트가 있어도, 새 장부에 옮겨 적기 전까지는 쓸 수 없습니다. 복원은 그 "옮겨 적기" 단계입니다. 이 절에서는 앞 절에서 만든 `backup.sql` 을 새 컨테이너의 PostgreSQL에 흘려 넣어 데이터를 되살리고, "백업 → 삭제 → 복원"으로 데이터가 정말 그대로 살아나는지 직접 검증합니다.

핵심 개념 설명

복원 (restore)

복원(restore) 은 백업해 둔 덤프 파일을 데이터베이스에 다시 실행해 테이블과 데이터를 되살리는 작업입니다. 백업이 "지시서를 만드는 일"이라면, 복원은 "그 지시서를 그대로 따라 실행하는 일"입니다.

- **왜(why) 필요한가:** 백업 파일은 그 자체로는 그냥 텍스트일 뿐입니다. 실제로 데이터를 되살리려면 그 SQL을 새 데이터베이스에 실행해야 합니다.
- **언제(when) 쓰나:** 데이터를 잃었을 때, 다른 PC로 옮길 때, 그리고 업그레이드 후 신버전에 데이터를 넣을 때 씁니다.

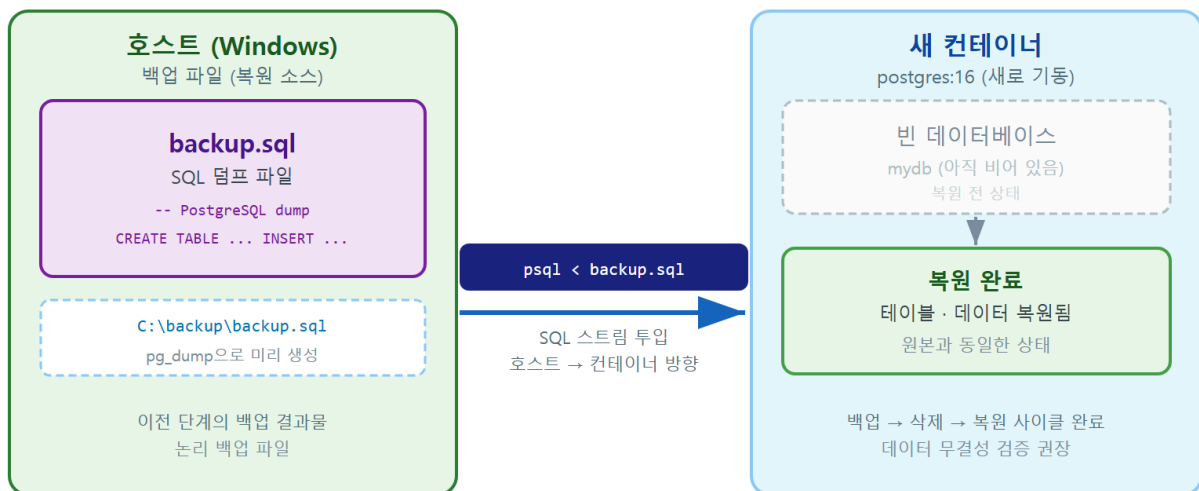
- **어떻게(how) 쓰나:** `psql` 로 백업 파일 속 SQL을 차례대로 실행합니다. 백업 파일이 `.sql` (텍스트) 형식이면 `psql` 로, `pg_dump` 의 아카이브 형식(`-Fc`)이면 `pg_restore` 로 복원합니다. 이 책에서는 가장 단순한 `.sql` + `psql` 조합을 씁니다.

psql로 흘려 넣기

복원의 방향은 백업과 정반대입니다. 백업은 컨테이너 → 호스트였다면, 복원은 **호스트의 .sql 파일 → 컨테이너**입니다. PowerShell의 `<` 기호(또는 `Get-Content ... |`)가 호스트 파일의 내용을 컨테이너 안 `psql` 의 입력으로 흘려보냅니다.

복원에서 입문자가 가장 자주 걸리는 부분은 **대상 데이터베이스가 미리 존재해야 한다**는 점입니다. `.sql` 덤프 안에는 보통 테이블을 만드는 구문은 있지만, 데이터베이스 자체를 만드는 구문은 없을 수 있습니다. 그래서 복원 전에 빈 데이터베이스를 먼저 만들어 두고, 그 안으로 덤프를 흘려 넣는 순서를 따릅니다.

psql 복원 — 호스트 파일을 새 컨테이너로 되돌리기



psql로 덤프 파일을 새 컨테이너에 투입 — 테이블과 데이터가 원본과 동일하게 복원

ch_09_s03 · 복원(restore) — s02 백업과 짝을 이루는 다이어그램

이렇게 설정/실행합니다

"백업 → 삭제 → 복원"을 한 흐름으로 따라가 보겠습니다. 앞 절에서 `backup.sql` 을 이미 떠 두었다고 가정합니다.

먼저 복원을 받을 새 컨테이너를 띄웁니다.

```
PS C:\Users\me> docker run -d --name pg16_new -e POSTGRES_PASSWORD=mysecret postgres:16
```

복원 대상이 될 빈 데이터베이스 `mydb` 를 만듭니다.

```
PS C:\Users\me> docker exec pg16_new createdb -U postgres mydb
```

이제 호스트의 `backup.sql` 을 새 컨테이너의 `mydb` 로 흘려 넣어 복원합니다.

```
PS C:\Users\me> Get-Content backup.sql | docker exec -i pg16_new psql -U postgres -d mydb
```

예상 화면:

```
SET
SET
CREATE TABLE
ALTER TABLE
COPY 3
```

`CREATE TABLE`, `COPY 3` 같은 줄이 차례로 출력됩니다. 각 줄은 백업 파일 속 명령문 하나가 성공적으로 실행되었다는 신호입니다. `COPY 3` 은 행 3개가 들어갔다는 뜻입니다. 마지막으로 데이터가 정말 되살아났는지 직접 확인합니다.

```
PS C:\Users\me> docker exec pg16_new psql -U postgres -d mydb -c "SELECT count(*) FROM orders;"
```

예상 화면:

```
count
-----
      3
(1 row)
```

백업하기 전과 같은 행 수(3)가 나오면 복원 검증 성공입니다. 데이터가 백업 → 새 컨테이너로 그대로 옮겨졌음을 확인한 것입니다.

Get-Content [파일.sql] | docker exec -i [컨테이너] psql -U [사용자] -d [DB이름]

명령 구성 요소 설명

요소	의미
<code>Get-Content backup.sql</code>	호스트의 백업 파일 내용을 읽어 냅니다
<code>\ </code>	읽은 내용을 다음 명령의 입력으로 흘려보냅니다
<code>docker exec -i pg16_new</code>	<code>pg16_new</code> 컨테이너 안에서 입력을 받아(<code>-i</code>) 명령을 실행합니다
<code>psql -U postgres -d mydb</code>	관리자로 <code>mydb</code> 에 접속해 들어온 SQL을 실행합니다
<code>createdb -U postgres mydb</code>	복원 전에 빈 대상 데이터베이스를 만듭니다

주의: `docker exec`에 `-i`(입력 받기) 옵션을 빠뜨리면, 컨테이너 안 `psql`이 흘러 들어오는 파일 내용을 받지 못해 복원이 비어 버립니다. 또한 복원 대상 `mydb`가 이미 같은 이름의 테이블을 가지고 있으면 `relation already exists` 같은 충돌 오류가 납니다. 복원은 비어 있는 깨끗한 데이터베이스에 하는 것이 안전합니다.

팁: 복원이 깔끔하게 끝났는지 확인하려면, 출력 중간에 `ERROR`가 섞여 있지 않은지 살펴봅니다. 한두 줄의 `ERROR`만 있어도 일부 테이블이나 행이 빠질 수 있습니다. 행 수를 세어 보는 `SELECT count(*)` 검증을 백업 직후 값과 비교하는 습관을 들이면, 복원이 온전했는지 한눈에 알 수 있습니다.

절 요약

- 복원은 백업 파일 속 SQL을 데이터베이스에 다시 실행해 데이터를 되살리는 작업입니다.
- `.sql` 텍스트 덤프는 `psql`로, 아카이브(`-Fc`) 덤프는 `pg_restore`로 복원합니다.
- 복원 방향은 호스트 파일 → 컨테이너이며, `docker exec -i`로 입력을 받아야 합니다.
- "백업 → 삭제 → 복원" 후 행 수를 비교해 데이터 보존을 검증합니다.

메이저 버전 업그레이드: dump/restore가 필요한 이유

도입

옛 양식의 장부를 새 양식으로 바꾸려면, 표지만 새것으로 바뀌서는 안 되고 내용을 새 양식에 그대로 옮겨 적어야 합니다. PostgreSQL의 메이저 버전 업그레이드도 똑같습니다. 이미지 태그

를 16에서 17로 바꾸기만 하면 될 것 같지만, 그렇게 하면 컨테이너가 뜨다가 오류를 내며 멈춥니다. 이 절에서는 그 이유와, 안전한 업그레이드 절차인 dump/restore를 배웁니다.

핵심 개념 설명

메이저 버전 업그레이드 (major upgrade)

메이저 버전 업그레이드(major upgrade)는 PostgreSQL의 큰 버전 번호를 올리는 일입니다(예: 16 → 17). 같은 16 안에서 16.2 → 16.3으로 올리는 **마이너 업그레이드**와는 성격이 완전히 다릅니다.

- 마이너 업그레이드(16.2 → 16.3): 데이터 디렉터리 형식이 그대로라, 이미지 태그를 `postgres:16`으로 두고 새로 받아 다시 띄우면 같은 볼륨을 그대로 씁니다.
- 메이저 업그레이드(16 → 17): 데이터 디렉터리의 내부 형식이 바뀌어, 기존 볼륨을 신버전에 그대로 붙일 수 없습니다.

데이터 디렉터리 비호환

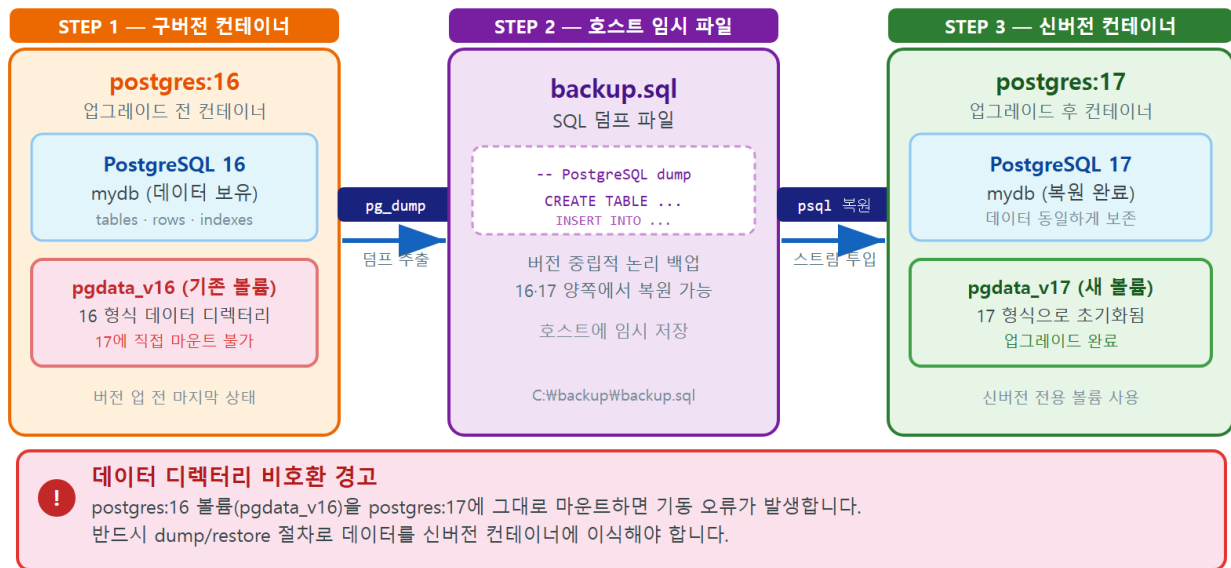
PostgreSQL은 실제 데이터를 `/var/lib/postgresql/data`안의 특정 내부 형식으로 저장합니다(5장에서 본 데이터 디렉터리입니다). 이 형식은 **메이저 버전마다 달라질 수 있습니다**. 그래서 16이 만든 데이터 디렉터리를 17 컨테이너가 그대로 읽으려 하면, "이 데이터 디렉터리는 버전 16용인데 나는 17이다"라며 기동을 거부합니다.

여기서 "이미지 태그만 바꾸면 안 되는 이유"가 분명해집니다. 태그를 `postgres:17`로 바꾸고 **기존 16 볼륨을 그대로 마운트하면**, 컨테이너는 다음과 비슷한 오류를 로그에 남기고 멈춥니다.

```
FATAL: database files are incompatible with server
DETAIL: The data directory was initialized by PostgreSQL version 16,
which is not compatible with this version 17.x.
```

그래서 정답은 **논리 백업을 거치는 것**입니다. s02에서 본 것처럼 `pg_dump` 결과물은 버전 형식에 묶이지 않은 SQL 지시서이므로, 16에서 떠낸 덤프를 17에 복원하면 17이 자기 형식으로 데이터를 새로 만듭니다(`pg_dump`는 상위 버전으로의 복원을 공식 지원합니다). 양식을 갈아타는 유일하게 안전한 길이 " 옮겨 적기", 즉 dump/restore입니다.

메이저 버전 업그레이드: dump/restore 절차 (16→17)



이미지 태그만 변경하지 말 것 — 구버전 dump → 신버전 restore 가 유일한 안전 업그레이드 경로

ch_09_s04 · 메이저 버전 업그레이드(major upgrade) — dump/restore 마이그레이션

이렇게 설정/실행합니다

16 컨테이너의 데이터를 17 컨테이너로 옮기는 업그레이드를 세 단계로 진행합니다. 핵심은 신버전을 반드시 새 볼륨으로 띄운다는 점입니다.

1단계, 구버전(16)에서 데이터를 덤프합니다(s02와 동일).

```
PS C:\Users\me> docker exec pg16 pg_dump -U postgres mydb > upgrade.sql
```

2단계, 신버전(17) 컨테이너를 새 볼륨(pgdata_v17)으로 띄웁니다. 기존 16 볼륨은 건드리지 않습니다.

```
PS C:\Users\me> docker run -d --name pg17 -e POSTGRES_PASSWORD=mysecret -v pgdata_v17:/var/lib/p
```

3단계, 17 컨테이너에 빈 mydb 를 만들고 덤프를 복원합니다(s03와 동일).

```
PS C:\Users\me> docker exec pg17 createdb -U postgres mydb
PS C:\Users\me> Get-Content upgrade.sql | docker exec -i pg17 psql -U postgres -d mydb
```

복원이 끝나면 17 컨테이너에서 데이터와 버전을 함께 확인합니다.

```
PS C:\Users\me> docker exec pg17 psql -U postgres -d mydb -c "SELECT version();"

```

예상 화면:

```

version
-----
PostgreSQL 17.x on x86_64-pc-linux-gnu, compiled by gcc...
(1 row)

```

PostgreSQL 17.x 가 보이고 데이터가 그대로 들어 있으면 메이저 업그레이드가 성공한 것입니다. 16 컨테이너와 그 볼륨은 검증이 끝날 때까지 지우지 않고 남겨 둡니다.

업그레이드 3단계 설명

단계	명령	의미
1	<code>pg_dump ... > upgrade.sql</code>	구버전(16)에서 버전 중립적 덤프를 만듭니다
2	<code>docker run ... -v pgdata_v17:... postgres:17</code>	신버전(17)을 새 볼륨으로 띄웁니다
3	<code>Get-Content upgrade.sql \ docker exec -i pg17 psql ...</code>	덤프를 17에 복원합니다

위험: 메이저 업그레이드에서 가장 위험한 실수는 신버전 컨테이너에 **기존 구버전 볼륨을 그대로 마운트**하는 것입니다. 운이 좋으면 기동만 거부되지만, 잘못된 도구로 강제로 열려다 데이터 디렉터리가 손상될 수 있습니다. 반드시 신버전은 **새 볼륨**으로 띄우고, 데이터는 dump/restore로만 옮깁니다. 구버전 컨테이너와 볼륨은 신버전이 정상임을 검증하기 전까지 절대 지우지 않습니다.

팁: 큰 데이터베이스에서는 `pg_dump` 대신 `pg_upgrade` 라는 전용 도구로 더 빠르게 업그레이드하기도 합니다. 다만 `pg_upgrade` 는 양쪽 버전의 실행 파일을 한 환경에 갖춰야 해 컨테이너 환경에서는 준비가 까다롭습니다. 입문·소규모 단계에서는 이해하기 쉽고 안전한 dump/restore 방식을 권합니다.

절 요약

- 메이저 업그레이드(16 → 17)는 데이터 디렉터리 형식이 달라 태그만 바뀌서는 안 됩니다.
- 기존 볼륨을 신버전에 그대로 붙이면 `incompatible` 오류로 기동이 거부됩니다.

- 안전한 절차는 구버전에서 `pg_dump` → 신버전을 새 볼륨으로 이동 → `psql` 로 복원입니다.
- 검증이 끝나기 전까지 구버전 컨테이너와 볼륨은 보존합니다.

실습 과제

해보기: 백업·복원·업그레이드 한 사이클 수행하기

앞에서 배운 흐름을 직접 손으로 한 바퀴 돌려, 데이터가 안전하게 옮겨지는 것을 두 눈으로 확인합니다. 각 단계의 출력은 메모해 두면 검증에 도움이 됩니다.

- 단계 1(데이터 준비): `pg16` 컨테이너의 `mydb` 에 `docker exec` 로 접속해 작은 테이블 하나를 만들고 행 몇 개를 넣습니다. `SELECT count(*)` 로 행 수를 적어 둡니다.
- 단계 2(백업): `docker exec pg16 pg_dump -U postgres mydb > backup.sql` 로 백업을 뜨고, `Get-Content backup.sql -TotalCount 15` 로 SQL이 들어 있는지 확인합니다.
- 단계 3(삭제): `docker stop pg16` 다음 `docker rm pg16` 으로 컨테이너를 지워, 원본이 사라진 상황을 만듭니다.
- 단계 4(복원): 새 컨테이너 `pg16_new` 를 띄우고 `createdb` 로 빈 `mydb` 를 만든 뒤, `Get-Content backup.sql | docker exec -i pg16_new psql -U postgres -d mydb` 로 복원합니다.
- 단계 5(복원 검증): `SELECT count(*)` 로 행 수가 단계 1과 같은지 비교합니다. 같으면 백업·복원 성공입니다.
- 단계 6(업그레이드): 같은 `backup.sql` 을 `postgres:17` 로 띄운 새 볼륨 컨테이너 `pg17` 에 복원하고, `SELECT version();` 으로 17 버전과 데이터가 함께 있는지 확인합니다.
- 점검: 단계 5와 단계 6에서 행 수가 모두 단계 1과 일치하면, 데이터가 삭제·버전 변경을 거치고도 온전히 살아남은 것입니다. 어느 단계에서 데이터가 위험했고 무엇이 그것을 구했는지 한 줄로 정리해 봅니다.

FAQ

Q1. 볼륨으로 데이터를 영속화했는데도 백업이 또 필요한가요? → **A1.** 필요합니다. 볼륨은 데이터를 "지금 그 상태"로 보존할 뿐, 실수로 지운 데이터를 되돌려 주지 않습니다. 잘못된 `DELETE` 로 테이블이 비면 볼륨에도 "비어 버린 상태"가 그대로 저장됩니다. 또 볼륨은 특정 버전 형식에 묶여 있어 다른 버전으로 옮길 때도 쓸 수 없습니다. 볼륨은 운영의 안정성, 백업은 실수·손상·이전·업그레이드 대비입니다.

Q2. 호스트에 PostgreSQL을 설치하지 않았는데 `pg_dump` 를 쓸 수 있나요? → A2. 있습니다.

`pg_dump` 는 공식 postgres 이미지 안에 이미 포함되어 있습니다. `docker exec` 컨테이너 `pg_dump` ... 로 컨테이너 안의 `pg_dump` 를 그대로 실행하고, PowerShell의 `>` 로 그 출력을 호스트 파일에 모으면 됩니다. 호스트에는 도구를 따로 깔 필요가 없습니다.

Q3. 16에서 뜯 백업을 17 컨테이너에 복원해도 되나요? → A3. 됩니다. `pg_dump` 가 만든 `.sql` 덤프는 데이터를 SQL 명령문으로 풀어 둔 것이라 버전 형식에 묶이지 않습니다. `pg_dump` 는 하위 버전에서 상위 버전으로의 복원을 공식 지원하므로, 16 덤프를 17에 복원하면 17이 자기 형식으로 데이터를 새로 만듭니다. 다만 반대 방향(상위에서 하위로)은 보장되지 않습니다.

Q4. 그냥 이미지 태그를 16에서 17로 바꿔 다시 띄우면 안 되나요? → A4. 안 됩니다. 기존 16 볼륨을 17 컨테이너에 그대로 붙이면 데이터 디렉터리 형식이 호환되지 않아 `database files are incompatible` 오류와 함께 기동이 거부됩니다. 메이저 업그레이드는 반드시 구버전에서 `pg_dump` 로 데이터를 떠내고, 신버전은 새 볼륨으로 띄운 뒤 복원하는 절차를 따라야 합니다.

장 요약

이 장에서는 데이터를 안전하게 지키고 옮기는 세 가지 흐름을 배웠습니다. 먼저 볼륨이 있어도 실수·손상·이전·업그레이드에는 별도 **백업**이 필요하며, 데이터를 SQL 명령문으로 저장하는 **논리 백업**(`pg_dump`) 이 버전과 환경을 넘나드는 가치를 가진다는 것을 이해했습니다. 이어 `docker exec ... pg_dump > backup.sql` 로 컨테이너 DB를 호스트 파일로 백업하고, `Get-Content backup.sql | docker exec -i ... psql` 로 새 컨테이너에 복원한 뒤 행 수를 비교해 데이터 보존을 검증했습니다. 마지막으로 메이저 버전 업그레이드에서는 데이터 디렉터리 형식이 달라 태그만 바뀌서는 안 되며, 구버전 dump → 신버전(새 볼륨) restore가 유일하게 안전한 길임을 확인했습니다. 이로써 우리는 컨테이너 데이터를 잃을 걱정 없이 다룰 수 있게 되었습니다.

핵심 키워드

백업(backup), 논리 백업(logical backup), `pg_dump`, `docker exec`, 복원(restore), `psql`, `pg_restore`, 메이저 버전 업그레이드(major upgrade), 데이터 디렉터리(data directory)

다음 장 미리보기

다음 장에서는 책 전체를 마무리합니다. 비밀번호를 안전하게 다루는 시크릿·`.env` 원칙을 정리하고, `compose` 로 app과 db를 사용자 정의 네트워크로 묶어 서비스 이름으로 통신하게 해 봅니

다. 끝으로 `docker system prune` 으로 리소스를 안전하게 정리하며, 띄우기부터 백업까지 이어진 전체 운영 흐름을 한 바퀴 되짚고 다음 학습 방향을 안내합니다.

실무 패턴과 마무리: 안전 운영과 다음 단계

학습 목표 - 비밀번호·시크릿을 명령·이미지·git에 남기지 않는 원칙을 적용할 수 있습니다. - compose로 app+db 다중 컨테이너를 사용자 정의 네트워크로 연결할 수 있습니다. - docker system prune으로 안전하게 리소스를 정리할 수 있습니다. - 전체 운영 흐름을 복습하고 다음 학습 방향을 정할 수 있습니다.

여기까지 오느라 수고하셨습니다. 우리는 1장에서 컨테이너가 무엇인지부터 시작해, PostgreSQL 컨테이너를 띄우고(3장), 접속하고(4장), 데이터를 영속화하고(5장), 환경변수로 구성하고(6장), 운영 명령으로 들여다보고(7장), compose로 선언하고(8장), 백업·복원·업그레이드(9장)까지 한 줄로 이어 달려왔습니다.

이 마지막 장은 그 여정에 **실무에서 꼭 챙겨야 할 마무리 손질**을 더하는 자리입니다. 비밀번호 같은 민감한 값을 안전하게 다루는 습관, 애플리케이션(app)과 데이터베이스(db)를 한 묶음으로 띄우는 다중 컨테이너 구성, 쌓인 찌꺼기를 안전하게 청소하는 법을 차례로 익힙니다. 그리고 마지막 절에서 전체 운영 흐름을 한 바퀴 되짚고, 이 책 다음에 무엇을 배우면 좋을지 안내합니다.

이 책은 Windows 11의 **PowerShell(파워셸)**을 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, 프롬프트는 `PS C:\Users\me>`처럼 생겼습니다. 아래 예시에서 `PS C:\Users\me>`는 "여러분이 입력하는 자리"를 뜻하고, 그 아래 줄들은 Docker가 돌려주는 출력입니다.

용어 미리 보기: **시크릿(secret)**은 비밀번호처럼 새어 나가면 안 되는 민감한 값입니다. **네트워크(network)**는 컨테이너끼리 서로 대화할 수 있게 이어 주는 통신망이고, **DNS(Domain Name System)**는 이름을 주소로 바꿔 주는 전화번호부 같은 장치입니다.

시크릿과 .env: 비밀번호를 안전하게 다루기

도입

집 열쇠를 현관 매트 밑에 두면 편하지만, 누구나 들출 수 있어 위험합니다. 비밀번호도 마찬가지로입니다. 지금까지 우리는 학습 편의를 위해 `-e POSTGRES_PASSWORD=mysecret` 처럼 비밀번호를 명령에 그대로 적어 왔습니다. 이 절에서는 그 습관을 실무용으로 다듬어, 비밀번호 같은 **시크릿(secret)**을 명령·이미지·git에 남기지 않고 안전하게 분리하는 원칙을 정리합니다.

핵심 개념 설명

시크릿(secret)

시크릿(secret)은 비밀번호나 접속 정보처럼, 노출되면 안 되는 민감한 값입니다. 집 열쇠를 따로 안전한 곳에 보관하듯, 시크릿은 코드나 명령과 섞지 않고 분리해서 다뤄야 합니다.

- **왜(why) 필요한가:** 비밀번호가 한 번 새어 나가면 데이터베이스 전체가 위험해집니다. 그런데 명령·이미지·git 같은 곳에는 우리가 무심코 적은 비밀번호가 생각보다 오래, 그리고 여러 사람이 볼 수 있는 형태로 남습니다.
- **언제(when) 챙기나:** 학습 단계에서도 습관을 들이는 것이 좋습니다. 혼자 쓰는 PC라도, 작성한 파일을 나중에 git에 올리거나 동료와 공유하는 순간 비밀번호가 함께 따라가기 때문입니다.
- **어떻게(how) 다루나:** 비밀번호를 `.env` 파일로 분리하고, 그 파일을 git에 올리지 않도록 `.gitignore`로 막습니다. 6장과 8장에서 맛본 방식을 여기서 원칙으로 굳힙니다.

비밀번호가 남는 대표적인 세 곳을 짚겠습니다.

- **명령(command):** 명령에 직접 적으면 PowerShell 명령 기록(히스토리)에 통째로 남습니다. 같은 PC를 쓰는 다른 사람이 위로 스크롤만 해도 보입니다.
- **이미지(image):** 직접 만든 이미지 안에 비밀번호를 박아 넣으면, 그 이미지를 받은 누구나 비밀번호를 꺼내 볼 수 있습니다.
- **git:** 비밀번호가 적힌 파일을 git에 올리면, 나중에 지워도 과거 기록(커밋 히스토리)에 영원히 남습니다.

환경별 설정 분리

같은 PostgreSQL이라도 내 PC에서 연습할 때와 실제 서비스에 쓸 때는 비밀번호·접속 정보가 달라야 합니다. 이렇게 상황(환경)마다 다른 값을 `.env` 파일로 갈아 끼우는 것을 **환경별 설정 분리**라고 합니다. 구성을 적은 `docker-compose.yml`은 그대로 두고, 값만 담은 `.env`만 바꾸면 같은 파일로 여러 환경을 안전하게 오갈 수 있습니다.

이렇게 설정/실행합니다

먼저 비밀번호를 명령에서 떼어 내 `.env` 파일에 담습니다.

```
# .env (비밀번호 등 민감한 값만 따로 보관 — git에 올리지 않음)
POSTGRES_USER=appuser
POSTGRES_PASSWORD=ChangeMe_2026!
POSTGRES_DB=appdb
```

이 값을 `docker-compose.yml` 에서 `${...}` 형태로 참조합니다. compose는 같은 폴더의 `.env` 를 자동으로 읽어 빈칸을 채웁니다.

```
# docker-compose.yml (값은 .env에서 가져오므로 비밀번호가 파일에 없음)
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

마지막으로 `.env` 가 git에 따라 올라가지 않도록 `.gitignore` 에 한 줄을 추가합니다.

```
# .gitignore (git이 무시할 파일 목록)
.env
```

설정이 끝났다면, compose가 `.env` 값을 제대로 읽어 넣었는지 `docker compose config` 로 확인합니다. 이 명령은 실제로 컨테이너를 띄우지 않고, 최종 구성만 펼쳐 보여 줍니다.

```
PS C:\Users\me> docker compose config
```

예상 화면:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: ChangeMe_2026!
      POSTGRES_DB: appdb
    ports:
      - "5432:5432"
  ...
```

`${POSTGRES_PASSWORD}` 이던 자리가 `.env` 의 실제 값으로 채워져 보이면 연결이 정상입니다. 이 펼친 결과는 화면에만 보여 줄 뿐, `docker-compose.yml` 파일 자체에는 여전히 비밀번호가 없습니다.

설정 키와 명령 설명

키 / 명령	의미
<code>.env</code> 의 <code>POSTGRES_PASSWORD</code>	비밀번호 실제 값을 담는 곳(이 파일만 git 제외)
<code>\${POSTGRES_PASSWORD}</code>	compose가 <code>.env</code> 에서 같은 이름 값을 끌어와 채우는 자리
<code>.gitignore</code> 의 <code>.env</code>	git이 <code>.env</code> 를 추적·업로드하지 않게 막는 줄
<code>docker compose config</code>	컨테이너를 띄우지 않고 최종 구성을 펼쳐 확인

위험: 비밀번호가 적힌 파일을 실수로 git에 한 번이라도 올리면, 그 뒤에 파일을 지워도 과거 커밋에 그대로 남습니다. 이런 경우 비밀번호가 이미 새어 나갔다고 보고 **비밀번호 자체를 바꾸는** 것이 정답입니다. 그래서 처음부터 `.gitignore` 로 막는 습관이 중요합니다.

팁: 동료에게 "이런 값들이 필요하다"고 알려 주고 싶을 때는, 실제 비밀번호 대신 빈 견본을 담은 `.env.example` 파일을 만들어 git에 올립니다. `POSTGRES_PASSWORD=` 처럼 키 이름만 적고 값은 비워 두면, 받는 사람이 자기 값을 채워 `.env` 로 복사해 쓰면 됩니다.

절 요약

- 시크릿(비밀번호 등)은 명령·이미지·git 세 곳에 남기지 않는 것이 원칙입니다.
- 비밀번호는 `.env` 에 담고, `docker-compose.yml` 에서는 `${...}` 로 참조합니다.
- `.gitignore` 에 `.env` 를 넣어 git 업로드를 막습니다.
- `docker compose config` 로 값이 제대로 채워졌는지 안전하게 확인할 수 있습니다.

app + db 다중 컨테이너와 사용자 정의 네트워크 맛보기

도입

지금까지는 PostgreSQL 컨테이너 하나만 다뤘습니다. 하지만 실제 서비스는 보통 **여러 컨테이너가 함께** 돕니다. 데이터를 보관하는 데이터베이스(db)와, 그 데이터를 쓰는 애플리케이션(app)이 짝을 이루는 식입니다. 이 절에서는 compose에 app 서비스를 하나 더해 db와 같은 묶음으로 띄우고, 두 컨테이너가 서비스 이름만으로 서로 대화하는 **사용자 정의 네트워크(user-defined network)**를 맛봅니다.

핵심 개념 설명

다중 컨테이너 (multi-container)

다중 컨테이너(multi-container)는 역할이 다른 여러 컨테이너를 한 묶음으로 함께 띄워 운영하는 구성입니다. 데이터베이스, 애플리케이션 서버, 캐시 등을 각각 따로 떼어 두고 서로 협력하게 만드는 방식입니다.

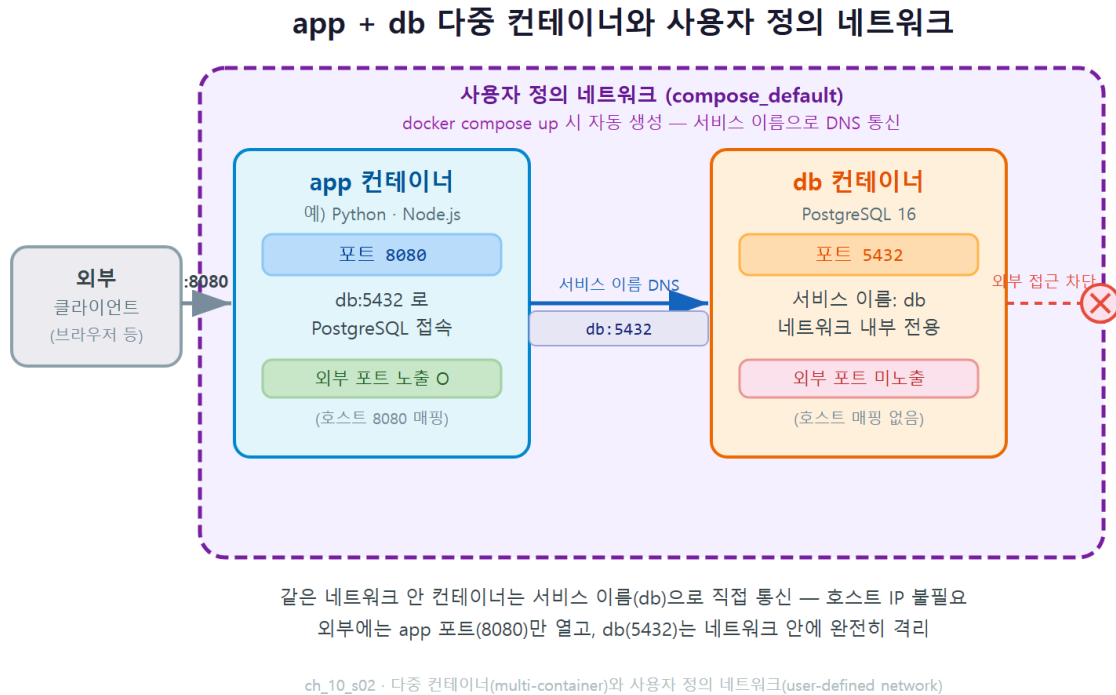
- **왜(why) 필요한가:** 역할마다 컨테이너를 나누면 한쪽만 따로 끄고 켜거나 버전을 바꾸기 쉽고, 문제가 생긴 부분만 떼어 살펴볼 수 있습니다.
- **언제(when) 쓰나:** 데이터베이스에 붙어 동작하는 애플리케이션을 함께 돌릴 때입니다. compose의 `services:` 아래에 서비스를 나란히 적으면 한 번에 띄울 수 있습니다.
- **어떻게(how) 쓰나:** 8장에서 만든 `db` 서비스 옆에 `app` 서비스를 추가합니다. `docker compose up` 한 번이면 둘이 같이 뜹니다.

사용자 정의 네트워크 (user-defined network)

여기가 이 절의 핵심입니다. compose로 여러 서비스를 띄우면, Docker가 그 묶음만을 위한 **전용 네트워크**를 자동으로 하나 만들어 줍니다. 이것이 **사용자 정의 네트워크(user-defined network)**입니다. 같은 사내 전화망에 속한 부서끼리 내선 이름만으로 연결되는 것과 같습니다.

이 네트워크 안에서는 컨테이너가 **서비스 이름**을 주소처럼 써서 서로 찾아갑니다. app이 db에 접속할 때 호스트 IP나 매핑한 포트(`localhost:5432`)를 쓸 필요가 없습니다. 그냥 `db`라는 이름을 쓰면 Docker의 내부 DNS(이름을 주소로 바꿔 주는 전화번호부)가 알아서 db 컨테이너로 안내합니다.

입문자가 가장 많이 헛갈리는 지점이 바로 이 **접속 주소**입니다. app 컨테이너 안에서 db에 접속할 때 `localhost` 를 쓰면 안 됩니다. 컨테이너 안의 `localhost` 는 자기 자신을 가리키기 때문입니다. db에 닿으려면 서비스 이름인 `db` 를 호스트 이름으로 써야 합니다.



이렇게 설정/실행합니다

8장의 compose 파일에 `app` 서비스를 추가합니다. 여기서 app은 설명을 위한 간단한 예시로, postgres 클라이언트 도구가 들어 있는 가벼운 이미지를 써서 db에 접속만 시도해 봅니다.

```
# docker-compose.yml (db 옆에 app 서비스를 추가)
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - pgdata:/var/lib/postgresql/data

  app:
    image: postgres:16
    depends_on:
      - db
    environment:
      PGPASSWORD: ${POSTGRES_PASSWORD}
    command: >
      psql -h db -U ${POSTGRES_USER} -d ${POSTGRES_DB}
      -c "SELECT 'app에서 db로 접속 성공' AS message;"

volumes:
  pgdata:
```

눈여겨볼 곳은 app의 `command` 에 들어간 `-h db` 부분입니다. 접속 대상을 IP나 `localhost` 가 아니라 **서비스 이름** `db` 로 적었습니다. 또 `depends_on: [db]` 는 db 서비스를 app보다 먼저 띄우라는 뜻입니다. 이제 한 번에 띄웁니다.

```
PS C:\Users\me> docker compose up
```

예상 화면:

```
[+] Running 2/2
 - Container me-db-1   Created
 - Container me-app-1  Created
me-db-1 | ... database system is ready to accept connections
me-app-1 |          message
me-app-1 | -----
me-app-1 | app에서 db로 접속 성공
me-app-1 | (1 row)
me-app-1 exited with code 0
```

app에서 db로 접속 성공 이라는 줄이 보이면, app 컨테이너가 호스트 IP가 아니라 서비스 이름 `db` 만으로 PostgreSQL을 찾아 접속에 성공한 것입니다. db는 외부에 포트를 열지 않았는데도 (여기서는 `ports` 를 db에 두지 않았습니다) app은 같은 네트워크 안에 있으므로 문제없이 닿습니다.

psql -h <서비스이름> -U <사용자> -d <데이터베이스>

설정 키 설명

키	의미
services: 아래 db, app	한 묶음으로 함께 뜨는 두 컨테이너(다중 컨테이너)
depends_on: [db]	db를 app보다 먼저 띄우도록 순서를 지정
-h db	접속 대상 호스트를 서비스 이름 db로 지정(IP 불필요)
PGPASSWORD	psql이 비밀번호를 물어보지 않게 미리 알려 주는 환경변수

주의: depends_on 은 db 컨테이너가 먼저 시작되도록 순서만 맞춰 줄 뿐, PostgreSQL이 접속을 받을 준비를 마칠 때까지 기다려 주지는 않습니다. db가 기동되는 몇 초 사이에 app이 먼저 접속을 시도하면 실패할 수 있습니다. 실무에서는 7장에서 배운 헬스체크(healthcheck)와 depends_on 의 condition: service_healthy 를 함께 써서 "db가 준비된 뒤에" app을 띄우도록 보강합니다.

팁: db에 외부 포트(ports)를 열지 않아도 같은 네트워크의 app은 접속할 수 있습니다. 오히려 데이터베이스는 외부에 포트를 노출하지 않고 app만 포트를 여는 것이 더 안전합니다. 내 PC의 DBeaver로 직접 들여다보아야 할 때만 db에 ports 를 잠깐 열어 두는 식으로 운영하면 좋습니다.

절 요약

- compose의 services: 아래에 app과 db를 나란히 적으면 다중 컨테이너로 함께 뜹니다.
- compose는 묶음 전용 사용자 정의 네트워크를 자동으로 만들어 줍니다.
- 같은 네트워크의 컨테이너는 서비스 이름(db)으로 서로 접속합니다(localhost 아님).
- db는 외부 포트를 열지 않아도 같은 네트워크의 app은 닿을 수 있습니다.

정리와 디스크 관리: docker system prune

도입

실습을 반복하다 보면 멈춘 컨테이너, 안 쓰는 이미지, 어디에도 연결되지 않은 볼륨이 디스크 한구석에 차곡차곡 쌓입니다. 안 쓰는 짐을 주기적으로 비워 창고를 되찾듯, Docker도 정리가

필요합니다. 이 절에서는 쌓인 리소스를 점검하고 `docker system prune` 으로 한 번에 청소하는 법, 그리고 이 강력한 명령을 쓸 때의 위험을 함께 다룹니다.

핵심 개념 설명

리소스 정리 (cleanup)

리소스 정리(cleanup) 는 멈춘 컨테이너·미사용 이미지·댕글링 볼륨처럼 더 이상 쓰지 않는 항목을 지워 디스크 공간을 회수하는 작업입니다. 여기서 **댕글링(dangling)** 은 "어디에도 매달려 있지 않은", 즉 이름표를 잃거나 아무 컨테이너도 쓰지 않는 상태를 가리킵니다.

- **왜(why) 필요한가:** 이미지 한 개가 수백 MB이므로, 정리하지 않으면 디스크가 금세 가득 찹니다. Windows에서는 WSL2가 쓰는 가상 디스크가 계속 커져 PC 용량을 압박하기도 합니다.
- **언제(when) 하나:** 실습을 한참 한 뒤, 또는 디스크가 부족하다는 경고가 뜰 때입니다.
- **어떻게(how) 하나:** 먼저 무엇이 쌓여 있는지 본 다음, 필요 없는 것만 골라 지웁니다. 한꺼번에 지우는 `docker system prune` 은 편하지만 위험하므로 마지막에 신중히 씁니다.

docker system prune

docker system prune 은 멈춘 컨테이너, 어떤 컨테이너도 쓰지 않는 네트워크, 댕글링 이미지, 빌드 캐시를 한 번에 정리하는 명령입니다. 강력한 만큼, 무엇이 지워지는지 정확히 알고 써야 합니다.

가장 조심할 점은 **볼륨**입니다. 기본 `docker system prune` 은 볼륨을 건드리지 않지만, 여기에 `--volumes` 옵션을 더하면 **아무 컨테이너도 쓰지 않는 볼륨까지** 지웁니다. 이때 잠깐 멈춰 둔 PostgreSQL의 데이터 볼륨이 "지금은 안 쓰는 볼륨"으로 분류되어 통째로 날아갈 수 있습니다. 5장에서 어렵게 영속화한 데이터가 한순간에 사라지는 가장 흔한 사고가 바로 이것입니다.

이렇게 설정/실행합니다

청소 전에는 반드시 무엇이 쌓여 있는지부터 봅니다. `docker system df` 는 컨테이너·이미지·볼륨이 차지하는 용량을 요약해 보여 줍니다.

```
PS C:\Users\me> docker system df
```

예상 화면:

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	5	1	1.8GB	1.4GB (77%)
Containers	3	1	12MB	11MB (90%)
Local Volumes	4	1	320MB	240MB (75%)
Build Cache	20	0	150MB	150MB

RECLAIMABLE (회수 가능)이 정리하면 되찾을 수 있는 용량입니다. 이제 안전하게 정리합니다. 기본 `prune` 은 실행 전에 정말 지울지 한 번 물어봅니다.

```
PS C:\Users\me> docker system prune
```

예상 화면:

```
WARNING! This will remove:
- all stopped containers
- all networks not used by at least one container
- all dangling images
- all dangling build cache

Are you sure you want to continue? [y/N]
```

여기서 무엇이 지워지는지 목록을 꼭 읽고, 지워도 되는 것뿐이라면 `y` 를 입력합니다. 목록에 **볼륨이 없다는 점**에 주목합니다. 기본 `prune` 은 데이터 볼륨을 건드리지 않으므로 PostgreSQL 데이터는 안전합니다.

```
Deleted Containers:
3f8a1c5d9e2b...

Deleted Images:
untagged: postgres:15

Total reclaimed space: 1.4GB
```

Total reclaimed space 만큼 디스크를 되찾았습니다.

명령과 옵션 설명

명령 / 옵션	의미
<code>docker system df</code>	컨테이너·이미지·볼륨이 쓰는 용량과 회수 가능량 요약
<code>docker system prune</code>	멈춘 컨테이너·미사용 네트워크·댕글링 이미지 정리(볼륨 제외)
<code>docker system prune -a</code>	위에 더해, 쓰지 않는 모든 이미지 까지 삭제(다시 받아야 함)
<code>docker system prune --volumes</code>	미사용 볼륨까지 삭제 — 데이터 유실 위험

위험: `docker system prune --volumes` 는 지금 어떤 컨테이너도 쓰지 않는 볼륨을 모두 지웁니다. 잠깐 컨테이너를 내려 둔 PostgreSQL의 데이터 볼륨도 여기에 휩쓸려 영구히 사라질 수 있습니다. 데이터가 든 볼륨을 살려야 한다면, 이 옵션을 쓰기 전에 9장에서 배운 `pg_dump` 로 백업을 한번 더 두십시오. 입문 단계에서는 `--volumes` 를 습관처럼 붙이지 않는 것이 안전합니다.

팁: 한꺼번에 지우는 `prune` 이 부담스럽다면, 대상을 나눠 정리할 수 있습니다. 멈춘 컨테이너만 지우려면 `docker container prune`, 미사용 이미지만 지우려면 `docker image prune` 을 씁니다. 무엇이 지워질지 더 좁게 통제할 수 있어 사고를 줄여 줍니다.

절 요약

- 청소 전에 `docker system df` 로 무엇이 얼마나 쌓였는지 먼저 확인합니다.
- `docker system prune` 은 멈춘 컨테이너·미사용 네트워크·댕글링 이미지를 정리합니다(볼륨 제외).
- `--volumes` 옵션은 데이터 볼륨까지 지울 수 있어 매우 위험합니다.
- 대상을 좁혀 지우려면 `docker container prune` · `docker image prune` 을 활용합니다.

마무리: 전체 운영 흐름 복습과 다음 단계

도입

이제 책의 마지막 절입니다. 재료 준비부터 조리·보관·뒷정리까지 요리 전 과정을 한 장의 순서도로 되짚듯, 우리가 1장부터 걸어온 PostgreSQL 컨테이너 운영의 전체 여정을 한 흐름으로 정

리합니다. 그리고 이 책 다음에 어디로 더 나아가면 좋을지 학습 방향을 안내합니다.

핵심 개념 설명

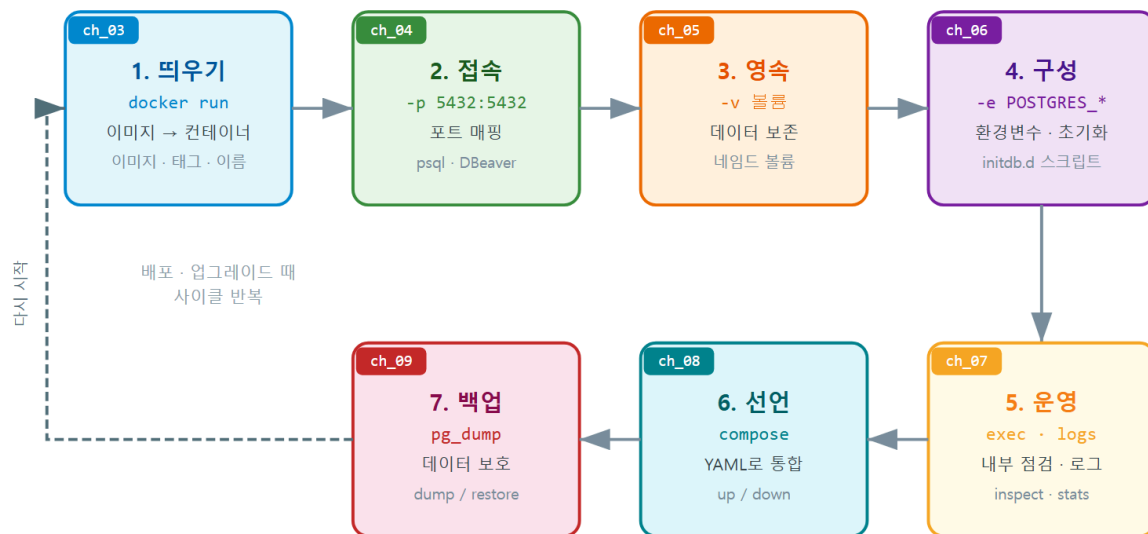
운영 흐름 종합 (operations recap)

운영 흐름 종합(operations recap) 은 이미지로 컨테이너를 띄워 접속·영속·구성·운영·선언·백업 까지 이어지는 전체 여정을 한눈에 정리한 것입니다. 명령을 하나하나 따로 외우는 것과, 그 명령들이 전체 흐름의 어디에 놓이는지 아는 것은 전혀 다릅니다. 후자가 곧 "운영을 이해했다"는 의미입니다.

이 책의 여정을 한 줄로 이으면 이렇게 됩니다.

- **띄우기(1~3장)**: 컨테이너·이미지 개념을 익히고, `docker pull` 로 이미지를 받아 `docker run` 으로 컨테이너를 띄웠습니다.
- **접속(4장)**: `-p` 포트 매핑으로 격리된 컨테이너 속 PostgreSQL에 호스트의 `psql`·`DBaver` 로 닿았습니다.
- **영속(5장)**: 네임드 볼륨을 `/var/lib/postgresql/data` 에 마운트해, 컨테이너를 지워도 데이터가 살아남게 했습니다.
- **구성(6장)**: 환경변수와 초기화 스크립트로 컨테이너가 뜨자마자 준비된 DB가 되도록 구성했습니다.
- **운영(7장)**: `docker exec` · `logs` · `inspect` 로 컨테이너 안을 들여다보고 점검했습니다.
- **선언(8장)**: 길어진 명령을 `docker-compose.yml` 한 파일로 선언해 `up` · `down` 으로 통째로 다뤘습니다.
- **백업(9장)**: `pg_dump` 로 논리 백업을 뜨고 복원하며, 메이저 버전 업그레이드까지 익혔습니다.
- **마무리(10장)**: 시크릿을 안전하게 다루고, `app+db` 다중 컨테이너를 묶고, 리소스를 안전하게 정리했습니다.

PostgreSQL 컨테이너 전체 운영 흐름 종합



띄우기(ch_03) → 접속(ch_04) → 영속(ch_05) → 구성(ch_06) → 운영(ch_07) → 선언(ch_08) → 백업(ch_09)
 전체 여정을 한 바퀴 돌아 안전하게 운영하고, 배포·업그레이드 시 사이클을 반복합니다.

ch_10_s04 · 운영 흐름 종합(operations recap) — 전체 7단계 한 바퀴

다음 학습 (next steps)

이 책은 "내 PC에서 PostgreSQL 컨테이너를 안전하게 다루는" 기초까지를 목표로 했습니다. 여기서 한 걸음 더 나아가고 싶다면 다음 방향을 권합니다.

- **오케스트레이션(orchestration)**: 컨테이너가 여러 대의 서버에 걸쳐 많아지면, 이를 자동으로 배치·복구·확장해 주는 도구가 필요합니다. 대표적으로 **Kubernetes(쿠버네티스)**가 있습니다.
- **운영 자동화**: 백업을 사람이 매번 손으로 뜨는 대신, 정해진 시각에 자동으로 실행되게 하는 스케줄링과, 컨테이너가 죽으면 자동으로 다시 띄우는 재시작 정책(`restart` 옵션)을 익힙니다.
- **이미지 직접 만들기**: `Dockerfile`을 작성해 우리 애플리케이션 전용 이미지를 직접 빌드하는 법을 배우면, app 서비스를 진짜 우리 코드로 채울 수 있습니다.
- **공식 문서**: 가장 정확하고 최신인 자료는 언제나 공식 문서입니다. Docker 공식 문서와 PostgreSQL 공식 이미지 문서를 곁에 두고 참고하는 습관을 들이면 좋습니다.

이렇게 설정/실행합니다

마무리로, 이 책에서 만든 전체 환경을 `compose` 한 파일로 정리한 모습을 다시 봅니다. 앞 절들에서 익힌 시크릿 분리·다중 컨테이너·볼륨 영속이 한곳에 모여 있습니다.

```
# docker-compose.yml (이 책의 운영 요소를 한곳에 정리한 최종 구성)
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: unless-stopped

  app:
    image: postgres:16
    depends_on:
      - db
    environment:
      PGPASSWORD: ${POSTGRES_PASSWORD}
    command: >
      psql -h db -U ${POSTGRES_USER} -d ${POSTGRES_DB}
      -c "SELECT 'stack ready' AS status;"

volumes:
  pgdata:
```

띄우고, 백업을 한 번 떠 두고, 정리하는 한 사이클을 명령으로 이으면 다음과 같습니다.

```
PS C:\Users\me> docker compose up -d
PS C:\Users\me> docker exec db pg_dump -U appuser appdb > backup.sql
PS C:\Users\me> docker compose down
PS C:\Users\me> docker system prune
```

예상 화면(요약):

```
[+] Running 2/2
 - Container me-db-1   Started
 - Container me-app-1 Started
...
[+] Running 2/2
 - Container me-app-1 Removed
 - Container me-db-1  Removed
Total reclaimed space: 320MB
```

띄우고(`up -d`), 안전하게 백업을 떠 두고(`pg_dump`), 깔끔히 내리고(`down`), 찌꺼기를 정리(`prune`) 하는 이 한 사이클이 곧 이 책이 가르친 운영의 축소판입니다. `down` 은 볼륨을 지우지 않으므로 `pgdata` 는 그대로 남아, 다음에 `up` 하면 데이터가 살아 있습니다.

전체 흐름 명령 설명

명령	운영 단계	의미
<code>docker compose up -d</code>	띄우기·선언	선언한 db-app을 백그라운드로 함께 기동
<code>docker exec db pg_dump</code> ...	백업	실행 중인 db의 데이터를 호스트 파일로 덤프
<code>docker compose down</code>	내리기	컨테이너·네트워크 제거(볼륨 pgdata 는 유지)
<code>docker system prune</code>	정리	멈춘 컨테이너·미사용 이미지 정리(볼륨 제외)

베스트 프랙티스: 운영의 핵심은 "데이터를 잃지 않는 것"입니다. 컨테이너는 언제든지 지우고 다시 만들 수 있는 일회용으로 여기되, 데이터는 볼륨으로 영속화하고 정기적으로 `pg_dump` 백업을 떠 두십시오. 무언가를 지우는 명령(`down -v`, `prune --volumes`)을 실행하기 전에는 항상 "이 작업으로 어떤 데이터가 사라지는가"를 한 번 스스로 묻는 습관이 가장 든든한 안전장치입니다.

절 요약

- 이 책의 여정은 띄우기→접속→영속→구성→운영→선언→백업→마무리로 이어집니다.
- 컨테이너는 일회용으로, 데이터는 볼륨과 백업으로 지키는 것이 운영의 핵심입니다.
- 다음 단계로 오케스트레이션(Kubernetes)·운영 자동화·Dockerfile·공식 문서를 권합니다.
- `up` → `pg_dump` → `down` → `prune` 한 사이클이 이 책이 가르친 운영의 축소판입니다.

실습 과제

해보기: app+db 미니 스택 구성하고 정리까지 마무리하기

이 책의 마무리로, 앞에서 익힌 시크릿 분리·다중 컨테이너·백업·정리를 한 번에 이어 직접 수행해 봅니다. 각 단계의 출력은 메모해 두면 좋습니다.

- 단계 1: `.env` 에 `POSTGRES_USER` · `POSTGRES_PASSWORD` · `POSTGRES_DB` 를 적고, `.gitignore` 에 `.env` 를 추가합니다.
- 단계 2: `docker-compose.yml` 에 db 서비스와, db에 `-h db` 로 접속하는 app 서비스를 함께 작성합니다(비밀번호는 `${...}` 로 참조).

- 단계 3: `docker compose config` 로 `.env` 값이 제대로 채워졌는지 확인합니다.
- 단계 4: `docker compose up` 으로 띄우고, app 로그에서 db 접속 성공 메시지가 나오는지 확인합니다.
- 단계 5: `docker exec db pg_dump -U <사용자> <DB> > backup.sql` 로 백업을 한 번 떠 둡니다.
- 단계 6: `docker compose down` 으로 내린 뒤, `docker system prune` 으로 미사용 리소스를 안전하게 정리합니다(`--volumes` 는 붙이지 않습니다).
- 점검: `docker volume ls` 에 `pgdata` 볼륨이 그대로 남아 있고, 다시 `docker compose up -d` 했을 때 데이터가 살아 있으면, 컨테이너는 정리하면서도 데이터는 지킨 안전한 운영 사이클을 완성한 것입니다.

FAQ

Q1. app 컨테이너에서 db에 접속할 때 왜 `localhost` 를 쓰면 안 되나요? → A1. 컨테이너 안에서 `localhost` 는 그 컨테이너 자기 자신을 가리키기 때문입니다. app 안에서 `localhost:5432` 로 접속하면 app 컨테이너 안에서 PostgreSQL을 찾게 되는데, 거기엔 PostgreSQL이 없습니다. 같은 compose 네트워크에서는 서비스 이름인 `db` 를 호스트로 써야 Docker 내부 DNS가 db 컨테이너로 안내합니다.

Q2. `.env` 에 비밀번호를 넣으면 정말 안전한가요? → A2. `.env` 는 비밀번호를 명령·코드와 분리해 두는 첫걸음일 뿐, 그 자체가 암호화되는 것은 아닙니다. 핵심은 `.gitignore` 로 `.env` 를 git에 올리지 않아 외부로 새는 경로를 막는 것입니다. 더 엄격한 운영 환경에서는 Docker Secrets나 전용 비밀 관리 도구를 쓰지만, 입문·학습 단계에서는 `.env` + `.gitignore` 습관만으로도 큰 위험을 줄일 수 있습니다.

Q3. `docker system prune` 을 실행하면 데이터가 사라지나요? → A3. 기본 `docker system prune` 은 멈춘 컨테이너·미사용 네트워크·댕글링 이미지만 지우고 **볼륨은 건드리지 않습니다**. 따라서 PostgreSQL 데이터 볼륨은 안전합니다. 위험한 것은 `--volumes` 옵션을 붙였을 때입니다. 이 경우 어떤 컨테이너도 쓰지 않는 볼륨까지 지워져 데이터가 사라질 수 있으므로, 입문 단계에서는 `--volumes` 를 붙이지 않는 것이 안전합니다.

Q4. 이 책을 끝낸 뒤에는 무엇을 배우면 좋을까요? → A4. 먼저 `Dockerfile` 로 우리 애플리케이션 전용 이미지를 직접 만드는 법을 익히면, 예시로만 다룬 app 서비스를 실제 코드로 채울 수 있습니다. 그다음 컨테이너가 많아질 때를 대비한 오케스트레이션 도구(Kubernetes)와, 백업·재시작을 자동화하는 운영 기법으로 넓혀 가면 좋습니다. 무엇을 배우든 Docker와 PostgreSQL의 공식 문서를 곁에 두는 습관이 가장 큰 도움이 됩니다.

장 요약

이 마지막 장에서는 PostgreSQL 컨테이너 운영을 실무 수준으로 다듬는 마무리를 다뤘습니다. 비밀번호 같은 시크릿을 `.env` 로 분리하고 `.gitignore` 로 막아 명령·이미지·git에 남기지 않는 원칙을 세웠습니다. `compose`에 `app` 서비스를 더해 `db`와 같은 사용자 정의 네트워크에 묶고, `localhost`가 아니라 서비스 이름 `db`로 통신하는 다중 컨테이너의 핵심을 익혔습니다. `docker system prune`으로 쌓인 리소스를 안전하게 정리하되, `--volumes`가 데이터 볼륨까지 지우는 위험을 분명히 구분했습니다. 끝으로 띄우기→접속→영속→구성→운영→선언→백업→마무리로 이어진 책 전체의 운영 여정을 한 흐름으로 되짚고, 오케스트레이션·운영 자동화·`Dockerfile`·공식 문서라는 다음 학습 방향을 제시했습니다. 컨테이너는 일회용으로 여기되 데이터는 볼륨과 백업으로 지킨다는 한 문장이, 이 책이 남기는 가장 중요한 운영 원칙입니다.

핵심 키워드

시크릿(secret), `.env`, `.gitignore`, 다중 컨테이너(multi-container), 사용자 정의 네트워크(user-defined network), 서비스 이름, `docker system prune`, 리소스 정리(cleanup), 운영 흐름 종합(operations recap)

다음 단계

이로써 "Docker 핸즈온: PostgreSQL로 익히는 컨테이너 운영" 여정을 마칩니다. 여러분은 이제 내 PC에서 PostgreSQL 컨테이너를 띄우고, 접속하고, 데이터를 지키고, 구성·운영·백업하고, 안전하게 정리하는 한 바퀴를 스스로 돌릴 수 있습니다. 여기서 멈추지 말고, `Dockerfile`로 직접 이미지를 만들고 오케스트레이션(Kubernetes)으로 넓혀 가며, 공식 문서를 곁에 두고 다음 한 걸음을 이어 가시길 바랍니다. 수고하셨습니다.